
SH2SC-EDT and C2P-ARQ: A Self-Healing Hardware and Software Complex Utilizing the ChaCha20-Poly1305 Automatic Repeat reQuest Protocol

Oleksandr Dreval

Project's GitHub Repository

github.com/OleksandrDreval/sh2sc-edt

Project Documentation Site

oleksandrdreval.github.io/sh2sc-edt

E-mail

drevaloleksandr@gmail.com

GitHub Profile

github.com/OleksandrDreval

LinkedIn

linkedin.com/in/oleksandr-dreval

March 2026

Abstract

Commercial off-the-shelf (COTS) IoT communication modules operating over unshielded physical channels are vulnerable to active jamming and provide no integrated authenticated encryption at the transport layer, leaving session state irrecoverably lost upon link disruption. This paper introduces SH2SC-EDT (Self-Healing Hardware and Software Complex for Encrypted Data Transmission) and its transport engine, the C2P-ARQ (ChaCha-Poly Automatic Repeat reQuest) protocol. The architecture is founded on strictly deterministic, non-blocking finite state machines and employs ChaCha20-Poly1305 AEAD for per-packet authenticated encryption, with session nonces derived from a hardware TRNG seeded by multiple physical entropy sources. Protocol correctness is stressed via a stateful, time-dependent data stream whose variable inter-packet timing simultaneously exercises the Smart Dynamic Watchdog adaptive timeout and the Auto-Resume mechanism — properties that a static or homogeneous payload cannot validate. Hardware simulation on ATmega328P (16 MHz, 2 KB SRAM) in SimulIDE demonstrates that C2P-ARQ recovers desynchronised sessions after sustained jamming of arbitrary duration without data loss and without operator intervention, confirming the self-healing property at the application layer.

Table of Contents

1	Вступ	7
1.1	Ландшафт уразливостей COTS RF-модулів	7
1.1.1	Атаки повторного відтворення	7
1.1.2	Ін'єкція відкритого тексту	8
1.1.3	Десинхронізація стану сеансу	8
1.2	Загроза глушіння та втрата синхронізації	8
1.3	Наш внесок: архітектура SH2SC-EDT	9
1.3.1	Інтегроване автентифіковане шифрування для кожного пакета	10
1.3.2	Строго неблокувальні скінченні автомати	10
1.3.3	Самовідновний ARQ з адаптивним сторожовим таймером	11
2	Архітектура системи та концептуальний дизайн	12
2.1	Доказ концепції через корисне навантаження: обґрунтування передавання мелодії	12
2.1.1	Часова залежність	12
2.1.2	Станозалежний потік даних	13
2.1.3	Акустична діагностика	13
2.1.4	Зворотна масштабованість	13
2.2	Парадигма "товстий сервер / тонкий клієнт"	14
2.3	Детермінована поведінка через скінченні автомати (FSM)	15
3	Апаратна архітектура та середовище симуляції	17
3.1	Тестовий стенд SimulIDE: топологія та абстракція фізичного рівня	17
3.2	Спільні компоненти вузлів	19
3.3	Асиметрія вузлів: TX, керований перериванням, і RX, керований PWM	20
3.3.1	Передавач (Node A)	20
3.3.2	Приймач (Node B)	21

4

Справжня генерація випадкових чисел (TRNG) та керування ентропією 23

4.1	Джерела апаратної ентропії	24
4.2	Криптографічний змішувач ентропії	28
4.2.1	Етап 1 — ротаційно-XOR-губка	28
4.2.2	Етап 2 — розширення потоку ключа ChaCha20	29
4.3	Неблокувальний доступ до CSPRNG під час виконання	31

5 Специфікація протоколу C2P-ARQ 34

5.1	Життєвий цикл сеансу: обмін SYN, передавання даних (DAT), завершення (FIN)	35
5.1.1	SYN-обмін	35
5.1.2	Фаза DAT	37
5.1.3	Завершення FIN	39
5.1.4	Повна діаграма послідовності сеансу	40
5.2	Формат кадрів: структури HelloPacket і DataPacket	40
5.2.1	AAD: Автентифікований відкритий заголовок	41
5.2.2	HelloPacket — 15 байтів на дроті	42
5.2.3	DataPacket — 17 байтів на дроті	42
5.3	Механізм підтвердження та запобігання неконтрольованим повторним спробам	44
5.3.1	Контур зворотного зв'язку ARQ	44
5.3.2	Умова тайм-ауту і інкремент повтору	45
5.3.3	Запобігання неконтрольованим повторним спробам і призупинення самовідновлення	45

6 Модель безпеки та криптографічне ядро 48

6.1	Автентифіковане шифрування з даними супроводу (ChaCha20-Poly1305)	49
6.1.1		

6.1.2		
	Роль додаткових автентифікованих даних — прив'язка заголовка кадру	49
6.1.3	Конвеєр шифрування та автентифікації	50
6.1.4	Формальний предикат верифікації	51
6.2	16-бітний номер послідовності та керування nonce	54
6.2.1	Небезпека повторного використання nonce	54
6.2.2	Виведення per-packet nonce	54
6.2.3	Ін'єктивність відображення	56
6.2.4	XOR проти модульного додавання	57
6.3	Стирання ключів і безпечне керування пам'яттю	57
6.3.1	Залишковість даних у SRAM ATmega328P	57
6.3.2	Пряма секретність через детерміноване стирання	58
6.3.3	Точки спрацювання стирання	59
6.3.4	Очищення початкового seed ентропії	61
6.3.5	Примітка щодо оптимізації компілятора	62
7	Відмовостійкість і механізми самовідновлення	63
7.1	Стан RECONNECTING і збереження стану корисного навантаження	64
7.1.1	Наївна IoT-архітектура	64
7.1.2	Поділ стану в C2P-ARQ	64
7.1.3	Інваріант самовідновлення	64
7.1.4	Неблокувальне опитування RECONNECTING	66
7.2		
	Розумний динамічний watchdog: адаптація тайм-ауту з урахуванням корисного навантаження	66
7.2.1	Проблема тупика на приймачі	66
7.2.2	Адаптація з урахуванням корисного навантаження	67
7.2.3	Формальна умова тайм-ауту	68
7.2.4	Відсутність гонок між TX і RX	69
7.3	Очищення застарілого буфера і відновлення після десинхронізації	70

7.3.1	Проблема отруєння застарілими байтами	70
7.3.2	Активне очищення TX — drainRxFifo()	71
7.3.3	Пасивне очищення RX — resetParser()	72
7.3.4	Пасивна синхронізація RX — пошук преамбули	72
7.3.5	Ймовірність хибної синхронізації	73
8	Модель супротивника та симуляція глушіння	75
8.1	Можливості вузла атакувальника	75
8.2	Архітектура цифрового генератора шуму та XOR-ін'єкція	76
8.3	Результати симуляції: стійкість під час тривалого глушіння	79
8.3.1	Сценарій 1: ініціалізація системи під активним глушінням	80
8.3.2	Сценарій 2: базове передавання корисного навантаження (чистий канал)	83
8.3.3	Сценарій 3: криптографічна варіація у наступних пакетах (чистий канал)	86
8.3.4	Сценарій 4: активне глушіння, пошкодження корисного навантаження і десинхронізація станів	89
8.3.5	Сценарій 5: тайм-аут передавача та призупинення сеансу	94
8.3.6	Сценарій 6: взаємне завершення сеансу та мертвий ефір	96
8.3.7	Сценарій 7: спроби повторної ініціалізації сеансу під активним глушінням	99
8.3.8	Сценарій 8: успішне відновлення сеансу та збереження стану застосунку	104
8.3.9	Сценарій 9: легітимне завершення сеансу і уникнення аналізу трафіку	109
9	Висновки та подальші роботи	115
9.1	Підсумок ефективності протоколу	115
9.2	Міграція до команд керування актуаторами	116
9.3	Глибокий сон і оптимізація живлення	117

1 Вступ

Поширення недорогих бездротових IoT-пристроїв створило інфраструктуру, яка залежить від COTS-радіомодулів, спочатку не призначених для роботи в умовах ворожого середовища. Такі модулі реалізують лише функції нижчих рівнів протоколу, перекладаючи керування сеансом, перевірку цілісності та конфіденційність повністю на розробника прикладного рівня — делегування, яке в ресурсно обмежених впровадженнях зазвичай опускають. SH2SC-EDT усуває цю структурну прогалину шляхом спільного проєктування фізичного каналу передавання з виділеним протоколом рівня сеансу C2P-ARQ, що інтегрує автентифіковане шифрування ChaCha20-Poly1305 як базову примітиву, а не як необов’язкове розширення. [1, 4, 5]

1.1 Ландшафт уразливостей COTS RF-модулів

COTS приймачі у діапазоні sub-GHz (наприклад, Si446x, CC1101, RFM69HW) реалізують рівень PHY — модуляцію, корекцію прямої помилки та вимірювання RSSI — і елементарний рівень MAC, що забезпечує фільтрацію адрес і валідацію кадру CRC-16. Їхній апаратно визначений стек протоколів завершується на рівні каналу. Поняття сеансу, прив’язки до номера послідовності або криптографічної автентифікації нижче межі прикладного рівня не існує; трансивер структурно «сліпий» до стану сеансу.

Це архітектурне обмеження створює три незалежно експлуатовані класи вразливостей.

1.1.1 Атаки повторного відтворення

За відсутності токенів свіжості на рівні сеансу будь-який захоплений PHY-кадр є валідною командою. Зловмисник, розташований у зоні радіодії, може повторно ввести раніше спостережений пакет і в будь-який момент викликати ідентичну реакцію виконавчого пристрою. CRC-16 не надає механізму для розрізнення легітимної передачі від захопленого повтору, оскільки обидві форми структурно ідентичні на рівні каналу. [3, 6, 7]

1.1.2 Ін'єкція відкритого тексту

Корисне навантаження у відкритому тексті дозволяє пасивному спостерігачеві відновити повну схему кодування команд після мінімального аналізу трафіку. Після цього нападник може конструювати та передавати довільні команди без володіння будь-яким ключовим матеріалом, оскільки нижче рівня прикладного каналу не існує криптографічного шлюзу автентифікації. Отже, відсутність конфіденційності є не лише ризиком розкриття інформації, а й прямим чинником активної ін'єкції команд. [1, 2, 4]

1.1.3 Десинхронізація стану сеансу

COTS-модулі не експонують транспортно-рівневого скінченного автомата. Один шумовий сплеск, достатній для пошкодження одного кадру, залишає передавач і приймач із несумісними лічильниками послідовності та без визначеного протоколом шляху до узгодження. Єдиним доступним механізмом відновлення є повне скидання живлення одного чи обох вузлів, що є операційно неприйнятним в автономних або вбудованих розгортаннях.

Ці вразливості не є дефектами реалізації в будь-якому окремому продукті; вони є прямим структурним наслідком межі OSI-рівня, на якій завершується прошивка COTS-модуля. Усунення таких проблем потребує явного транспортного протоколу, який вимагає AEAD для кожного пакета, керує попси сеансу та забезпечує детерміноване відновлення після десинхронізації — саме той набір функцій, який надає C2P-ARQ. [1, 4, 5]

1.2 Загроза глушіння та втрата синхронізації

Активне RF-глушіння — це не пасивне явище навколишнього середовища, а навмисна атака на фізичний рівень: зловмисник передає сигнал завад на цільовому діапазоні частот, піднімаючи шумову рівня вище порога чутливості приймача. Безпосереднім наслідком на рівні РНУ є зростання бітової похибки, яке систематично перевищує здатність будь-якої використаної FEC-схеми до виправлення, утворюючи безперервний потік помилок CRC. На відміну від багатопроменевого замирання чи теплового шуму, які пошкоджують кадри ймовірно та періодично, тривале глушіння робить канал детерміновано непридатним до використання на весь час дії атаки. За повномасштабного глушіння

рівень втрат пакетів наближається до 100 % незалежно від політики повторної передачі, оскільки пошкодження виникає до будь-якої видимості для шару ARQ.

Критичним наслідком для станозалежних протоколів є невідповідність стану сеансу. Коли DATA-кадр із номером послідовності N пошкоджується під час передавання, FSM передавача залишається в `WAITING_ACK` (очікуючи відповіді на N), тоді як FSM приймача лишається у своєму попередньому стані (оскільки N ніколи не було оброблено). Якщо відповідний ACK або NACK згодом губиться в шумі, передавач не може відрізнити мовчання каналу від збою приймача. Симетрично, якщо NACK губиться, передавач може просунути свій лічильник послідовності, тоді як приймач очікує повторної передачі N — що породжує невідновлювану розбіжність номера послідовності. Будь-який із цих сценаріїв призводить до невідповідності стану FSM, з якої жоден кінець не може вийти без явної протокольної підтримки.

У розгортаннях на основі COTS без визначеного механізму відновлення сеансу така невідповідність станів ескалує до тупика FSM: передавач утримує відкритий дескриптор сеансу в очікуванні підтвердження, яке ніколи не надійде, а приймач займає пам'ять сеансу в очікуванні кадру даних, який ніколи не буде надіслано. Потік станозалежних даних остаточно переривається. Єдиним виходом із цього тупика є позапротокольне скидання — ручне вимкнення живлення або спрацювання watchdog, що відкидає весь стан сеансу та примушує до холодного перезапуску із початку послідовності передавання. Для будь-якого застосування, яке вимагає безперервності крізь збої, така поведінка є фундаментальним дефектом протоколу, а не аномалією розгортання.

1.3 Наш внесок: архітектура SH2SC-EDT

У цій статті представлено SH2SC-EDT, апаратно-програмний комплекс і його транспортне ядро C2P-ARQ, спроектовані для усунення трьох класів вразливостей, визначених у [Section 1.1](#), та тупикового стану, описаного в [Section 1.2](#). Архітектура ґрунтується на трьох співпроекттованих опорах, які разом перетворюють незахищений, безстанний радіоканал на відмовостійкий, криптографічно автентифікований канал сеансу.

1.3.1 Інтегроване автентифіковане шифрування для кожного пакета

C2P-ARQ вимагає використання ChaCha20-Poly1305 AEAD для кожного DATA- та FIN-кадру без винятку. Per-packet попсо детерміновано виводиться з попсо сеансу та 16-бітового номера послідовності, що гарантує відсутність спільного IV у будь-яких двох пакетів із усіх 65,536 можливих значень послідовності. 3-байтовий заголовок кадру — що складається з байта прапорів і little-endian номера послідовності — пов’язується з шифротекстом як Additional Authenticated Data (AAD), тому будь-яке однобітове пошкодження прапорів або лічильника послідовності робить тег Poly1305 недійсним. Така конструкція робить атаки повторного відтворення, ін’єкцію відкритого тексту та приховану модифікацію даних криптографічно практично неможливими без знання 256-бітового попередньо розподіленого ключа, вбудованого у прошивку.

1.3.2 Строго неблокувальні скінченні автомати

Уся логіка передавача та приймача реалізована як детерміновані FSM без викликів `delay()` у головному циклі виконання. Кожен перехід FSM умовлено порівняннями різниць `millis()`, що робить використання RAM і час CPU на один прохід циклу статично обмеженими та передбачуваними. FSM передавача визначає дев’ять явних станів, які охоплюють повний життєвий цикл сеансу — `IDLE`, `RECONNECTING`, `SENDING_HELLO`, `WAITING_HELLO_ACK`, `SENDING`, `WAITING_ACK`, `WAIT_BETWEEN_NOTES`, `SENDING_FIN` і `WAITING_FIN_ACK` — тоді як приймач реалізує чотиристовпний байтовий парсер, показаний у Figure 1.1, що керується кожним вхідним UART-байтом. Така конструкція усуває клас помилок конкуренції, які виникають через блокувальний I/O, та гарантує детерміновану поведінку в реальному часі, властиву середовищу симуляції SimulIDE.

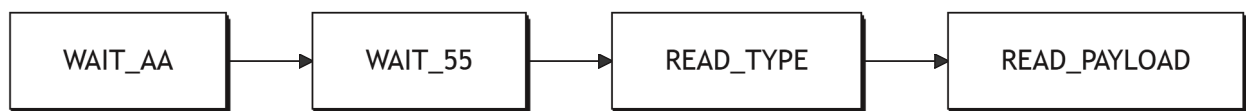


Figure 1.1: Стани побайтового парсера кадру RX (`processReceivedByte()`): кожен вхідний байт UART просуває парсер на один крок; `resetParser()` після помилки MAC або тайм-ауту повертає парсер до `WAIT_AA`.

1.3.3 Самовідновний ARQ з адаптивним сторожовим таймером

C2P-ARQ містить механізм Auto-Resume, який явно зберігає стан прогресу корисного навантаження (`melodyIndex`) через збої сеансу. Після вичерпання `MAX_RETRIES` у будь-якому стані `WAITING_*` передавач викликає `suspendSession()` — нулюючи попсо сеансу для забезпечення прямої секретності, скидаючи лічильник повторів і переходячи до `RECONNECTING`, — при цьому індекс передавання лишається незмінним. Новий SYN-обмін відновлює сеанс, після чого передавання даних продовжується з того самого пакета, який передувало порушенню. На стороні приймача Smart Dynamic Watchdog обчислює тайм-аут на пакет як `note_duration_ms` + `NETWORK_GRACE_PERIOD_MS`, забезпечуючи своєчасне звільнення пам'яті сеансу, коли передавач зникає без надсилання FIN-кадру, незалежно від тривалості перерваної ноти. Уся архітектура орієнтована на ATmega328P (16 MHz, 8-bit AVR, 2 KB SRAM) і перевіряється наскрізно в SimulIDE під дією апаратно ін'єктованого шуму, який створюється XOR-елементом, керованим генератором імпульсів 1 kHz.

2 Архітектура системи та концептуальний дизайн

SH2SC-EDT будується навколо двох рівнозначних обмежень: асиметрії вузлів, яка зосереджує весь стан сеансу на передавачі, та суворій детермінованості, яка усуває будь-який блокувальний шлях з обох моделей виконання прошивки. Цих двох обмежень недостатньо для верифікації: протокол, який обробляє лише статичні, однорідні корисні навантаження, може пройти перевірки коректності в безпечних умовах, але приховати латентні збої в логіці таймінгів і відновлення стану. Кінцева верифікація тому потребує корисного навантаження, яке є внутрішньо станозалежним, часово неоднорідним і придатним для людської перевірки без інструментації.

2.1 Доказ концепції через корисне навантаження: обґрунтування передавання мелодії

Концептуальна інстанція C2P-ARQ передає монофонічну музичну партитуру — зокрема тему Super Mario Bros. — закодовану як послідовність пар (`note_index`, `duration_ms`), що зберігаються в `PROGMEM` на вузлі передавача. Кожна пара окремо шифрується, автентифікується та підтверджується перед відправленням наступної. Вибір цього корисного навантаження не є випадковим; це інженерне рішення, ґрунтоване на чотирьох властивостях верифікації, яких не дає дискретне або статичне навантаження.

2.1.1 Часова залежність

Тривалості нот у реальній музичній партитурі охоплюють щонайменше два порядки величини — від шістнадцяткових нот (≈ 100 ms) до довгих нот понад 1000 ms. Саме ця неоднорідність дає змогу випробувати Smart Dynamic Watchdog у реалістичних умовах. Watchdog встановлює тайм-аут звільнення сеансу T_{RX} динамічно після кожного автентифікованого пакета за формулою:

$$T_{RX} = d_k + T_{\text{grace}}$$

де T_{RX} — тайм-аут звільнення сеансу, обчислений для кожного пакета, d_k — тривалість k -ї ноти в мілісекундах, а T_{grace} — фіксований мережевий grace period, що поглинає worst-case retry storm. Однорідне за тривалістю навантаження перетворило б цей адаптивний механізм на статичний таймер, лишивши його ядро адаптації взагалі неперевіреною.

2.1.2 Станозалежний потік даних

Музична партитура є впорядкованою послідовністю з чітко визначеним індексом прогресу. Це дає змогу прямо й однозначно перевірити механізм Auto-Resume: після порушення сеансу на індексі ноти N коректна реалізація має відновитися точно з N після повторного з'єднання — не з індексу 0. Спостерігач може підтвердити або спростувати цю гарантію на слух у реальному часі. Безстанне payload — наприклад, один перемикач актуатора — не дає такої контрольної структури; механізм Auto-Resume породжував би вихід, не відрізний від холодного перезапуску.

2.1.3 Акустична діагностика

Вихід зумера на приймачі є завжди ввімкненим діагностичним каналом без інструментації. Будь-яка втрата пакета створює чутний розрив; будь-яка дубльована передача створює повтор ноти; будь-яке відхилення таймінгу псує ритмічну структуру. Ці аномалії миттєво сприймаються людиною без аналізу журналів, підключення осцилографа чи постобробки. Це властивість human-in-the-loop verification: увесь життєвий цикл сеансу C2P-ARQ — включно з повторною передачею, відхиленням MAC і Auto-Resume — чутний як безперервна мелодія, коли протокол працює коректно.

2.1.4 Зворотна масштабованість

Сценарій передавання мелодії є найскладнішим випадком у класі поточкових, часочутливих корисних навантажень. Формат дроту C2P-ARQ є незалежним від типу корисного навантаження: пара (note_index, duration_ms) займає ті самі чотири зашифровані байти, які в production IoT-розгортанні могла б нести будь-яка пара (command_id, parameter). Якщо протокол забезпечує коректну, впорядковану, автентифіковану доставку під тривалим глушінням і з неоднорідним профілем

таймінгів, його коректність для простішого дискретного командного навантаження впливає безпосередньо. Перехід від нот мелодії до команд актуаторів не потребує жодних модифікацій на рівні протоколу.

2.2 Парадигма "товстий сервер / тонкий клієнт"

C2P-ARQ навмисно концентрує весь тягар стану сеансу на вузлі передавача. Передавач — вузол А — володіє масивом мелодії в `PROGMEM`, підтримує поточний `melodyIndex`, керує повним дев'ятистатним життєвим циклом FSM від `IDLE` до `WAITING_FIN_ACK`, виконує `suspendSession()` у разі втрати каналу та керує `retryCount`, `seqNum`, усіма порівняннями тайм-аутів на базі `millis()` і інтервалом повторного з'єднання. Таке асиметричне розподілення складності не є наслідком зручності реалізації, а свідомим протокольним вибором: у будь-якій системі ARQ типу точка-точка рівно один кінцевий вузол має володіти авторитетним станом прогресу. Розміщення цього стану на передавачі замикає поверхню логіки сеансу в одному вузлі й робить приймач структурно безстанним щодо прогресу корисного навантаження. [14, 15, 16]

Приймач — вузол В — виконує лише мінімальний обсяг роботи, необхідний для автентифікації, декодування та виконання кожної вхідної команди. Отримавши повний кадр, приймач виводить попсе для кожного пакета, виконує повний конвеєр ChaCha20-Poly1305 для розшифрування та перевірки Poly1305 tag і умовно відтворює декодовану ноту. На приймачі немає знання мелодії, лічильника `melodyIndex` чи логіки відновлення сеансу. Його єдиний стан на рівні сеансу — це 12-байтовий буфер попсе та один булевий прапорець `s_sessionActive`. Обробка помилок на приймачі відповідно мінімальна: кадр, який не проходить перевірку тегу, мовчки відкидається після надсилання `NACK_BYTE` по каналу зворотного зв'язку; приймач не журналює помилку, не планує запит на повторну передачу і не змінює свій стан FSM. Уся розкладка повторних передач, лічильник повторів і керування таймаутами делегуються повністю передавачу.

Це асиметричне розподілення складності дає не лише концептуальну елегантність, а й конкретну інженерну перевагу. У реальних IoT-розгортаннях приймач часто є більш обмеженим за ресурсами вузлом — батарейним актуатором, контролером реле або сенсорним хабом із кілобайтами доступної RAM і без постійного сховища. Парадигма товстого сервера / тонкого клієнта гарантує, що обсяг RAM і вартість CPU на один цикл приймача залишаються статично обмеженими та мінімальними незалежно від складності сеансу, довжини корисного

навантаження або стану мережі. Передавач, який зазвичай працює від мережі або більшого батарейного бюджету, поглинає весь обчислювальний та пам'яттєвий кошт протоколу. Це розподілення стабільне при масштабуванні навантаження: додавання нових нот, розширення `MAX_RETRIES` або збільшення `RECONNECT_INTERVAL_MS` впливає лише на поведінку передавача й не змінює прошивку приймача. [14, 15, 16]

2.3 Детермінована поведінка через скінченні автомати (FSM)

Блокувальні процедури є фундаментальною архітектурною несумісністю з транспортно-рівневими протокольними реалізаціями на однопроцесорних мікроконтролерах. Виклик `delay()` безумовно призупиняє CPU на весь свій час, упродовж якого процесор не може обслуговувати вхідні UART-байти, оцінювати тайм-аути ACK, просувати цикл збору ентропії TRNG або оновлювати LCD. На ATmega328P, що працює на 9600 bps, одна команда `delay(50)` охоплює той самий інтервал реального часу, що й майже п'ять повних UART-передач байта, — цього достатньо, щоб ACK, який прийшов, переповнив 64-байтовий апаратний FIFO і був мовчки втрачений. Результатом є голодування CPU: протокольний шар стає глухим до подій каналу саме в ті інтервали, коли має бути найбільш чутливим. Будь-яка коректна реалізація C2P-ARQ тому має без винятків усунути блокувальні процедури з головного циклу виконання. [17, 18]

C2P-ARQ усуває це обмеження через виключне використання неблокувальних FSM, чиї переходи станів умовлені порівняннями різниць `millis()`. Кожен виклик `loop()` виконує не більше одного переходу FSM, споживає статично обмежену кількість інструкцій і повертається негайно — досягаючи латентності на ітерацію, вимірюваної мікросекундами, а не мілісекундами. Умови тайм-аутів, такі як вікно очікування ACK (`ACK_TIMEOUT_MS = 50 ms`), інтервал повторного широкомовлення (`RECONNECT_INTERVAL_MS = 2000 ms`), тривалість відтворення ноти та межа Smart Dynamic Watchdog, усі реалізовані як порівняння виду `(millis() - t_start) >= threshold`, що перевіряються раз на ітерацію циклу й просувають FSM лише тоді, коли умова виконується. Немає жодної синхронізації потоків, жодної interrupt-орієнтованої мутації стану й жодного блокувального шляху I/O в усьому стосі протоколу.

Ця конструкція надає темпоральну детермінованість як властивість першого класу: у будь-який момент виконання точний стан системи закодовано в одному значенні `enum` без прихованого неявного стану. FSM передавача визначає дев'ять станів — `IDLE`, `RECONNECTING`, `SENDING_HELLO`, `WAITING_HELLO_ACK`, `SENDING`, `WAITING_ACK`,

WAIT_BETWEEN_NOTES, SENDING_FIN і WAITING_FIN_ACK — які покривають повний життєвий цикл сеансу від спокою до автентифікованого завершення. Приймач реалізує чотиристановий байтовий парсер, показаний у Figure 2.1, який просувається по одному байту за раз і атомарно скидається до WAIT_AA на будь-яку помилку MAC або parser timeout, гарантуючи, що пошкодження межі кадру не може поширитися на наступні кадри. Оскільки вся логіка відновлення сеансу — `suspendSession()`, стирання `ponse`, збереження `melodyIndex` — виконується в межах чітко визначених переходів станів, механізм Auto-Resume є вільним від race conditions і витоків пам'яті за конструкцією: система може залишатися в RECONNECTING нескінченно довго під тривалим глушінням, не накопичуючи необмежене споживання ресурсів. [19, 20]

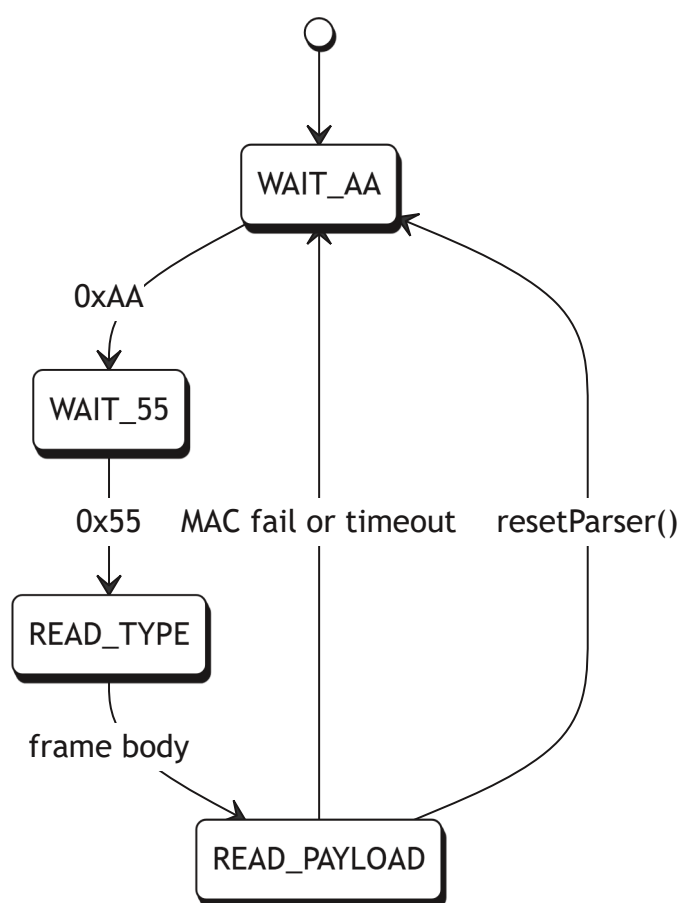


Figure 2.1: Шлях відновлення парсера RX. Побайтовий FSM просувається через приймання преамбули та повертається до WAIT_AA після помилки MAC, тайм-ауту парсера або явного виклику `resetParser()`.

3 Апаратна архітектура та середовище симуляції

SH2SC-EDT було спроектовано від самого початку для верифікації у повністю відтворюваному середовищі симуляції. У SimulIDE змодельовано кожен фізичний компонент — UART-канал, I2C-шину, п'єзозумер, ентропійний кільцевий осцилятор і ін'єктор шуму XOR — що дає змогу виконувати суворо повторювані експерименти з відмовостійкості без доступу до фізичного обладнання. Усі часові константи, налаштування швидкості передавання та результати перевірки MAC, наведені в цій статті, отримано безпосередньо з цієї симуляції; жодної постфактум калібровки чи підбору параметрів поза середовищем симуляції не виконувалося.

3.1 Тестовий стенд SimulIDE: топологія та абстракція фізичного рівня

Вузол А (Transmitter, “The Conductor”) — це Arduino Nano на базі ATmega328P, відповідальний за весь життєвий цикл сеансу C2P-ARQ: він володіє таблицею мелодії в `PROGMEM`, підтримує авторитетний `melodyIndex`, керує дев'ятистанним FSM передавача від `IDLE` до `WAITING_FIN_ACK` і виконує `suspendSession()` у разі втрати каналу. Вузол В (Receiver, “The Synthesizer”) — другий Arduino Nano, який зберігає лише універсальний словник частот нот із 21 елемента та 12-байтовий попсе сеансу; він навмисно є безстанним щодо прогресу мелодії. Атакувальник — це лише апаратна підсистема: він не є мікроконтролером, не є третім вузлом і не є прошивним елементом. Він складається з XOR-елемента, генератора імпульсів 1 kHz із 15% duty cycle, AND-елемента та ручного перемикача, з'єднаних послідовно на UART-лінії даних між вузлами А та В.

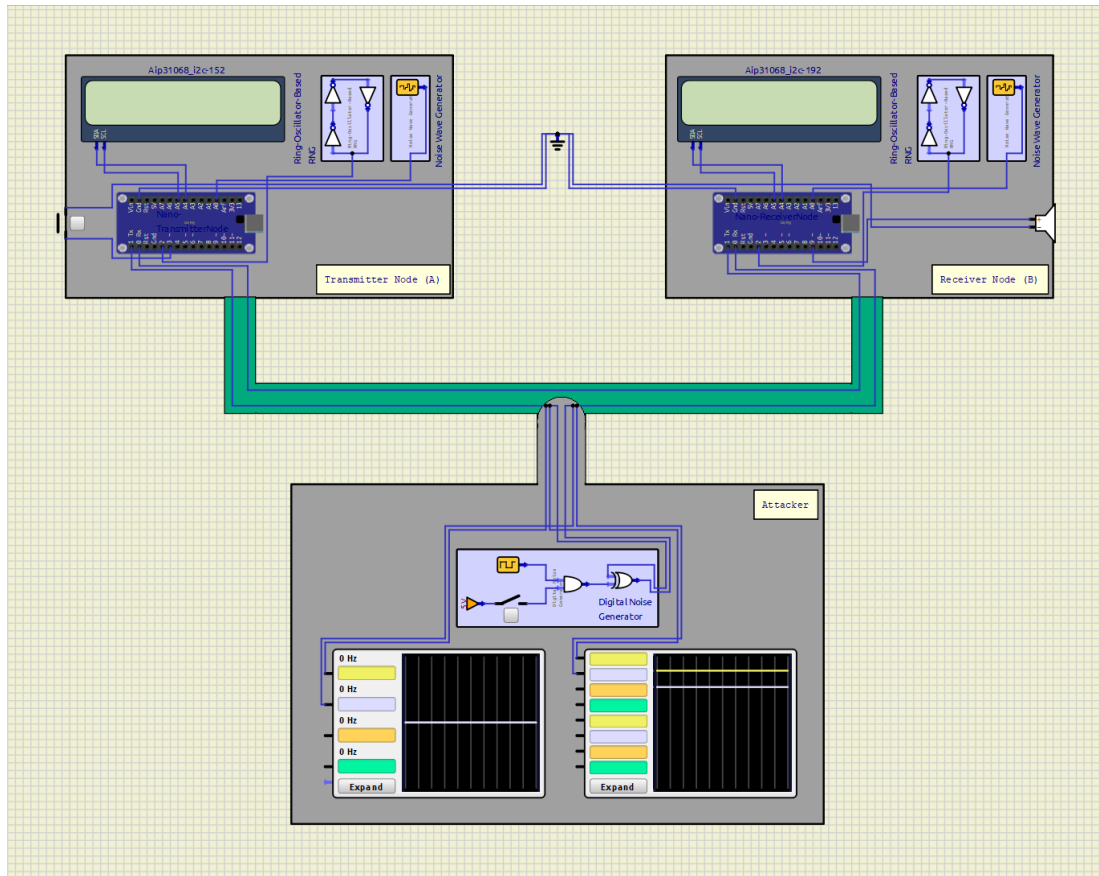


Figure 3.1: Повна топологія симуляції в SimulIDE. Вузол А (передавач) з'єднано з вузлом В (приймачем) через UART-шину, прокладену крізь ін'єктор шуму XOR. Чистий зворотний канал повністю обходить ін'єктор.

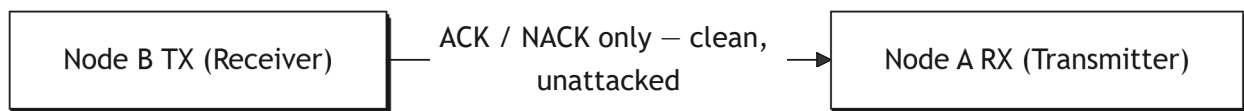


Figure 3.2: Напрямок зворотного UART-каналу у випробувальному стенді SH2SC-EDT. TX вузла В досягає RX вузла А лише через чистий, неатакований зворотний провід, що переносить ACK_BYTE і NACK_BYTE.

SimulIDE не надає моделі поширення RF. Відповідно, фізичний рівень абстраговано як двоточкову UART-цифрову шину, що працює на 9600 bps (8N1). Це абстрагування є не обмеженням, а свідомим вибором: усунення складності RF-рівня — багатоприменевості, частотно-селективного завмирання, діаграм підсилення антени — ізолює протокольний рівень як єдину змінну, що досліджується. UART-шина переносить два односпрямовані канали, як показано в Figure 3.3. Канал даних, прокладений від вузла А TX до вузла В RX, проходить через атакувальник XOR: коли перемикач шуму відкритий, вихід XOR збігається з оригінальним сигналом;

коли перемикач закрито, прямокутна хвиля генератора імпульсів 1 kHz накладається на шину через XOR, перевертаючи окремі UART-біти та утворюючи реалістичне часткове пошкодження кадрів. За 9600 bps один UART-біт триває приблизно 104 μ s; отже, шумовий імпульс 150 μ s (15% від періоду 1000 μ s) достатньо широкий, щоб пошкодити від одного до дев'яти послідовних бітів за один сплеск, не знищуючи кожен біт у кожному кадрі, — утворюючи ймовірнісний розподіл помилок у пакетах, який шар ARQ C2P-ARQ і покликаний пережити. Канал зворотного зв'язку, що працює від вузла B TX до вузла A RX, є чистим, неін'єктованим прямим дротом. Він переносить лише одnobайтові керувальні кадри: `ACK_BYTE = 0x06` після успішної перевірки Poly1305 і `NACK_BYTE = 0x15` у разі помилки MAC.

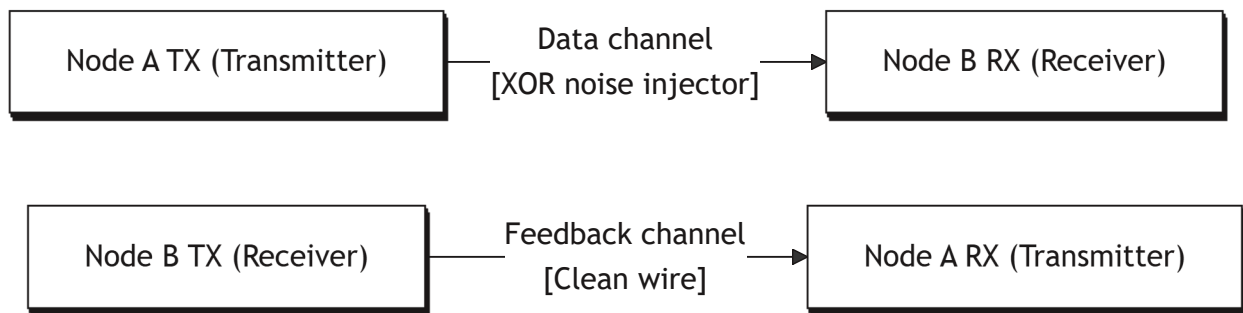


Figure 3.3: Топологія асиметричного UART-каналу в цьому розділі. Прямий канал даних проходить через ін'єктор шуму XOR, тоді як зворотний канал залишається чистим прямим проводом.

3.2 Спільні компоненти вузлів

Обидва легітимні вузли мають ідентичне ядро цифрової обробки. Обчислювальна основа — це ATmega328P із тактовою частотою 16 MHz, 8-бітовий AVR з 32 KB Flash, 2 KB SRAM і 1 KB EEPROM. Бюджет SRAM у 2 KB є домінантним обмеженням усього дизайну прошивки: він забороняє динамічне виділення пам'яті (`new`, `malloc()`), накладає жорстку верхню межу на глибину стеку та вимагає, щоб усі буфери пакетів, попсе та змінні стану FSM оголошувалися статично під час компіляції. Кожна структура даних у стеку C2P-ARQ — `HelloPacket`, `DataPacket`, `s_sessionNonce`, 64-байтовий кеш keystream CSPRNG — має розмір, визначений на етапі компіляції, і займає фіксоване, наперед відоме місце в карті SRAM.

Обидва вузли оснащено дисплеєм Air31068 16×2 I2C, підключеним через апаратну I2C-шину (піни A4/A5, адреса 0x3E). Дисплей є не декоративним елементом, а критично важливим позасмуговим діагностичним інструментом.

Оскільки запис до I2C виконується прямо з коду переходів FSM, а не в окремому блокувальному циклі виводу, LCD відображає поточний стан протоколу в реальному часі, не додаючи затримки до основного шляху виконання. На вузлі А верхній рядок показує PKT:<seqNum> RTY:<retryCount>, а нижній — поточну мітку стану FSM (SENDING, WAITING_ACK, RECONNECTING тощо). На вузлі В верхній рядок відображає стан FSM, а нижній — SEQ:<n> OK або SEQ:<n> MAC! після кожного валідованого або відхиленого кадру. Такий безперервний, читабельний для людини дисплей стану забезпечує повну візуальну верифікацію життєвого циклу сеансу C2P-ARQ — обмін, передавання даних, шторм повторних передач, самовідновне повторне з'єднання і автентифіковане завершення — без підключення логічного аналізатора або серійного монітора. У контексті академічної верифікації LCD виступає синхронізованим, однозначним каналом істини для спостереження за станом протоколу.

Обидва вузли додатково містять апаратну підсхему ентропії для ініціалізації TRNG. Архітектура та стратегія змішування ентропії цієї підсхеми детально розглядаються в [Section 4](#).

3.3 Асиметрія вузлів: TX, керований перериванням, і RX, керований PWM

Поза спільним обчислювальним ядром два вузли відрізняються своїм набором периферії, і ці апаратні відмінності є прямим фізичним вираженням асиметрії Fat Server / Thin Client, описаної в [Section 2.2](#).

3.3.1 Передавач (Node A)

Node A є ініціатором сеансу та єдиним джерелом керування протоколом. Миттєва кнопка, підключена до цифрового піна 3 (INT1) з INPUT_PULLUP, асинхронно запускає встановлення сеансу через апаратне переривання. Фільтр антидребезгу на основі `millis()` із вікном 50 ms усуває шум контакту з лінії переривання, гарантуючи, що одне фізичне натискання кнопки породжує рівно один перехід FSM від IDLE до SENDING_HELLO. Передавач не має аудіовиходу; вся його периферійна поверхня складається з I2C LCD, підсхеми ентропії та кнопки, що породжує переривання. Апаратну схему показано в Figure 3.4.

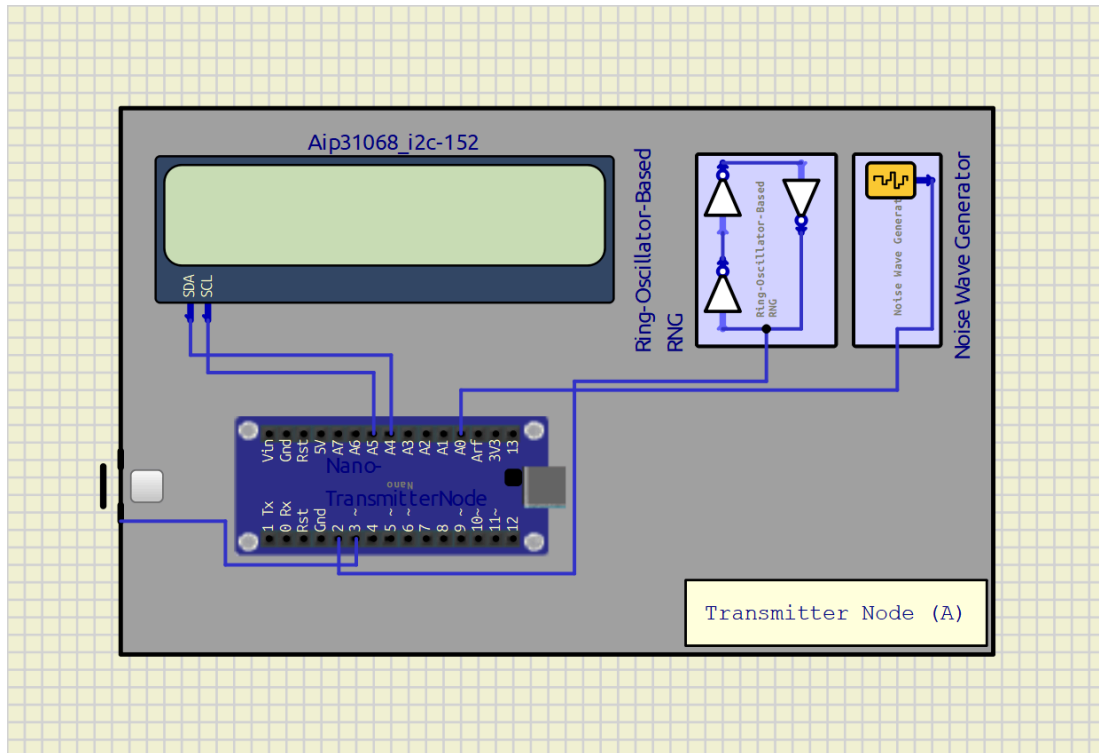


Figure 3.4: Апаратна архітектура передавача (вузол А) в SimulIDE. Переферія: I2C LCD Aip31068 (A4/A5), підконтур TRNG на кільцевому осциляторі (INT0/вивід 2), підсистема Noise Wave Generator та кнопка запуску сеансу (вивід 3).

3.3.2 Приймач (Node B)

Node B є виконавчим кінцем і не генерує керувальних сигналів протоколу, окрім однобайтових кадрів ACK/NACK. Він не має кнопки та не потребує вводу, керованого перериванням. Натомість він оснащений п'єзодинаміком, підключеним до PWM-спроможного піна 9 (Timer 1, OC1A), який керується API Arduino `tone()` / `noTone()` — це налаштовує апаратний таймер на генерування квадратної хвилі потрібної частоти без участі ПЗ, звільняючи CPU для подальшого обслуговування вхідних UART-байтів під час відтворення. Відтворення нот є строго неблокувальним: `tone(pin, frequency)` викликається один раз на початку ноти, фіксується timestamp завершення, а `noTone()` викликається в наступній ітерації `loop()`, у якій виконується `(millis() - noteStartTime) >= durationMs`. Жодного `delay()` у послідовність відтворення не вставляється. Апаратну схему приймача показано в Figure 3.5.

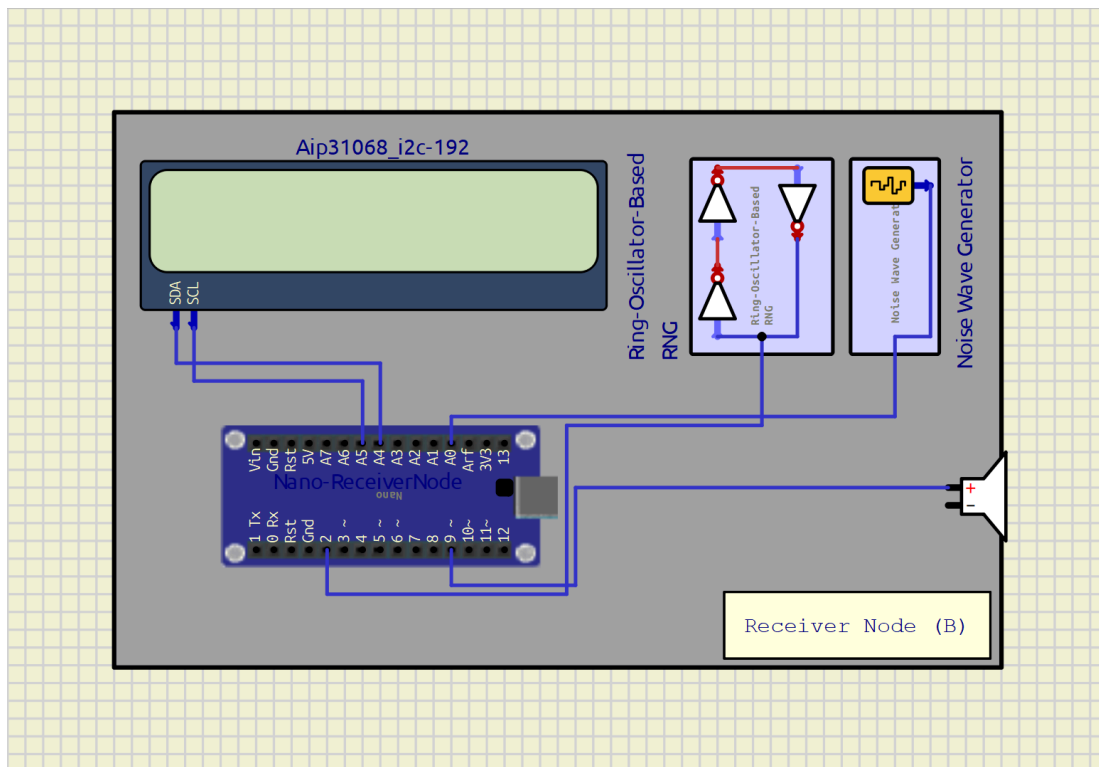


Figure 3.5: Апаратна архітектура приймача (вузол B) в SimulIDE. Переферія: I2C LCD Air31068 (A4/A5), підконтур TRNG на кільцевому осциляторі (INT0/вивід 2), підсистема Noise Wave Generator та п'єзоелектричний динамік (PWM вивід 9).

4 Справжня генерація випадкових чисел (TRNG) та керування ентропією

Криптографічна безпека C2P-ARQ ґрунтується на одній, безкомпромісній вимозі: кожен попсо сеансу має походити з джерела справжньої фізичної випадковості. Це не питання переваг реалізації, а формальна безпекова умова, що впливає з самої AEAD-конструкції. ChaCha20-Poly1305 є шифром, стійким до IND-CPA, за умови що жоден попсо ніколи не повторюється під тим самим ключем. Формально, для простору попсо розміру 96 біт і 256-бітового ключа K , будь-які два шифрування стають небезпечними, якщо попсо повторюється [21, 36, 37]:

$$E_{K,IV_1}(m_1), E_{K,IV_2}(m_2)$$

$$IV_1 = IV_2$$

У такому разі стає спостережуваною XOR-суміш відкритих текстів:

$$m_1 \oplus m_2$$

оскільки keystream ChaCha20 є детермінованим:

$$E_{K,N}(m) = m \oplus \text{ChaCha20}(K, N)$$

де символ

$$E_{K,N}(m)$$

позначає шифрування повідомлення m під ключем K і попсо N , а $\text{ChaCha20}(K, N)$ — відповідний суто детермінований блок keystream. Один-єдиний збіг попсо, таким чином, катастрофічно порушує і конфіденційність, і цілісність Poly1305 — MAC-ключ, похідний від першого блоку keystream ChaCha20, є ідентичним для обох повідомлень, що дає змогу зловмиснику підробляти довільні автентифіковані шифротексти. Це не теоретичний крайовий випадок; будь-який детермінований pseudo-random number generator (PRNG) із передбачуваним або відтворюваним станом — зокрема функція Arduino `random()`, ініціалізована

фіксованою константою, — робить збіги попси між циклами живлення практично неминучими.

Програмні PRNG структурно недостатні як самостійне джерело ентропії для цієї мети. Linear-congruential generator або xorshift PRNG породжують детерміновану послідовність, повністю параметризовану початковим seed. На вбудованому вузлі, який між сеансами вимикається та вмикається, — що є звичайною операційною подією, — будь-який seed, отриманий із компіляторної константи, лише з одного знімка невикористаної SRAM або фіксованого значення ADC, відтворюватиме той самий seed під час кожного запуску, а отже, і той самий перший попси. C2P-ARQ потребує Cryptographically Secure Pseudo-Random Number Generator (CSPRNG), чий seed включає справжню фізичну ентропію — випадковість на рівні бітів, отриману з непередбачуваних фізичних процесів, статистично незалежних від будь-якого попереднього стану, історії завантажень та спостережень з боку супротивника.

4.1 Джерела апаратної ентропії

Фізична випадковість у SH2SC-EDT збирається функцією `generateEntropyPool()`, одноразовою функцією, що викликається під час ініціалізації вузла (`setup()` на TX, `rx_setup()` на RX) і заповнює 32-байтовий seed-буфер шляхом виконання восьми послідовних ітерацій за словами. На кожній ітерації кілька фізично незалежних джерел згортаються в 32-бітовий акумулятор через rotate-XOR sponge; отримане слово записується у вихідний буфер у вигляді чотирьох little-endian байтів. Отже, сумарна вартість блокування становить:

$$8 \times 2 \text{ ms} = 16 \text{ ms}$$

Це прийнятна одноразова витрата в контексті ініціалізації, який не несе обов'язків прийому UART. Передавач використовує сім джерел для кожного слова; приймач — шість, оскільки член `human-timing micros()` відсутній, бо приймач не має кнопки. Апаратну підсхему, що реалізує Джерела 1 і 5, показано в Figure 4.1.

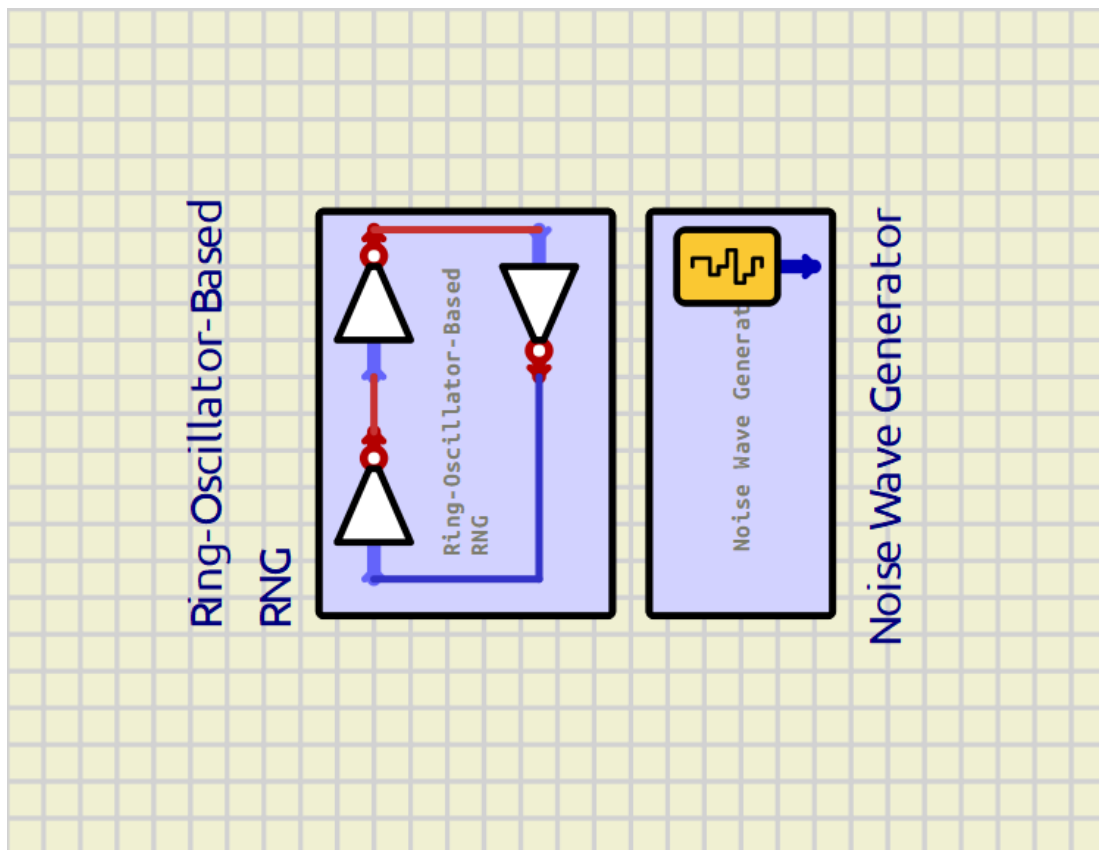


Figure 4.1: Підконтур апаратної ентропії в SimulIDE, що містить кільцевий осцилятор і генератор шумової хвилі. Кільцевий осцилятор (три каскадовані інвертори CMOS у петлі зворотного зв'язку) керує INT0; Noise Wave Generator ін'єктує частотно-модульовані збурення, щоб підсилити фазовий джиттер у контексті симуляції SimulIDE.

Source 1 — ring-oscillator gate (INT0, pin 2, RISING). На початку кожної ітерації за словом `s_ringOscPulses` обнуляється, а `ISR onRingOscPulse()` прив'язується до INT0 по фронту RISING. MCU далі блокується на 2 ms через `delay(2)`, упродовж яких апаратний ring oscillator живить INT0; ISR інкрементує лічильник імпульсів на кожному фронті наростання. Після закриття вікна переривання від'єднується, а лічильник згортається у pool. Миттєвий період k-го циклу ring oscillator виражається як:

$$T_k = T_0 + \Delta t_k$$

де T_0 — номінальний, специфічний для пристрою період осциляції, а Δt_k — накопичений фазовий джиттер на етапі вікна, розподілений як гаусівський із нульовим середнім і дисперсією σ_j^2 :

$$\Delta t_k \sim \mathcal{N}(0, \sigma_j^2)$$

Оскільки Δt_k є фізичним, кількість фронтів у вікні 2 ms змінюється непередбачувано між ітераціями. Ентропія Шеннона для вибірки рахунку C задовольняє [22, 23, 24]:

$$H(C) = - \sum_c P(C = c) \log_2 P(C = c)$$

За номінальних умов джиттера емпіричні рахунки охоплюють 8–16 різних значень, що дає оцінену ентропію Шеннона на одне вікно gate у діапазоні:

$$H(C) \approx 3\text{--}4$$

бітів на вікно gate. У контексті симуляції SimulIDE підсхема Noise Wave Generator ін'єктує частотно-модульовані збурення на pin 2, компенсуючи детерміновані затримки поширення ідеалізованих логічних елементів, які інакше придушили б фізичний джиттер.

Source 2 — uninitialised SRAM. Комірки SRAM на ATmega328P зберігають залишкові зарядові конфігурації, що виникають унаслідок варіацій під час виробництва та попередніх циклів живлення. 64 байти, починаючи з адреси 0x0100, зчитуються без попередньої ініціалізації. Кожна з 8 ітерацій за словами споживає окремий, неперекривний 8-байтовий фрагмент — байти від 0x0100 + wordIndex × 8 до +7 — згортаючи кожен байт окремо через mixEntropy(). Це джерело забезпечує невідтворювану, прив'язану до конкретного пристрою ентропію, що залежить від фізичного розподілу заряду комірок саме цього кристала. [25]

Source 3 — on-die temperature sensor ADC. ATmega328P містить діод температурного сенсора, доступний через канал ADC 8. Встановлення ADMUX = 0xC8 (_BV(REFS1) | _BV(REFS0) | _BV(MUX3)) вибирає внутрішню опорну напругу 1.1 V і мультиплексує вихід температурного діода на вхід ADC. Одне перетворення одразу після перемикавання опори відкидається, щоб конденсатор sample-and-hold встиг усталитися. Потім виконуються вісім послідовних перетворень; LSB кожного значення ADCL виділяється і зсувається в 8-бітовий акумулятор, який згортається в pool. Зчитування температурного сенсора підлягає квантовому шуму та тепловим флуктуаціям, не синхронізованим із CPU clock; LSB у нормальних умовах є недетермінованими, що дає приблизно:

$$H_{\text{LSB}} \lesssim 1 \text{ bit per sample}$$

Це відповідає до 8 біт на слово в сумі.

Source 4 — TCNT1 timer capture. 16-бітовий Timer/Counter 1 працює без скидання MCU. Його значення на межі кожної ітерації за словом визначається кумулятивним часом виконання всіх попередніх джерел у цій ітерації — тривалістю gate ring oscillator, затримками перетворення ADC, часом доступу до SRAM і накладними витратами обслуговування переривань, — жодне з яких не синхронізоване з будь-яким спостережуваним зовнішнім сигналом. TCNT1 зчитується один раз на ітерацію, а його 16-бітове значення згортається в pool, вносячи фазову різноманітність, ортогональну до інших джерел.

Source 5 — A0 noise generator readout. Pin A0 у схемі SimulIDE підключений до виходу підсхеми Noise Wave Generator, яка формує частотно-модульовану хвилю, миттєва амплітуда якої в момент кожного виклику `analogRead(A0)` є псевдовипадковою в межах аналогового діапазону входу. Кожне слово використовує вісім викликів `analogRead(A0)`; LSB кожного 10-бітного зчитування виділяється і зсувається в 8-бітовий акумулятор, який згортається в pool. На реальному апаратному ATmega328P це джерело збирало б тепловий шум або аналоговий сигнал довкілля, під'єднаний до A0.

Source 6 — `micros()` human-timing jitter (TX node only). Момент виклику `generateEntropyPool()` на передавачі визначається натисканням кнопки користувачем, яке неможливо передбачити з точністю до субмікросекунд. `micros()` зчитується один раз на ітерацію за словом і згортається в pool. Це джерело структурно відсутнє на вузлі приймача — варіант `generateEntropyPool()` для RX явно зазначає його відсутність — тому RX має шість джерел на слово замість семи на TX.

Source 7 — Arduino `random()` (obfuscation layer). Один виклик лінійного конгруентного PRNG Arduino згортається в кожне слово як фінальний етап. Це джерело не додає фізичної ентропії — його послідовність детермінована станом PRNG — однак збільшує складність передобраза для супротивника з частковим знанням фізичних джерел, виконуючи роль шару обфускації поверх попередніх шести термінів.

32-байтовий seed, сформований функцією `generateEntropyPool()`, негайно передається до `initCSPRNG()` і очищається зі стеку за допомогою `memset()`, щоб запобігти експозиції залишкового ключового матеріалу. Домінує внесок від ring oscillator. За номінальних умов джиттера ентропія на одне вікно перебуває на рівні:

$$H(C) \approx 3-4$$

бітів на вікно gate 2 ms. За 8 незалежних ітерацій це дає:

$$H_{\text{ring}} \approx 24\text{--}32$$

бітів. Невідініціалізований SRAM bias під час увімкнення, LSB температурного ADC, фазове захоплення TCNT1, зчитування генератора шуму A0 та — на передавачі — джиттер human-timing у `micros()` додають додаткову структурно ортогональну ентропію, яку важко точно кількісно оцінити окремо. За консервативного припущення, що вимірно вносить лише ring-oscillator, загальний сирий бюджет ентропії оцінюється як:

$$H_{\text{raw}} \approx 24\text{--}32$$

бітів на 32-байтовий seed.

4.2 Криптографічний змішувач ентропії

Сирі внески бітів, зібрані з семи фізичних джерел, описаних у [Section 4.1](#), не можуть напряму подаватися в двигун ChaCha20 як ключ. Це обумовлено двома структурними недоліками. По-перше, кожне джерело вносить власне статистичне зміщення: рахунки фронтів ring oscillator групуються навколо специфічного для пристрою середнього, LSB ADC демонструють апаратно-зумовлений дисбаланс 0/1, а член `random()` є повністю детермінованим. По-друге, згортання довільної кількості бітів у 32-бітовий акумулятор не може підняти ентропію цього слова вище 32 біт — фактичний бюджет обмежений домінантним фізичним джерелом. `initCSPRNG()` усуває обидва недоліки через двоступеневий конвеєр змішування, який виконується одразу після повернення `generateEntropyPool()`.

4.2.1 Етап 1 — ротаційно-XOR-губка

Перший етап стискає внески бітів від кожного джерела у компактний 32-байтовий сирий seed за допомогою rotate-XOR sponge, застосованого до кожного з восьми 32-бітових слів акумулятора. Після кожного внеску джерела акумулятор оновлюється за допомогою такої примітиви [28, 29, 30, 31]:

$$\text{pool}' = (\text{pool} \lll 1) \oplus \text{bits}$$

де `pool'` — оновлений 32-бітовий акумулятор, $\lll 1$ позначає ліве обертання акумулятора на один біт (реалізоване як $((\text{pool} \ll 1\text{u}) \mid (\text{pool} \gg 31\text{u})) \wedge \text{bits}$ у C++), а `bits` — внесок, згорнутий із поточного джерела ентропії. Застосована

ітеративно — по одному разу для кожного джерела на слово, для семи джерел і восьми ітерацій за словами — ця примітива запобігає патологічній послідовності однакових значень `bits`, що б взаємно занулювалися, тобто дефекту, властивому простому побітовому XOR-згортанню, і дифузно розподіляє статистичне зміщення кожного окремого джерела по всіх 32 бітових позиціях акумулятора. Вісім отриманих слів конкатенуються в сирий `seed`, який негайно передається до Етапу 2:

$$K_{\text{hw}} = w_0 \| w_1 \| w_2 \| w_3 \| w_4 \| w_5 \| w_6 \| w_7$$

де кожне

$$w_i \in \{0, 1\}^{32}$$

є фінальним значенням акумулятора після згортання всіх внесків джерел для ітерації i , а $\|$ позначає конкатенацію. Після того як Етап 2 споживає цей `seed`, 32-байтовий буфер очищається зі стеку за допомогою `memset()`, щоб запобігти експозиції залишкового ключового матеріалу.

4.2.2 Етап 2 — розширення потоку ключа ChaCha20

Другий етап усуває залишковий недолік: K_{hw} лишається нерівномірною випадковою змінною, ефективна ентропія якої на порядки нижча за 256-бітовий параметр безпеки, який формально вимагає повноцінний ключ ChaCha20.

$$K_{\text{hw}} \in \{0, 1\}^{256}$$

За консервативними припущеннями ця ефективна ентропія становить приблизно:

$$H_{\text{raw}} \approx 24\text{--}32$$

бітів.

Якби цей `seed` напряду використовувався як криптографічний ключовий матеріал, супротивник міг би перебрати простір ключів набагато швидше, ніж за 2^{256} операцій. `initCSPRNG()` вирішує це шляхом криптографічного кондиціювання в один бік: K_{hw} встановлюється як 256-бітовий ключ ChaCha20 через `s_chacha.setKey(hardwareSeed, 32)`, а фіксований 8-байтовий нульовий IV встановлюється через `s_chacha.setIV(zeroIV, 8)` [26, 27, 32, 33]:

$$IV = 0^{64}$$

Бібліотека Rhys Weatherley ChaCha працює в оригінальній djb-ChaCha-конфігурації з 64-бітовим попсе — відмінним від 12-байтового IETF попсе, який використовується в шифрі пакета C2P-ARQ. Нульовий IV є безпечним, оскільки K_{hw} щоразу заново виводиться з фізичної ентропії під час кожного циклу живлення; отже, пара є унікальною між сеансами з надзвичайно високою ймовірністю:

$$(K_{hw}, IV)$$

`s_fillKeystreamBlock()` далі формує перший 64-байтовий блок keystream шляхом шифрування буфера, заповненого нулями, безпосередньо на місці:

$$B_0 = \text{ChaCha20}(K_{hw}, 0^{64}, c_0)$$

де всі нулі в plaintext, XOR-ені з keystream ChaCha20, дають keystream напряду — $\text{output} = 0x00\dots0 \text{ XOR } \text{keystream} = \text{keystream}$ — без потреби у другому 64-байтовому виділенні. Увесь 64-байтовий блок B_0 зберігається в `s_keystreamBuf` і становить початковий кеш CSPRNG. Подальші виклики `getSecureRandom32()` споживають по 4 байти з `s_keystreamBuf` на кожний виклик; коли 64 байти вичерпано, `s_fillKeystreamBlock()` викликається знову, генеруючи блок B_n :

$$B_n = \text{ChaCha20}(K_{hw}, 0^{64}, c_0 + n)$$

де

$$c_0 + n$$

є внутрішнім 64-бітовим лічильником блоків ChaCha20, що гарантує

$$B_n \neq B_m$$

для всіх

$$n \neq m$$

і дає нерівномірний keystream аж до 2^{64} блоків — місткість, що далеко перевищує будь-яку правдоподібну тривалість сеансу C2P-ARQ.

Криптографічні властивості етапу кондиціювання

Усунення зміщення

20-раундова двораундна ARX-перетворювальна схема ChaCha20 задовольняє Strict Avalanche Criterion: комплементція будь-якого одного біта K_{hw} змінює кожен біт B_0 з імовірністю ≈ 0.5 . Отже, статистична нерівномірність виходів Етапу 1 — групування рахунків ring oscillator, дисбаланс LSB ADC і повністю детермінований член `random()` — відображається у обчислювально рівномірний розподіл виходу незалежно від величини вхідного зміщення.

Обчислювальна невідрізнимість

За стандартним припущенням про псевдовипадкову функцію для ChaCha20-20, B_n є обчислювально невідрізним від рівномірно випадкового 512-бітового рядка для будь-якого ймовірного поліноміального супротивника. Отже, ефективна безпека виходу `getSecureRandom32()` становить $\min(H_{raw}, 256)$ бітів — вона обмежується саме бюджетом фізичної ентропії, що входить до Етапу 1, а не довжиною розширення keystream.

Пряма секретність між циклами живлення

Кожен цикл живлення виводить свіжий K_{hw} через Етап 1; усі попередні виходи CSPRNG є обчислювально недоступними з нового стану. Супротивник, який спостерігає поточний блок CSPRNG B_n , не може відновити жоден попсо сеансу, згенерований у попередньому циклі живлення, без інверсії ChaCha20.

Оновлення кешу завершується приблизно за 100 μ s на 16 MHz — один виклик блока ChaCha20 над 64 нульовими байтами — і не має помітного впливу на часові характеристики протоколу C2P-ARQ.

4.3 Неблокувальний доступ до CSPRNG під час виконання

Блокувальна вартість 16 ms, яку накладає `generateEntropyPool()` — вісім послідовних 2-ms вікон ring oscillator із синхронним читанням ADC та SRAM, — є архітектурно безпечною саме через момент її виконання. І `tx_setup()`, і `rx_setup()` викликають `generateEntropyPool()` після `Serial.begin()`, але до першого виклику

`loop()`. У цей момент UART-апаратура вже ініціалізована, але трафік пакетів ще не може надійти: сеанс C2P-ARQ не відкрито, жоден стан FSM не потребує супроводу тайм-аутів, і жоден UART-байт не очікується у буфері прийому. Отже, ці 16 ms повністю авансуються на фазу ініціалізації MCU і не впливають на жодну гарантію таймінгу протоколу.

Після повернення `initCSPRNG()` ISR ring oscillator `onRingOscPulse()` більше не під'єднується, а ADC не переводиться в режим free-running. Усі фізичні джерела ентропії, використані під час `generateEntropyPool()`, повністю звільнюються: INT0 не використовується, канал ADC 8 повертається до збереженого стану ADMUX, а A0 доступний для будь-якого аналогового застосування. Під час активної роботи протоколу фонового накопичення ентропії не відбувається. Від цього моменту CSPRNG працює як детермінований ChaCha20 keystream у режимі counter-mode, ініціалізований одноразовим стартовим harvest K_{hw} , як описано в [Section 4.2](#).

Єдиним споживачем ентропії під час runtime є `sendHelloPacket()`, який тричі поспіль викликає `getSecureRandom32()`, щоб заповнити 12-байтовий nonce сеансу `s_sessionNonce`. Ця функція викликається виключно зі стану `TxState::SENDING_HELLO` — короткого перехідного стану, що виконується один раз на SYN-handshake сеансу і один раз на кожную спробу auto-reconnect. Отже, виведення nonce відбувається саме тоді, коли потрібен новий ключ сеансу, і більше ніколи. Вузол приймача не викликає `getSecureRandom32()` у жоден момент роботи протоколу; його CSPRNG ініціалізується під час запуску для потенційного майбутнього використання, але не споживається шляхом прийому C2P-ARQ.

`getSecureRandom32()` є неблокувальним за будь-яких умов виконання. Звичайний шлях виконує читання чотирьох байтів безпосередньо з `s_keystreamBuf` — доступ до масиву та інкремент лічильника, що завершується за кілька AVR-циклів. Оновлення кешу автоматично запускається, коли `s_keystreamUsed + 4 > 64`, і потребує одного виклику блока ChaCha20 (~100 μ s на 16 MHz). Оскільки `sendHelloPacket()` бере лише 12 байтів (три виклики) на сеанс, а блоки по 64 байти генеруються під час кожного оновлення кешу, вартість refill амортизується до приблизно 4.7 μ s на один виклик `getSecureRandom32()` — мізерно порівняно з часом UART-кадру 9600 bps, який становить 1.04 ms на байт.

Архітектурний зв'язок між доменом ентропії та доменом протоколу є, отже, суворо односпрямованою залежністю: фізична ентропія тече з апаратних джерел під час запуску в K_{hw} , далі з K_{hw} — у двигун ChaCha20, а з нього — у session nonce за потреби. Жодна подія протоколу не повертає впливу назад у стан CSPRNG; жоден стан FSM не інспектує і не модифікує підсистему ентропії. Така односпрямована структура забезпечує формальний поділ доменів: доказ коректності або безпеки транспортного рівня C2P-ARQ може трактувати `getSecureRandom32()` як ідеальний

random oracle, оскільки його виходи обчислювально невідрізні від рівномірно випадкових значень за припущенням про псевдовипадкову функцію ChaCha20 незалежно від стану викликаючого FSM.

Forward secrecy між сеансами підтримується не поповненням ентропії, а стиранням nonce. Після кожного закриття сеансу — чи то через успішний FIN-handshake, чи через `suspendSession()` — `s_sessionNonce` перезаписується нулями (`memset(s_sessionNonce, 0, HELLO_NONCE_SIZE)`). Наступний сеанс використовує наступний блок keystream ChaCha20 для свого nonce, який є обчислювально незалежним від усіх попередніх блоків. Супротивник, що захопив B_n , не може відновити жоден nonce, похідний від:

$$B_0, \dots, B_{n-1}$$

без інверсії ChaCha20, що задовольняє backward secrecy. Супротивник, який захопив поточний стан CSPRNG, не може передбачити жоден майбутній nonce, якщо лічильник ChaCha20 просунувся — властивість, гарантована counter-mode конструкцією.

5 Специфікація протоколу C2P-ARQ

C2P-ARQ — це спеціальний станозалежний протокол рівня сеансу, який одночасно забезпечує надійну впорядковану доставку по ненадійному фізичному каналу з бітовими спотвореннями; автентифіковане шифрування кожного пакета, що запобігає повторному відтворенню, ін'єкції та прихованому спотворенню; трифазний життєвий цикл сеансу, який підтримує узгоджений криптографічний стан між передавачем і приймачем; і автоматичне відновлення сеансу, яке продовжує доставлення корисного навантаження після втрати каналу без втручання оператора. Увесь стек протоколу виконується в межах бюджету 2 KB SRAM мікроконтролера ATmega328P на 16 MHz — обмеження, що забороняє динамічне виділення пам'яті, розміщення таблиць сеансів у heap та черг retransmit з необмеженою глибиною. Увесь стан протоколу оголошено статично на етапі компіляції з розмірами, відомими компілятору.

Базова модель транспорту — Stop-and-Wait ARQ: передавач надсилає рівно один кадр, а потім блокує весь прогрес у стані `WAITING_* FSM` доти, доки не отримає явне однобайтове підтвердження або не вичерпає ліміт повторів для поточного стану (`MAX_RETRIES = 50`, `ACK_TIMEOUT_MS = 50 ms`). Ця модель відповідає UART-каналу 9600 bps: за швидкості приблизно один байт на 1.04 ms найбільший on-wire кадр — 17 байтів разом із preamble — займає близько 18 ms канального часу, залишаючи вікну ACK у 50 ms достатній запас на round-trip. Отже, тривалість worst-case retry storm обмежується [49, 50, 51, 52]:

$$50 \times 50 = 2500 \text{ ms}$$

Це значення навмисно обрано так, щоб воно було меншим за 3000 ms grace period приймального Smart Dynamic Watchdog ([Section 7.2](#)).

Асиметрична модель каналу є визначальною структурною властивістю C2P-ARQ, як показано в Figure 5.1. Канал прямої передачі даних, прокладений від Node A TX до Node B RX, проходить через XOR noise injector і підлягає довільним сплескам бітових інверсій; канал зворотного зв'язку, що працює від Node B TX до Node A RX, є прямим безшумним дротом, який переносить лише `ACK_BYTE (0x06)` і `NACK_BYTE (0x15)`. Рішення щодо надійності — чи приймати кадр, повторювати передачу або призупиняти сеанс — отже, ухвалюються на чистому сигналі незалежно від рівня шуму на прямому шляху. У наступних підрозділах специфіковано трифазний

життєвий цикл ([Section 5.1](#)), формат пакетів на дроті ([Section 5.2](#)) і механізм підтвердження та обмеження повторів ([Section 5.3](#)).

Figure 5.1: Асиметрична топологія UART-каналу C2P-ARQ у випробувальному стенді SimulIDE. Прямий канал даних піддається XOR-спотворенню бітів; зворотний канал, що переносить ACK_BYTE і NACK_BYTE, є прямим неатакованим проводом.

5.1 Життєвий цикл сеансу: обмін SYN, передавання даних (DAT), завершення (FIN)

Життєвий цикл сеансу C2P-ARQ складається з трьох строго впорядкованих фаз. Фаза SYN-обміну узгоджує криптографічний стан шляхом доставлення згенерованого CSPRNG 96-бітового попсе сеансу від передавача до приймача; цей попсе визначає кожен `per-packet IV` протягом усього сеансу і, таким чином, є єдиним станом, щодо якого обидва кінці мають досягти згоди до початку передавання даних. Фаза передавання DAT доставляє впорядковану послідовність корисного навантаження під Stop-and-Wait ARQ, де кожен кадр окремо автентифікується Poly1305 MAC, тег якого охоплює і шифротекст, і 3-байтовий заголовок у відкритому тексті. Фаза завершення FIN автентифікує межу сеансу та запускає негайне симетричне стирання попсе на обох вузлах, встановлюючи пряму секретність для всіх шифротекстів, захоплених під час сеансу.

5.1.1 SYN-обмін

Фаза SYN синхронізує криптографічний стан між двома вузлами. Її єдиний результат — 12-байтовий попсе сеансу, згенерований на передавачі та відтворений на приймачі.

$$N_s \in \{0, 1\}^{96}$$

У `TxState::IDLE` передавач опитує цифровий `pin 3 (INPUT_PULLUP)` із вікном `debounce 50 ms` на базі `millis()`. Після першого підтвердженого фронту натискання FSM скидає `melodyIndex = 0` і `seqNum = 0`, а потім переходить у `TxState::SENDING_HELLO`. У

стані `SENDING_HELLO` `sendHelloPacket()` тричі поспіль викликає `getSecureRandom32()`, щоб заповнити `s_sessionNonce[0..11]` трьома незалежними 32-бітовими словами CSPRNG; далі він викликає `drainRxFifo()` — споживаючи й відкидаючи всі байти, що наразі очікують у 64-байтовому апаратному FIFO приймача UART, — перед записом повного 15-байтового кадру в `Serial`: двох байтів `preamble` (0xAA, 0x55), байта прапорів `FLAG_SYN` (0x01) і дванадцяти байтів `nonce`. Одразу після передавання FSM переходить у `TxState::WAITING_HELLO_ACK` і фіксує `ackWaitStart = millis()`.

Виклик `drainRxFifo()` перед кожною передачею є захисним інваріантом протоколу: застарілі байти, накопичені в апаратному FIFO під час попереднього `idle`-або `retry`-інтервалу, інакше могли б бути помилково інтерпретовані як відповідь на щойно надісланий кадр, спричинивши хибний `storm` повторів або, у гіршому випадку, хибний `ACK_BYTE`, який просунув би FSM без реальної відповіді приймача.

На приймачі `processReceivedByte()` просуває чотиристановий байтовий парсер, ілюстрований у Figure 5.2, по одному байту за раз, використовуючи пару `preamble` 0xAA 0x55 як синхронізаційний шлюз. Коли парсер зібрав усі 13 байтів тіла `HelloPacket` у `rx_buffer` і виконується умова `rx_buffer[0] & FLAG_SYN`, він передає керування `processHelloBody()`. Ця функція копіює `rx_buffer[1..12]` до `s_sessionNonce`, викликає `s_cipher.setKey(MASTER_PSK, 32)`, щоб попередньо завантажити 256-бітовий PSK у AEAD-стан, встановлює `s_sessionActive = true`, записує `ACK_BYTE` (0x06) до `Serial` і ініціалізує `Dynamic Smart Watchdog` із `current_timeout_limit = 5000 ms` та `last_valid_packet_time = millis()`.

Figure 5.2: Шлях приймання SYN-обміну для кадрів `HelloPacket`. Лише `FLAG_SYN` переходить до `processHelloBody()`, який встановлює `session nonce`, надсилає `ACK_BYTE` та активує `Dynamic Smart Watchdog`.

Після отримання `ACK_BYTE` у `TxState::WAITING_HELLO_ACK` передавач переходить у `TxState::SENDING`. Якщо протягом `ACK_TIMEOUT_MS = 50 ms` `ACK_BYTE` не надходить, FSM повертається до `TxState::SENDING_HELLO` і повторно викликає `sendHelloPacket()` — що генерує свіжий CSPRNG `nonce` на кожному виклику, забезпечуючи монотонне просування вікна `nonce` незалежно від числа повторів. Після `MAX_RETRIES = 50` послідовних невдач викликається `suspendSession()`, і FSM входить у `TxState::RECONNECTING` із збереженням `melodyIndex`.

`Nonce` сеансу N_s передається у відкритому тексті, оскільки він не є секретом: він функціонує як публічний IV, єдина мета якого — гарантувати семантичну безпеку, тобто те, що шифрування тієї самої ноти двічі в межах одного сеансу дає різні

шифротексти. Конфіденційність сеансу повністю спирається на `MASTER_PSK`, який вбудовано в обидва образи прошивки і ніколи не передається жодним каналом.

5.1.2 Фаза DAT

Фаза DAT реалізує Stop-and-Wait ARQ із повним ChaCha20-Poly1305 AEAD для кожного пакета. Її центральний інваріант — прив'язка `seqNum`: кожен пакет, що досягає приймача, містить унікальний `seqNum`, а per-packet nonce детерміновано виводиться з цього `seqNum` — забезпечуючи, що AEAD-контекст ніколи не повторно використовується для будь-яких двох успішно підтверджених кадрів в межах одного сеансу.

Виведення nonce для кожного пакета

Для пакета з номером послідовності i (де i — поточний `seqNum`) per-packet IV виводиться з локальної копії N_s зі словами в позиціях 10 і 11, XOR-еними з високим і низьким байтами i відповідно:

$$IV_i[j] = \begin{cases} N_s[j] & 0 \leq j \leq 9 \\ N_s[10] \oplus \lfloor i / 256 \rfloor & j = 10 \\ N_s[11] \oplus (i \bmod 256) & j = 11 \end{cases}$$

Оригінальний `s_sessionNonce` ніколи не модифікується; XOR застосовується виключно до копії, розміщеної у стеку `packetNonce[HELLO_NONCE_SIZE]`. Відображення є ін'єктивним на 16-бітовому домені:

$$i \mapsto IV_i$$

$$i \in [0, 65535]$$

Якщо

$$i \neq i'$$

then:

$$((i \bmod 256) \neq (i' \bmod 256)) \vee (\lfloor i/256 \rfloor \neq \lfloor i'/256 \rfloor)$$

and therefore:

$$IV_i \neq IV_{i'}$$

Усі 65,536 можливих значень номера послідовності, таким чином, породжують відмінний 96-бітовий nonce, і в межах одного сеансу повторення IV не може виникнути незалежно від змісту повідомлення чи тривалості сеансу. 16-бітовий діапазон i цілком перевищує практичну верхню межу, зумовлену бюджетом Flash ATmega328P: мелодія Super Mario Bros., збережена в `PROGMEM`, містить 80 нот, що значно менше за межу в 65,536 пакетів.

Конвеєр AEAD

У `TxState::SENDING` функція `sendPacket()` виконує такий конвеєр: (1) `s_cipher.clear()`, `s_cipher.setKey(MASTER_PSK, 32)`, `s_cipher.setIV(packetNonce, 12)`; (2) `s_cipher.addAuthData({FLAG_DAT, seq_lo, seq_hi}, 3)`, який хешує 3-байтовий заголовок відкритого тексту в Poly1305 accumulator без його шифрування; (3) `s_cipher.encrypt(plaintext, ciphertext, 4)`, шифруючи 4-байтове корисне навантаження `{note_index, duration_ms}`; (4) `s_cipher.computeTag(fullMac, 16)`, з якого лише перші 8 байтів (`TRUNCATED_MAC_SIZE = 8`) додаються до поля `DataPacket.mac` і передаються. Симетричний конвеєр виконується на приймачі всередині `authenticateAndPlay()`, де `decrypt()` виконується перед `computeTag()`, а перевірка здійснюється через constant-time `memcmp` із отриманим 8-байтовим тегом; `startNote()` викликається лише після того, як `memcmp` повертає нуль. Прив'язка байта прапорів і обох байтів `seq_num` до AAD означає, що будь-який однобітовий шумовий сплеск, який впливає на будь-яке з цих полів, гарантує MAC failure незалежно від вмісту `ciphertext`. Повний формат дроту `DataPacket` наведено в [Section 5.2](#).

Підтвердження та просування номера послідовності

Після кожного передавання FSM чекає в `TxState::WAITING_ACK`. На `ACK_BYTE` FSM переходить у `TxState::WAIT_BETWEEN_NOTES`, де він чекає `pendingNoteDuration` мілісекунд перед виконанням `melodyIndex++`; `seqNum++`. Це єдиний шлях у прошивці, де `seqNum` інкрементується — він ніколи не просувається на NACK або під час ACK timeout. Якщо після інкременту `melodyIndex` вичерпав таблицю мелодії, FSM переходить у `TxState::SENDING_FIN`; інакше він повертається до `TxState::SENDING` для наступної ноти.

На NACK або ACK timeout FSM повторно передає знімок (`pendingNoteIndex`, `pendingNoteDuration`) з тим самим `seqNum`. Така idempotency коректна, оскільки одна й та сама пара (N_s, i) завжди породжує ідентичний `ciphertext`: приймач, отримавши

повторну передачу, виконує той самий AEAD pipeline після попередньої MAC-помилки і, якщо повторно переданий кадр не пошкоджено, отримує коректний результат `memcmp` і надсилає `ACK_BYTE`. Після `MAX_RETRIES = 50` послідовних невдач викликається `suspendSession()`, який нулює `s_sessionNonce`, зберігає `melodyIndex` і переходить у `TxState::RECONNECTING`.

5.1.3 Завершення FIN

Коли `TxState::WAIT_BETWEEN_NOTES` виявляє, що всі ноти підтверджено, FSM переходить у `TxState::SENDING_FIN`. Функція `sendFinPacket()` створює `DataPacket` з `flags = FLAG_FIN (0x04)` та повністю нульовим корисним навантаженням `{0, 0}`, після чого виконує повний AEAD конвеєр з поточним `seqNum` і AAD `{FLAG_FIN, seq_lo, seq_hi}`. Отриманий автентифікований кадр із нульовим корисним навантаженням є криптографічно підписаним сигналом завершення сеансу: будь-який шумовий шаблон `0x04` у потоці даних провалить перевірку MAC на приймачі й викличе `NACK`, залишаючи сеанс відкритим для повторної передачі FIN.

На приймачі `processFinPacket()` виконує ідентичний AEAD конвеєр. Структурною вимогою ChaCha20-Poly1305 є те, що `decrypt()` має бути викликаний перед `computeTag()` навіть тоді, коли plaintext відкидається: Poly1305 accumulator інкорпорує ciphertext bytes як побічний ефект `decrypt()`; пропуск цього виклику залишає accumulator у неправильному проміжному стані й породжує хибний MAC mismatch для легітимного FIN. Реалізація `processFinPacket()` виконує цю вимогу, розшифровуючи чотири нульові байти у локальний масив `discardBuf[DATA_PAYLOAD_SIZE]` перед викликом `computeTag()`.

На MAC success приймач (1) записує `ACK_BYTE` до `Serial`, (2) викликає `memset(s_sessionNonce, 0, HELLO_NONCE_SIZE)`, (3) встановлює `s_sessionActive = false` і (4) викликає `resetParser()`. Після отримання `ACK_BYTE` у `TxState::WAITING_FIN_ACK` передавач (1) симетрично викликає `memset(s_sessionNonce, 0, HELLO_NONCE_SIZE)`, (2) скидає `melodyIndex = 0` і `seqNum = 0` та (3) переходить у `TxState::IDLE`. Після завершення сеансу єдиним сталим секретним матеріалом у системі є `MASTER_PSK` — вбудований в обидва образи прошивки й ніколи не переданий, не залогований і не відображений на жодному вихідному каналі. Жоден вузол не зберігає session nonce, і жоден блок ciphertext, захоплений із завершеного сеансу, не може бути відновлений без знання `MASTER_PSK`. Формальні властивості безпеки та forward secrecy, що виникають із стирання nonce, аналізуються в [Section 6.3](#).

5.1.4 Повна діаграма послідовності сеансу

Повний життєвий цикл C2P-ARQ показано в Figure 5.3. SYN handshake (Phase 1) синхронізує session nonce; DAT transfer phase (Phase 2) демонструє і нормальний шлях ACK, і цикл повторної передачі, викликаний MAC failure; FIN teardown (Phase 3) показує автентифіковане завершення й симетричне стирання nonce. Зверніть увагу, що seqNum просувається виключно в TxState::WAIT_BETWEEN_NOTES — ACK-success path — і ніколи не інкрементується на NACK receipt або ACK timeout.

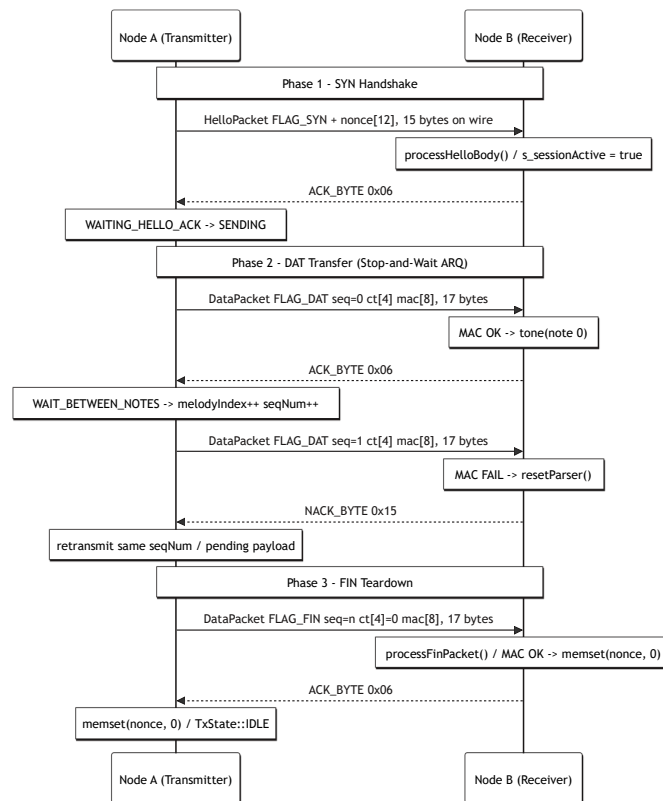


Figure 5.3: Повна послідовність життєвого циклу C2P-ARQ через SYN handshake, передавання даних Stop-and-Wait ARQ із повторною передачею та FIN teardown із симетричним стиранням nonce на обох вузлах.

5.2 Формат кадрів: структури HelloPacket і DataPacket

Усі кадри C2P-ARQ мають двобайтовий preamble синхронізації — SYNC_BYTE_1 = 0xAA, за яким слідує SYNC_BYTE_2 = 0x55 — і цей preamble передувє кожному пакету на forward data channel. Preamble не є полем жодної packet struct; передавач випромінює його двома явними викликами Serial.write() безпосередньо перед

записом `packed struct body`, а байтовий парсер приймача використовує пару станів `WAIT_AA` — `WAIT_55`, як показано в Figure 5.2, щоб відновити вирівнювання межі кадру після будь-якої шумової десинхронізації. Використання двох різних некоплементарних байтів — замість одного повторюваного байта — зменшує частоту хибного захоплення: 16-бітовий шаблон `0xAA55` не може виникнути внаслідок одиначної бітової інверсії ні з `0xAAAA`, ні з `0x5555`, тож ізольовані шумові імпульси не здатні створити хибний lock на межі кадру.

Обидва типи пакетів оголошено `packed` через `#pragma pack(push, 1) / #pragma pack(pop)` в обох образах прошивки. На AVR природне вирівнювання полів `uint8_t` і `uint16_t` уже дає байтово-орієнтовану структуру без вставок `padding`, але `pragma` зберігається як архітектурно незалежна гарантія: `in-memory C struct layout` точно збігається з `on-wire byte sequence`, роблячи `packed struct body` безпосередньо `castable` до і з `hardware UART buffer` без будь-якого `explicit serialisation step`.

5.2.1 AAD: Автентифікований відкритий заголовок

Перші три байти кожного `DataPacket body` — байт `flags` (1 byte) і little-endian `seq_num` (2 bytes) — передаються у відкритому тексті й становлять Additional Authenticated Data (AAD) конструкції ChaCha20-Poly1305 AEAD. Poly1305 tag τ обчислюється над конкатенацією AAD, ciphertext, їхніх 16-byte-aligned padding blocks і encoded lengths — стандартна IETF ChaCha20-Poly1305 конструкція:

$$\tau = \text{Poly1305}_{K_{\text{MAC}}}(\text{AAD} \parallel \text{pad}_{16}(\text{AAD}) \parallel \text{ciphertext} \parallel \text{pad}_{16}(\text{ciphertext}) \parallel \text{len}_{64}(\text{AAD}) \parallel \text{len}_{64}(\text{ciphertext}))$$

де K_{MAC} — 32-byte one-time key, похідний від block 0 ChaCha20 keystream під per-packet nonce IV_i . Оскільки `flags` і `seq_num` прив'язуються до Poly1305 computation, будь-яка single-bit corruption у будь-якому з цих полів — чи то від noise injector, чи то від активного супротивника, який намагається змінити frame reclassification або reordering — робить τ недійсним з імовірністю $1 - 2^{-64}$, що змушує constant-time `memcmp` провалитися та викликає `NACK_BYTE` і retransmission. Жодне з цих полів не потребує шифрування: їхню цілісність забезпечує MAC, а не конфіденційність, і жодне не містить секретних даних.

`HelloPacket` не містить AAD і не використовує AEAD construction. Його поле `nonce` передається у відкритому тексті, оскільки session IV є публічним параметром, єдина роль якого — забезпечити semantic security між сеансами; знання nonce не дає супротивнику жодної переваги у відновленні `MASTER_PSK`.

5.2.2 HelloPacket — 15 байтів на дроті

`HelloPacket` struct — це packed 13-byte body, якому передуює 2-byte preamble, тож загалом він займає 15 bytes on the wire. Він містить два поля. Байт `flags` несе `FLAG_SYN = 0x01` і є frame-type discriminator; приймач зіставляє його за допомогою bitwise-AND predicate `rx_buffer[0] & FLAG_SYN` замість strict equality, зберігаючи толерантність до майбутніх multi-role flag combinations без модифікації parser. Масив `nonce[HELLO_NONCE_SIZE]` несе 12-byte (96-bit) CSPRNG session IV, згенерований на передавачі трьома послідовними викликами `getSecureRandom32()`, по одному виклику на 32-bit word, заповнюючи повний 12-byte масив. Повний byte-level wire layout показано в Figure 5.4.

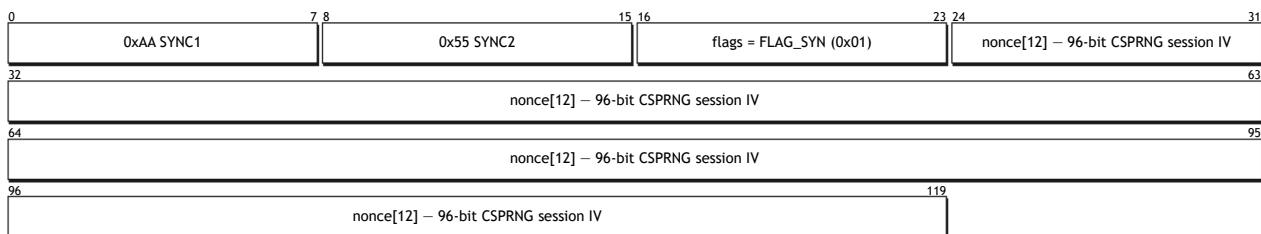


Figure 5.4: Побайтове розміщення `HelloPacket` на дроті у форматі кадру C2P-ARQ. 2-байтову преамбулу передають окремо, після чого йде 13-байтове упаковане тіло, що несе `FLAG_SYN` і `nonce[12]`.

Поле `nonce[12]` регенерується під час кожного виклику `sendHelloPacket()`, зокрема під час кожної reconnect attempt у `TxState::RECONNECTING`. Тож retransmitted SYN frames несуть свіжий nonce замість replay попереднього, просуваючи CSPRNG output монотонно через усі retry cycles і гарантуючи, що `melodyIndex`, збережений через `suspendSession()`, завжди поєднується з freshly derived session key, коли канал встановлено знову.

5.2.3 DataPacket — 17 байтів на дроті

`DataPacket` struct — це packed 15-byte body, якому передуює 2-byte preamble, тож загалом він займає 17 bytes on the wire. Він використовується як для `FLAG_DAT = 0x02` (data transfer), так і для `FLAG_FIN = 0x04` (session teardown); AEAD pipeline в обох випадках ідентичний. Struct містить чотири поля.

Поле `flags` (1 byte) ідентифікує роль frame і формує AAD byte 0. Поле `seq_num` (2 bytes, `uint16_t`, little-endian) є 16-bit packet sequence number і формує AAD bytes 1–2; воно також слугує входом для per-packet nonce derivation, описаного в [Section 5.1](#).

Разом `flags` і `seq_num` утворюють 3-byte AAD region, криптографічно прив'язану до ciphertext через Poly1305 tag. Поле `payload` (4 bytes, `uint16_t[2]`) містить ChaCha20 stream-cipher encryption пари `{note_index, duration_ms}`, зашифрованої атомарно одним викликом `s_cipher.encrypt()` довжиною `DATA_PAYLOAD_SIZE = 4`. Поле `mac` (8 bytes, `uint8_t[TRUNCATED_MAC_SIZE]`) містить перші `TRUNCATED_MAC_SIZE = 8` bytes із 16-byte Poly1305 output; решта 8 bytes обчислюються всередині `s_cipher.computeTag()`, але не передаються, тож імовірність forgerу для одного кадру становить 2^{-64} . Повний byte-level wire layout показано в Figure 5.5.

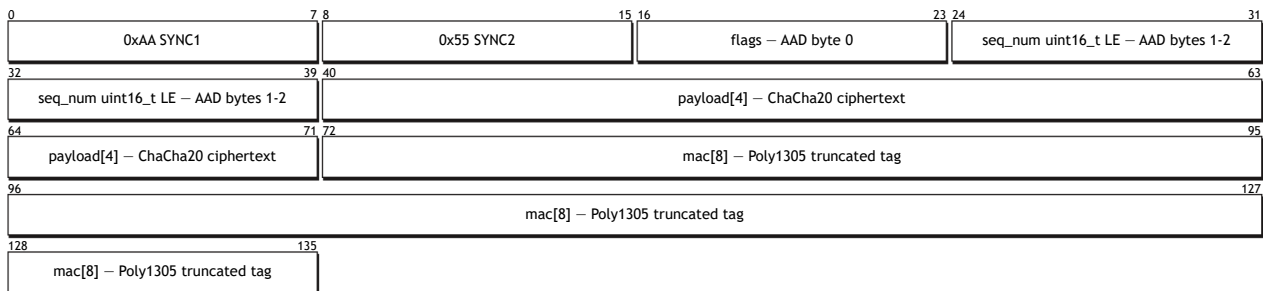


Figure 5.5: Побайтове розміщення DataPacket на дроті у форматі кадру C2P-ARQ. 3-байтовий AAD-регіон (`flags` + `seq_num`) автентифікується тегом Poly1305, але не шифрується, а 4-байтовий `payload` є шифротекстом ChaCha20.

Для `FLAG_FIN` frames поле `payload` містить ChaCha20 encryption нульового plaintext `{0, 0}` — двох `uint16_t` значень, що дорівнюють нулю. На приймачі `processFinPacket()` decrypts чотири ciphertext bytes у локальний масив `discardBuf[DATA_PAYLOAD_SIZE]`; значення, отримані після decrypt, відкидаються, але виклик `s_cipher.decrypt()` є архітектурно обов'язковим, оскільки Poly1305 accumulator інкорпорує ciphertext bytes як побічний ефект decryption. Пропуск цього виклику перед `s_cipher.computeTag()` залишає accumulator у неправильному проміжному стані й породжує false MAC mismatch для криптографічно коректного FIN frame — `discardBuf` pattern, який будь-який майбутній порт або розширення firmware приймача має відтворити точно.

5.3 Механізм підтвердження та запобігання неконтрольованим повторним спробам

5.3.1 Контур зворотного зв'язку ARQ

Гарантія надійності C2P-ARQ ґрунтується на двобайтовому feedback vocabulary, визначеному в `protocol.h`. Байт `ACK_BYTE = 0x06` сигналізує, що перевірка ChaCha20-Poly1305 MAC на приймачі успішна: ciphertext дійшов без корупції, Poly1305 tag збігся з незалежно обчисленим очікуваним значенням, і decoded note поставлено в чергу на відтворення. Байт `NACK_BYTE = 0x15` сигналізує про протилежний результат: приймач обчислив Poly1305 tag, який не збігся з отриманим 8-byte truncated MAC, і payload був відкинутий повністю без дії на будь-яке поле, включно з bound-check `noteIndex`. Значення `0x06` і `0x15` розділені Hamming distance у три біти, що зменшує ймовірність того, що один шумовий біт на nominally clean feedback channel перетворить одне значення в інше.

```
1 // TransmitterNode/protocol.h (shared C2P-ARQ constants on TX and RX)
2 const uint8_t ACK_BYTE = 0x06;
3 const uint8_t NACK_BYTE = 0x15;
4
5 const uint32_t ACK_TIMEOUT_MS = 50UL;
6 const uint8_t MAX_RETRIES = 50u;
```

Listing 1: ACK/NACK control bytes and ARQ retry limits

Тут `ACK_BYTE/NACK_BYTE` — це feedback control bytes, тоді як `ACK_TIMEOUT_MS` і `MAX_RETRIES` визначають Stop-and-Wait ARQ retry policy.

У межах `TxState::WAITING_ACK` функція `tx_loop()` на кожній ітерації викликає `Serial.available()`. Якщо байт присутній, він читається і тестується проти обох control values. Отримання `ACK_BYTE` скидає `retryCount` до нуля, записує `noteWaitStart = millis()` і переводить FSM у `TxState::WAIT_BETWEEN_NOTES`. Будь-який байт, який не є ані `ACK_BYTE`, ані `NACK_BYTE`, мовчки відкидається; передавач залишається в `WAITING_ACK`, а вікно ACK timeout продовжує відраховуватися. Отримання `NACK_BYTE` сигналізує MAC failure і вимагає retransmission того самого пакета з тим самим se-

приймачі XOR-ить `seqNum` у базовий `session nonce` ідентично до того, як це робить передавач, тож приймач очікує той самий AEAD IV на наступній спробі `retransmission`. Просування `seqNum` на NACK породило б IV mismatch і гарантовану лавину MAC failures на всіх наступних пакетах. Номер послідовності просувається лише в `TxState::WAIT_BETWEEN_NOTES`, одночасно з `melodyIndex++`, і більше ніде в життєвому циклі FSM.

5.3.2 Умова тайм-ауту і інкремент повтору

Zero-byte-received branch у `WAITING_ACK` реалізує неблокувальний ACK timeout через `millis()` delta comparison:

$$(t_{\text{now}} - t_{\text{send}}) \geq \text{ACK_TIMEOUT_MS}$$

де

$$t_{\text{now}} = \text{millis()}, \quad t_{\text{send}} = \text{ackWaitStart}$$

а `ackWaitStart` — це snapshot `millis()`, узятий одразу після завершення `formAndSendPacket()`. Коли умова виконується, retry counter R інкрементується на одиницю, і — за умови, що guard проти runaway спрацює — той самий пакет `retransmitted`, а `ackWaitStart` скидається, щоб перезапустити вікно.

І шлях NACK, і шлях timeout інкрементують R , але вони відрізняються однією оборонною деталлю. На шляху NACK перед `retransmission` вставляється `delay(10)` call. Ця коротка пауза є цільовим defensive invariant: NACK означає, що parser приймача щойно видав response byte на feedback channel, за яким через мікросекунди можуть іти залишкові noise pulses, що ще поширюються UART-лінією. 10 ms drain window гарантує, що будь-які такі хвостові байти встигають затихнути до того, як `retransmitted packet` прибуде, запобігаючи тому, щоб застарілий байт був помилково прочитаний як ACK для `retransmission`. На timeout path жодна drain pause не вставляється — жодного feedback byte не було отримано, тож feedback FIFO уже порожній, і `retransmission` відбувається негайно, зберігаючи якнайбільше з 50 ms window для наступної відповіді.

5.3.3 Запобігання неконтрольованим повторним спробам і призупинення самовідновлення

Без жорсткої верхньої межі на R тривалий jamming event втримував би передавач у `WAITING_ACK` нескінченно: кожен `retransmission` був би спотворений, кожен ACK був

би втрачений, і жоден endpoint не міг би вийти зі свого поточного стану через будь-яку подію каналу, досягну за безперервної інтерференції. Це становить protocol-layer deadlock, з якого неможливо вийти без explicit out-of-band support. C2P-ARQ усуває цей deadlock, примушуючи таку умову перед кожним retransmission:

$$R < MAX_RETRIES$$

Коли guard провалюється:

$$R \geq MAX_RETRIES$$

`suspendSession()` викликається безумовно, і retransmission path не входить.

`suspendSession()` виконує чотири операції у точно визначеному порядку. По-перше, `memset(s_sessionNonce, 0x00, HELLO_NONCE_SIZE)` стирає 12-byte session nonce з SRAM. Це дія forward secrecy: старий nonce криптографічно анулюється, тож знання post-suspension SRAM image не може відновити жоден ciphertext із перерваного сеансу. По-друге, і `seqNum`, і `retryCount` скидаються до нуля, очищаючи весь ARQ accounting state. По-третє — і критично — `melodyIndex` зберігається без змін. Це визначальний self-healing invariant SH2SC-EDT: передавач зберігає точну позицію в потному ряді, на якій сталася перерва, і передавання даних буде продовжено від `melodyIndex` після reconnection, а не з початку з індексу нуля. По-четверте, `currentState` встановлюється в `TxState::RECONNECTING`, а `lastReconnectAttempt` попередньо озброюється до `millis()`, плануючи перше автоматичне випромінювання `HelloPacket` рівно через `RECONNECT_INTERVAL_MS = 2000 ms`. Блокувальний `delay(2000)` утримує повідомлення LCD “LINK LOST / RECONNECTING...” — єдиний прийнятний випадок використання `delay()` поза `setup()`, виправданий його суто операторською візуалізацією та розміщенням у стані, який не має UART receive obligation.

Найгірша тривалість retry storm, яка вичерпує `MAX_RETRIES`, обмежується:

$$\begin{aligned} T_{\text{storm}} &= MAX_RETRIES \times ACK_TIMEOUT_MS \\ &= 50 \times 50 \text{ ms} = 2500 \text{ ms} \end{aligned}$$

Ця межа не є довільною: вона constrained by receiver’s `NETWORK_GRACE_PERIOD_MS = 3000 ms` Smart Dynamic Watchdog period. Після кожного authenticated `DataPacket` приймач встановлює timeout звільнення сеансу як `current_timeout_limit = duration_ms + NETWORK_GRACE_PERIOD_MS`. Наступна нерівність гарантує, що приймач не занулить `s_sessionNonce` і не повернеться до `WAIT_AA` раніше, ніж передавач завершить свій повний retry cycle і перейде у `RECONNECTING`:

$$T_{\text{storm}} < \text{NETWORK_GRACE_PERIOD_MS}$$

Explicitly:

$$2500 < 3000$$

Якби приймач звільнив nonce buffer передчасно, перший `DataPacket`, надісланий після успішного reconnect handshake, провалив би MAC verification — бо приймач уже стер би `s_sessionNonce` і не зміг би вивести правильний per-packet IV — що змусило б здійснити ще один SYN round-trip і підірвало б seamlessness Auto-Resume. Margin у 500 ms усуває цю race condition конструктивно. Повна decision logic для `TxState::WAITING_ACK`, включно з усіма трьома event branches і runaway-prevention gate, показана в Figure 5.6.

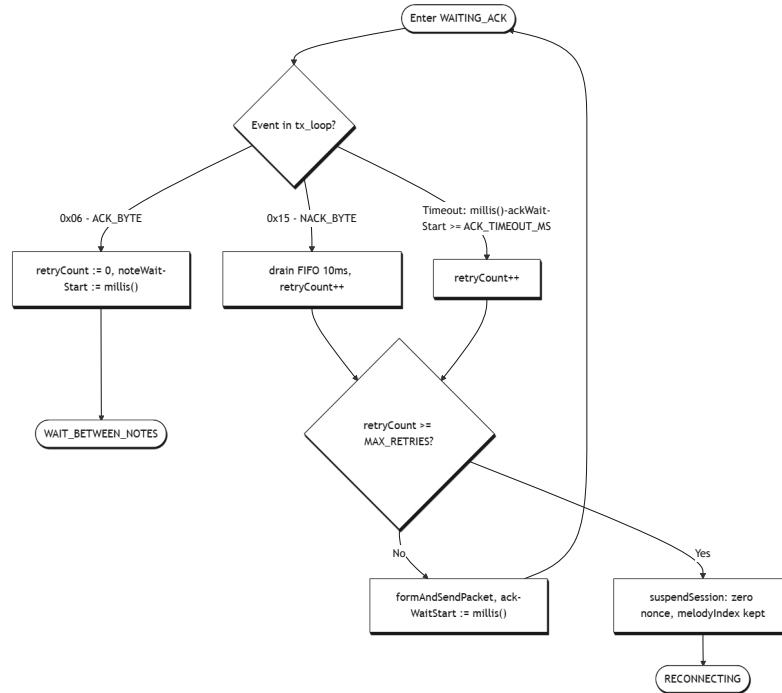


Figure 5.6: Дерево рішень FSM для оброблення підтвердження у стані `WAITING_ACK`. Гілки `NACK` і `timeout` збільшують лічильник повторів і спільно проходять через запобіжний бар'єр від runaway; лише гілка `NACK` вставляє 10-мс паузу `drain FIFO` перед повторною передачею. За провалу бар'єра `suspendSession()` обнуляє session nonce для forward secrecy та переходить до `RECONNECTING`, зберігаючи `melodyIndex` — інваріант self-healing.

6 Модель безпеки та криптографічне ядро

Безпеку C2P-ARQ аналізують у стандартній моделі мережевого супротивника Dolev-Yao [8]: супротивник займає communication channel між Node A і Node B як повноцінний man-in-the-middle і отримує необмежені можливості над forward data channel. Зокрема, супротивник може спостерігати кожен transmitted frame повністю, replay будь-який раніше захоплений frame у будь-який майбутній момент, delay або reorder frames довільно, silently drop будь-який frame і inject frames будь-якого вмісту в будь-яку байтову позицію. Ці можливості точно моделюють threat surface, створюваний SimulIDE XOR noise injector — hardware switch, що піддає кожен біт forward data channel довільному bit-flip під керуванням оператора; модель Dolev-Yao є суворішою, оскільки супротивник може craft targeted bit patterns, а не лише XOR із фіксованою waveform, що робить SimulIDE експеримент консервативною оцінкою протоколу, спроектованого під сильнішу threat standard. [9, 10]

Два обмежувальні припущення визначають криптографічний trust boundary. По-перше, стандартна модель криптографічної ідеалізації: ChaCha20 моделюється як pseudorandom permutation, computationally indistinguishable from a uniform random permutation; Poly1305 моделюється як computationally secure one-time MAC; жоден polynomial-time adversary не може зламати жодну primitive за стандартних PRF/MAC assumptions. По-друге, фізична ізоляція пам'яті: супротивник не може читати SRAM contents безпосередньо з жодного вузла — MASTER_PSK і s_sessionNonce живуть у firmware-controlled memory, ніколи не потокуються в будь-який output channel і не експонуються через жодний communication interface.

За цих припущень session-layer security кожного C2P-ARQ сеансу зводиться до секрету єдиного 256-bit MASTER_PSK, скомпільованого в обидва firmware images. PSK ніколи не передається, ніколи не друкується на Serial або Aip31068 LCD і ніколи не виводиться з жодного observable parameter. Це єдиний shared secret, з якого випливає вся cryptographic authority для кожного per-session і per-packet обміну: супротивник без MASTER_PSK стикається з обчислювально нездійсненною forgery, decryption і session-hijacking against any frame produced by the AEAD pipeline. Три наступні розділи формально встановлюють властивості, які C2P-ARQ надає в цій моделі: per-packet AEAD authentication ([Section 6.1](#)), nonce injectivity і sequence-number binding ([Section 6.2](#)), та forward-secrecy nonce erasure ([Section 6.3](#)). [11, 12, 13]

6.1 Автентифіковане шифрування з даними супроводу (ChaCha20-Poly1305)

6.1.1 Чому цілісність домінує над конфіденційністю для каналів керування

Інтуїтивний, але неповний design для embedded command channel застосовує stream cipher лише до payload field, забезпечуючи конфіденційність без цілісності. Така конструкція фундаментально недостатня проти активного супротивника: stream-cipher ciphertext є additively malleable. Якщо супротивник перевертає біт k 4-byte ciphertext block у `DataPacket.payload`, біт k відповідного decrypted byte flips identically after decryption, незалежно від ключа. У payload C2P-ARQ `note_index` займає bytes 0–1, а `duration_ms` — bytes 2–3 (обидва `uint16_t`, little-endian). Одиначний bit-flip у byte 0 ciphertext завжди дає структурно валідний, але інший `note_index` після decryption — приймач не має механізму виявити substitution. Супротивник, який спостерігає один plaintext session, відновлює complete payload field layout і надалі може синтезувати довільні command substitutions over encrypted channel без знання MASTER_PSK. Це bit-flip injection variant plaintext-injection атаки, визначеної в [Section 1.1](#). [34, 35]

AEAD усуває цей клас атак на MAC gate. Poly1305 tag τ є commitment до exact byte sequence автентифікованого frame: будь-яка single-bit modification ciphertext або plaintext header породжує — з імовірністю $1 - 2^{-64}$ — інший expected tag на приймачі, змушуючи constant-time memcmp провалитися. Security invariant, enforced by `authenticateAndPlay()`, є абсолютним: `startNote()` виконується тоді й лише тоді, коли memcmp повертає нуль. Жодне поле MAC-failed packet не впливає на observable state приймача, і жодне часткове виконання note-play path не відбувається під час спроби підробки. Тож integrity у C2P-ARQ не додається до confidentiality як окрема властивість — вона структурно передує їй: без MAC gate супротивник може ін'єктувати команди через ciphertext malleability незалежно від того, чи зашифровано payload.

6.1.2 Роль додаткових автентифікованих даних — прив'язка заголовка кадру

Ciphertext-only authentication захищало б зашифрований payload, але лишало б 3-byte plaintext header — `flags` (1 byte) і `seq_num` (2 bytes) — не захищеним.

Супротивник, який перехопив `FLAG_DAT` frame, може перевернути bit 1 у `byte flags`, перетворивши 0x02 на 0x04 (`FLAG_FIN`) і таким чином перекласифікувати `data-transfer` frame на `session-teardown` command. Незмінений ciphertext і 8-byte MAC пройдуть `verification`, оскільки `tag` було обчислено на оригінальному контексті `FLAG_DAT`. Приймач виконає `processFinPacket()`, `erase s_sessionNonce` і завершить сеанс — примусове, неавтентифіковане `session-close` через `replayed partially modified frame`.

C2P-ARQ запобігає цьому через явне AAD binding. У `sendPacket()` виклик `s_cipher.addAuthData(aadBuf, 3)` подає 3-byte local array `{flags, seq_lo, seq_hi}` у Poly1305 accumulator перед тим, як `encrypt()` обробляє `payload`. Poly1305 construction незворотна: accumulator зміщує AAD, ciphertext і length encoding в єдине 128-bit commitment. Симетричний виклик на приймачі, `s_cipher.addAuthData(rxAad, 3)`, подає той самий triple полів із `received frame`. Будь-яка невідповідність між `flags` або `seq_num`, побаченими на передавачі, і тими, що побачив приймач — однобітова зміна в будь-якому з трьох AAD bytes — породжує інший `expected tag`, що примушує `NACK_BYTE`. Жодне поле не може бути змінено без руйнування Poly1305 commitment.

6.1.3 Конвеєр шифрування та автентифікації

Повний per-packet AEAD pipeline — виконуваний `sendPacket()` на передавачі і віддзеркалений `authenticateAndPlay()` на приймачі — складається з п'яти послідовних стадій: (1) `s_cipher.clear()`, `s_cipher.setKey(MASTER_PSK, 32)`, `s_cipher.setIV(packetNonce, 12)` ініціалізує новий ChaChaPoly instance для кожного пакета з 256-bit PSK і per-packet nonce IV_i , запобігаючи будь-якому state carry-over між frame; (2) `s_cipher.addAuthData(aadBuf, 3)` подає 3-byte AAD у Poly1305 accumulator; (3) `s_cipher.encrypt(plaintext, ciphertext, DATA_PAYLOAD_SIZE)` шифрує 4-byte payload через ChaCha20 counter-block keystream і одночасно подає отриманий ciphertext у Poly1305 accumulator; (4) `s_cipher.computeTag(fullMac, AUTH_TAG_SIZE)` фіналізує Poly1305 accumulator з IETF length-encoding block і видає full 128-bit tag; (5) лише перші `TRUNCATED_MAC_SIZE = 8 bytes fullMac` додаються до `DataPacket.mac` і виходять на дріт. [36, 37, 38, 39, 40]

Трансмісійний AEAD pipeline підсумовано в Figure 6.1.

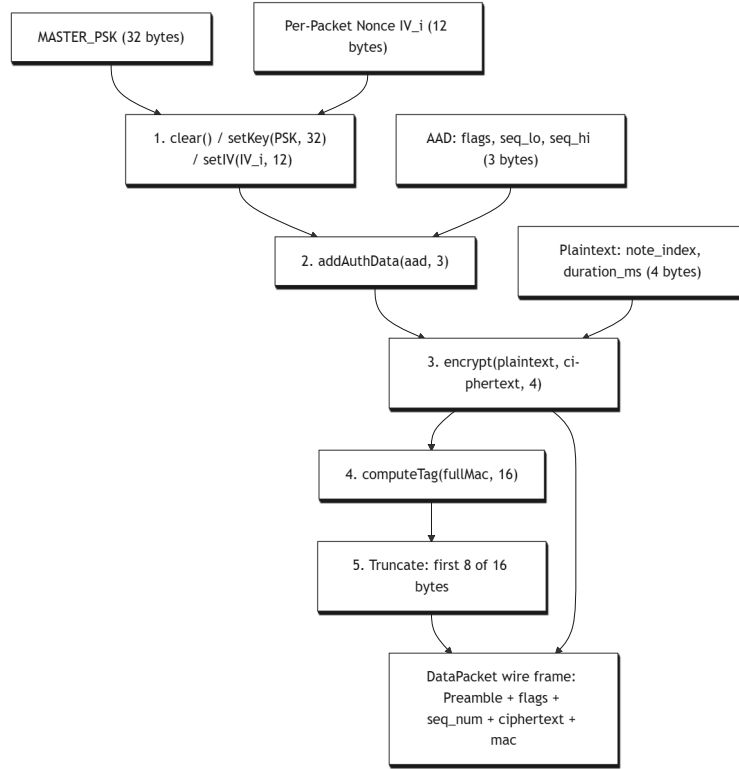


Figure 6.1: TX-пайплайн ChaCha20-Poly1305 AEAD, який виконує `sendPacket()` у C2P-ARQ. Кожен пакет використовує новий екземпляр ChaChaPoly, автентифікує 3-байтовий заголовок plaintext як AAD, шифрує 4-байтовий payload та передає лише перші 8 байтів тегу Poly1305.

6.1.4 Формальний предикат верифікації

Receiver authentication gate формалізується як детермінований бінарний predicate. Нехай K позначає `MASTER_PSK`, IV_i — per-packet nonce, похідний для sequence number i , A — 3-byte AAD vector `{flags, seq_lo, seq_hi}`, C — 4-byte received ciphertext у `DataPacket.payload`, а τ_r — received 8-byte truncated MAC у `DataPacket.mac`. Функція AEAD decryption-and-verification, виконувана `authenticateAndPlay()`, повертає або 4-byte plaintext P , або символ відхилення $\perp$:

$$\text{Verify}(K, IV_i, A, C, \tau_r) \rightarrow \{P, \perp\}$$

Нехай

$$au_e \in \{0, 1\}^{128}$$

позначає full 16-byte expected tag, виданий `s_cipher.computeTag(expectedMac, 16)` після виконання симетричного AEAD pipeline над

$$(K, IV_i, A, C)$$

Нехай $\text{trunc}_8(\tau_e)$ позначає перші $\text{TRUNCATED_MAC_SIZE} = 8$ bytes від τ_e . Результат є таким:

$$\text{Verify}(K, IV_i, A, C, \tau_r) = \begin{cases} P & \text{if } \text{trunc}_8(\tau_e) = \tau_r \\ \perp & \text{otherwise} \end{cases}$$

де equality перевіряється через constant-time `memcmp(expectedMac, pkt->mac, TRUNCATED_MAC_SIZE)` для запобігання timing side-channel leakage. У success branch функція повертає $P = \{\text{note_index}, \text{duration_ms}\}$; failure branch emits `NACK_BYTE` на feedback channel і викликає `resetParser()`. `startNote()` викликається виключно на гілці

$$P \neq \perp$$

— security invariant, що прямо задокументований у `receiver.h`. Після успіху поле `note_index` у P додатково перевіряється на `noteIndex < NOTE_DICT_SIZE` перед індексацією `universal_notes[]`; валідний MAC не гарантує in-range note index, а out-of-bounds read у SRAM ATmega328P звернувся б до сусіднього protocol state з невизначеними наслідками.

```

1 // ReceiverNode/receiver.h
2 const uint8_t NOTE_DICT_SIZE = 21;
3
4 void authenticateAndPlay(const DataPacket* pkt);
```

Listing 2: Receiver-side dictionary bound and handler declaration

```

1  // ReceiverNode/ReceiverNode.ino (authenticateAndPlay)
2  uint8_t expectedMac[16];
3  s_cipher.computeTag(expectedMac, 16u);
4
5  if (memcmp(expectedMac, pkt->mac, TRUNCATED_MAC_SIZE) != 0) {
6      resetParser();
7      Serial.write(NACK_BYTE);
8      return;
9  }
10
11 Serial.write(ACK_BYTE);
12 const uint16_t noteIndex = plaintext[0];
13 const uint16_t durationMs = plaintext[1];
14
15 if (noteIndex == REST_INDEX) {
16     stopNote();
17 } else if (noteIndex < NOTE_DICT_SIZE) {
18     startNote(universal_notes[noteIndex], durationMs);
19 }

```

Listing 3: MAC verification gate and safe note playback path

Forgery probability for the truncated-tag scheme follows from the Poly1305 security proof. For a uniformly random one-time MAC key [41, 42]

$$r \in \{0, 1\}^{128}$$

derived from ChaCha20 block 0 under (K, IV_i) , the probability that a computationally unbounded adversary produces a forgery τ' satisfying:

$$\text{trunc}_8(\tau') = \text{trunc}_8(\tau_e)$$

without knowledge of K is at most 2^{-64} per frame, independently of the number of prior valid MAC-ciphertext observations. At 9600 bps the data channel supports at most:

$$\left\lfloor \frac{9600}{17 \times 8} \right\rfloor = 70$$

DataPacket frames per second. At this rate a brute-force forged-MAC campaign exhausts the 2^{64} tag space in approximately:

$$\frac{2^{64}}{70} \approx 2.6 \times 10^{17} \text{ s}$$

roughly 8 billion years at the maximum channel rate.

6.2 16-бітний номер послідовності та керування nonce

6.2.1 Небезпека повторного використання nonce

ChaCha20 — це stream cipher, безпека якого спирається на сувору унікальність пари (K, IV) у всіх шифруваннях, виконаних під ключем K . Якщо два різні plaintexts P_1 і P_2 шифруються під однаковою парою (K, IV) , XOR отриманих ciphertexts безпосередньо відкриває XOR plaintexts [43, 44, 45]:

$$C_1 \oplus C_2 = P_1 \oplus P_2$$

оскільки deterministic ChaCha20 keystream взаємно скорочується. У термінах C2P-ARQ чотирибайтовий payload несе `note_index` і `duration_ms`; якщо обидва значення відомі або здогадні для одного повідомлення — наприклад, якщо вже спостерігалася `rest note REST_INDEX = 255` із відомою тривалістю — другий plaintext негайно відновлюється. Більше того, IV collision розкриває Poly1305 one-time MAC key r , який виводиться як перші 32 bytes ChaCha20 block 0 під (K, IV) : знаючи r , супротивник може сфальсифікувати дійсний Poly1305 tag для будь-якого вибраного ciphertext і ін'єктувати його в session. Тож унікальність IV — це не preferred security property, а protocol correctness prerequisite.

6.2.2 Виведення per-packet nonce

C2P-ARQ гарантує унікальність IV у межах одного сеансу через explicit injective mapping від 16-bit sequence number i до 96-bit per-packet nonce IV_i . Виведення реалізується ідентично на TX і RX, працюючи з stack-local copy session nonce, щоб `s_sessionNonce` ніколи не змінювався:

```

1 // TransmitterNode/TransmitterNode.ino (sendPacket)
2 uint8_t packetNonce[HELLO_NONCE_SIZE];
3 memcpy(packetNonce, s_sessionNonce, HELLO_NONCE_SIZE);
4 packetNonce[10] ^= static_cast<uint8_t>((seqNumber >> 8u) & 0xFFu);
5 packetNonce[11] ^= static_cast<uint8_t>(seqNumber          & 0xFFu);

```

Listing 4: TX per-packet nonce derivation from 16-bit sequence

```

1 // ReceiverNode/ReceiverNode.ino (authenticateAndPlay)
2 uint8_t packetNonce[HELLO_NONCE_SIZE];
3 memcpy(packetNonce, s_sessionNonce, HELLO_NONCE_SIZE);
4 packetNonce[10] ^= static_cast<uint8_t>((seqNum >> 8u) & 0xFFu);
5 packetNonce[11] ^= static_cast<uint8_t>(seqNum          & 0xFFu);

```

Listing 5: RX mirrored per-packet nonce derivation

$$IV_i[k] = \begin{cases} S[k] & k \in \{0, 1, \dots, 9\} \\ S[10] \oplus \lfloor i/256 \rfloor & k = 10 \\ S[11] \oplus (i \bmod 256) & k = 11 \end{cases}$$

де

$$S \in \{0, 1\}^{96}$$

— це 12-byte session nonce, збережений у `s_sessionNonce` і встановлений під час SYN handshake; i — 16-bit packet sequence number (`uint16_t seqNum`) з:

$$i \in \{0, 1, \dots, 65535\}$$

$$\lfloor i/256 \rfloor$$

— high byte i , обчислений як `(seqNum >> 8u) & 0xFFu`; а

$$i \bmod 256$$

— low byte, обчислений як `seqNum & 0xFFu`. Bytes 0–9 nonce залишаються незмінними; XOR впливає лише на bytes 10 і 11. Непрухомість `s_sessionNonce` забезпечується коментарем у вихідних файлах: `// NEVER XOR directly into`

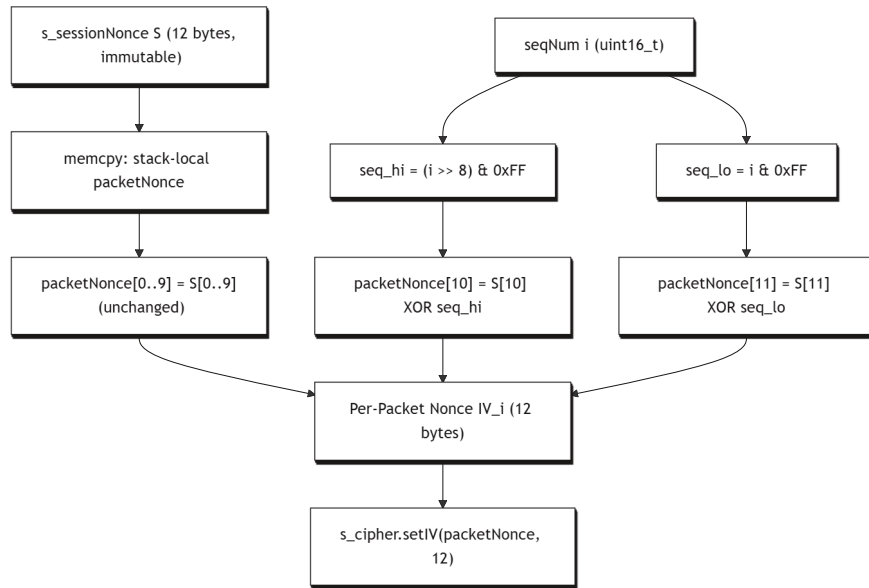


Figure 6.2: Похідне nonce для кожного пакета в `sendPacket()` та `authenticateAndPlay()`. Базовий session nonce копіюється у стековий локальний буфер, і лише байти 10 та 11 збурюються XOR-ом із старшим і молодшим байтами 16-бітного номера послідовності.

6.2.3 Ін'єктивність відображення

Відображення

$$f : \{0, \dots, 65535\} \rightarrow \{0, 1\}^{96}, i \mapsto IV_i$$

є ін'єктивним. Нехай

$$IV_i = IV_j \quad \text{for some } i \neq j$$

Оскільки bytes 0–9 обох nonces дорівнюють

$$S[0..9]$$

тотожно, equality of IV_i and IV_j зводиться до двох умов:

$$S[10] \oplus i_{hi} = S[10] \oplus j_{hi}$$

$$S[11] \oplus i_{lo} = S[11] \oplus j_{lo}$$

Застосовуючи XOR cancellation law до кожного рівняння, отримуємо:

$$i_{hi} = j_{hi}, \quad i_{lo} = j_{lo}$$

отже:

$$i = j$$

— суперечність. Тому всі 65,536 можливих sequence numbers породжують distinct 96-bit nonce, і в межах будь-якого одного сеансу IV repetition не може виникнути незалежно від змісту повідомлення чи тривалості сеансу. 16-bit range of i comfortably exceeds practical upper bound, imposed by ATmega328P Flash budget: Super Mario Bros. melody, stored in PROGMEM, comprises 80 notes, far below the 65,536-packet ceiling.

6.2.4 XOR проти модульного додавання

У цій конструкції для збурення байтів 10 і 11 використовується побітовий XOR, а не модульне додавання. Обидві операції є ін'єктивними на діапазоні 8-бітових беззнакових значень i , отже, могли б породжувати коректний per-packet IV. Вибір XOR зумовлено обмеженнями архітектури AVR: на ATmega328P інструкція EOR Rd, Rr є одноцикловою й не породжує carry, а її результат у байті 10 статистично незалежний від її результату в байті 11. Модульне додавання в C++ на 8-бітовому AVR потребує неявного піднесення до `int`, а отже, явного маскуванню назад до 8 бітів, щоб уникнути артефактів `unsigned-wraparound`, тоді як XOR за визначенням не переносить carry і не потребує маскуванню. Така побудова архітектурно узгоджується з rotate-XOR sponge, використаним у змішувачі ентропії, описаному в [Section 4.2](#), підтримуючи єдине використання тієї самої примітиви в усіх операціях бітових перетворень у стеку C2P-ARQ.

6.3 Стирання ключів і безпечне керування пам'яттю

6.3.1 Залишковість даних у SRAM ATmega328P

На відміну від Flash або EEPROM, SRAM є волатильним середовищем, яке зберігає свій вміст доти, доки підтримується напруга живлення, і втрачає його після знеструмлення. Однак ця залежність від живлення не усуває всіх шляхів фізичного вилучення. Зловмисник, який має фізичний доступ до живого вузла, може витягти вміст SRAM через інтерфейс debugWIRE/ISP, зупинивши CPU між двома інструкціями, або під'єднавши пробник до шини I2C чи UART і ініціювавши soft res-

et із подальшою командою читання пам'яті, або — у cold-boot варіанті — використавши залишкове збереження заряду, яке клітинки CMOS SRAM демонструють протягом десятків або сотень мілісекунд нижче мінімальної робочої напруги. Для session nonce [58]

$$S \in \{0, 1\}^{96}$$

який перебуває в SRAM, будь-який із цих шляхів дає повну здатність до виведення per-packet IV, описану в [Section 6.2](#): з S і публічно спостережуваних номерів послідовності i всі IV_i реконструюються, а всі session ciphertext стають придатними до розшифрування проти статичного MASTER_PSK, вбудованого у Flash.

6.3.2 Пряма секретність через детерміноване стирання

C2P-ARQ протидіє загрозі data remanence через обов'язкову політику zeroization: на кожній межі сеансу — чи то чистій, чи нечистій — 12-байтовий nonce сеансу негайно перезаписується нульовими байтами. Нехай t_{close} позначає момент, коли спрацьовує термінальний перехід FSM і виконується перший виклик `memset`. Операція має вигляд:

$$S \leftarrow \mathbf{0}^{96} \quad \text{at} \quad t = t_{\text{close}}$$

Усі n пакетів, передані протягом сеансу, були зашифровані під nonce

$$IV_0, IV_1, \dots, IV_{n-1}$$

похідними від S за відображенням із [Section 6.2](#). Встановлення

$$S = \mathbf{0}^{96}$$

робить усі IV_i недосяжними зі стану SRAM: зловмисник, який отримує дамп RAM у будь-який момент

$$t > t_{\text{close}}$$

спостерігає лише

$$S_{\text{dump}} = \mathbf{0}^{96}$$

Відновлення будь-якого IV_i вимагає перебору початкового S у повному 96-бітовому просторі nonce. Навіть у найгіршій моделі фізичного доступу, де MASTER_PSK уже

вилучено з Flash, перевага супротивника в декодуванні на рівні одного сеансу обмежується:

$$\Pr[\mathcal{A}(\mathbf{0}^{96}, \text{MASTER_PSK}) \rightarrow P_i] \leq 2^{-96}$$

де $\mathcal{A}$ — будь-який ймовірнісний поліноміальний супротивник, а P_i — будь-який plaintext, переданий під час закритого сеансу. Грубий пошук попсо зі швидкістю 10^9 спроб на секунду вичерпує повний простір попсо:

$$2^{96} \approx 7.9 \times 10^{28}$$

приблизно за:

$$\frac{2^{96}}{10^9} \approx 7.9 \times 10^{19} \text{ s}$$

— тобто більш ніж у 180 разів довше за вік Всесвіту, що спостерігається. Це встановлює forward secrecy на рівні сеансу: поточний стан SRAM не надає жодної здатності до розшифрування шифротекстів із будь-якого закритого сеансу. [46, 47, 48]

6.3.3 Точки спрацювання стирання

Повний життєвий цикл `s_sessionNonce` у SRAM — від початкового невизначеного значення через активне використання до детермінованого стирання — показано в Figure 6.3. Реалізація C2P-ARQ визначає чотири точки виклику стирання, що вичерпно покривають усі шляхи завершення сеансу.

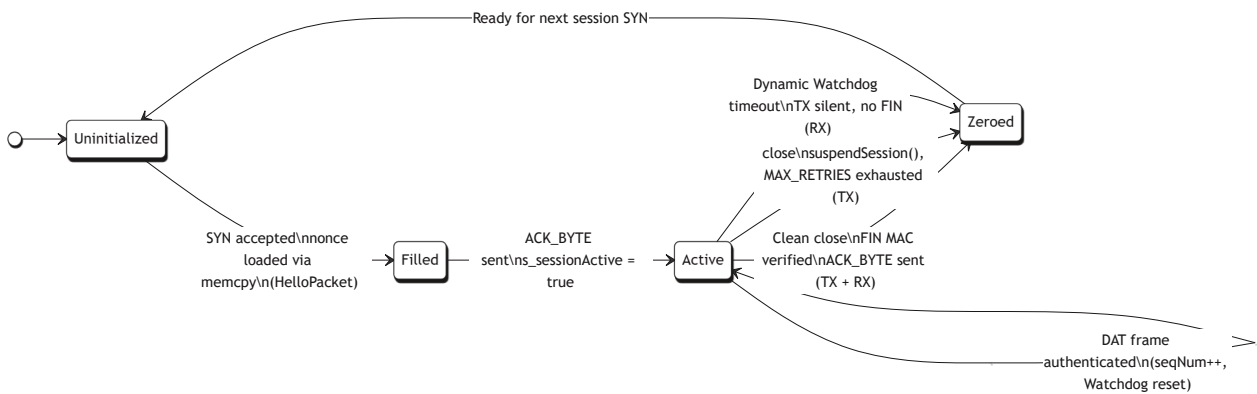


Figure 6.3: Життєвий цикл пам'яті `s_sessionNonce` у C2P-ARQ. Три незалежні тригери переводять її в стан Zeroed, забезпечуючи стирання ключового матеріалу як під час завершення протоколу, так і при односторонньому обриві лінії.

Вузол TX — коректне завершення

Успішна гілка `WAITING_FIN_ACK` (`TransmitterNode.ino`). Після отримання `ACK_BYTE`, що підтверджує `FLAG_FIN`-пакет, FSM виконує `memset(s_sessionNonce, 0, HELLO_NONCE_SIZE)` перед тим, як скинути `melodyIndex`, `seqNum` і `retryCount`, а також перейти до `TxState::IDLE`:

```
1 // TransmitterNode/TransmitterNode.ino (WAITING_FIN_ACK -> clean close)
2 if (response == ACK_BYTE) {
3     memset(s_sessionNonce, 0, HELLO_NONCE_SIZE);
4     melodyIndex = 0;
5     seqNum      = 0;
6     retryCount  = 0;
7     currentState = TxState::IDLE;
8     updateTxDisplay(currentState, seqNum, retryCount);
9 }
```

Listing 6: Clean session close on FIN acknowledgment (TX)

Вузол TX — некоректне завершення

`suspendSession()` (`TransmitterNode.ino`). Коли `retryCount` досягає `MAX_RETRIES` у будь-якому стані `WAITING_*` FSM, `suspendSession()` виконує `memset(s_sessionNonce, 0, HELLO_NONCE_SIZE)` як свою першу дію — до збереження `melodyIndex`, до скидання лічильників і до переходу в `TxState::RECONNECTING`:

```
1 // TransmitterNode/TransmitterNode.ino (suspendSession)
2 void suspendSession() {
3     memset(s_sessionNonce, 0, HELLO_NONCE_SIZE);
4     seqNum      = 0;
5     retryCount  = 0;
6     currentState = TxState::RECONNECTING;
7     lastReconnectAttempt = millis();
8 }
```

Listing 7: Unclean suspend path with reconnect transition (TX)

скидає `s_sessionActive` перед викликом `resetParser()`:

```
1 // ReceiverNode/ReceiverNode.ino (processFinPacket)
2 if (memcmp(expectedMac, pkt->mac, TRUNCATED_MAC_SIZE) != 0) {
3     Serial.write(NACK_BYTE);
4     resetParser();
5     return;
6 }
7
8 Serial.write(ACK_BYTE);
9 memset(s_sessionNonce, 0, HELLO_NONCE_SIZE);
10 s_sessionActive = false;
11 resetParser();
```

Listing 8: FIN verification and secure teardown on RX

Вузол RX — тайм-аут динамічного watchdog

`rx_loop()` (`ReceiverNode.ino`). Коли жоден authenticated `DataPacket` не надходить протягом `current_timeout_limit` мілісекунд після `last_valid_packet_time` — умова, що виявляє зникнення передавача без надсилання `FLAG_FIN`, — обробник `watchdog` виконує `memset(s_sessionNonce, 0, HELLO_NONCE_SIZE)`, скидає `s_sessionActive` і перезапускає парсер:

```
1 // ReceiverNode/ReceiverNode.ino (Dynamic Watchdog timeout)
2 if (s_sessionActive &&
3     (millis() - last_valid_packet_time) > current_timeout_limit) {
4     memset(s_sessionNonce, 0, HELLO_NONCE_SIZE);
5     s_sessionActive = false;
6     resetParser();
7 }
```

Listing 9: Dynamic watchdog-triggered session cleanup on RX

```
1  {  
2      uint8_t entropySeed[32];  
3      generateEntropyPool(entropySeed);  
4      initCSPRNG(entropySeed);  
5      memset(entropySeed, 0, sizeof(entropySeed));  
6  }
```

Listing 10: Entropy seed initialization and stack scrubbing

Без цього очищення raw TRNG-вихід — який до кондиціювання ChaCha20 може містити до:

$$H_{\text{raw}} \approx 24\text{--}32$$

бітів фізичної ентропії — залишався б доступним на фіксованій адресі стеку протягом усього часу виконання програми, надаючи супротивникові часткову опору, якщо він може спостерігати SRAM після ініціалізації.

6.3.5 Примітка щодо оптимізації компілятора

C Standard дозволяє компілятору усувати виклики `memset()`, що застосовуються до буферів, які більше не читаються, — це оптимізація `dead store elimination`. У `avr-gcc` на рівні `-Os` (типовий рівень оптимізації Arduino) таке усунення на практиці не відбувається для перевірених записів у SRAM; однак у `production`-розгортаннях, що потребують формальної гарантії на рівні машини, слід замінити прості `memset()` на `memset_s()` (C11 Annex K) або явне приведення до `volatile uint8_t *`, обидва з яких визначені так, щоб запобігати `dead-store elimination` на всіх сумісних рівнях оптимізації. [59]

7 Відмовостійкість і механізми самовідновлення

Криптографічні механізми, проаналізовані в [Section 6](#) — автентифікація AEAD, унікальність `per-packet nonce` і детерміноване стирання ключів — формують захист від супротивника, чия мета полягає у тому, щоб *маніпулювати* `payload`: підробляти ноти, повторно відтворювати старі кадри або реконструювати `plaintext` із захопленого шифротексту. Проти такого супротивника C2P-ARQ надає сильні гарантії безпеки, обмежені обчислювальною складністю компрометації ChaCha20-Poly1305.

Однак фізично активному супротивнику зовсім не обов’язково взаємодіяти з криптографічним шаром. Вводячи достатньо шуму, щоб пошкодити кожен переданий біт, нападник перетворює канал на такий, у якому жоден валідний кадр не може досягти приймача. Poly1305 MAC правильно відкидає кожен пошкоджений кадр; однак така коректна поведінка не просуває мелодію вперед. Кожен відхилений кадр споживає одну одиницю `retry budget`. Коли вичерпується `MAX_RETRIES = 50` повторів із `ACK_TIMEOUT_MS = 50 ms` кожен — тобто під час тривалого інтервалу глушіння:

$$50 \times 50 = 2,500 \text{ ms}$$

— сеанс завершується. Безпека збережена повністю; доступність знищена.

Ця дихотомія мотивує механізми відмовостійкості, описані в цьому розділі. Замість того щоб трактувати переривання каналу як фатальну помилку, яка потребує втручання оператора, C2P-ARQ визначає автономний автомат відновлення, що повторно встановлює сеанс і відновлює передавання з тієї самої ноти, на якій воно було порушене. Цю властивість — *termed Self-Healing* — є визначальною відмінністю SH2SC-EDT від звичайних вбудованих комунікаційних реалізацій.

7.1 Стан RECONNECTING і збереження стану корисного навантаження

7.1.1 Наївна IoT-архітектура

Переважаючою реакцією на збій зв'язку в ресурсно обмежених вбудованих системах є скидання через hardware watchdog: якщо застосунок не обслуговує watchdog у межах заданого інтервалу, MCU перезавантажується. Хоча перезавантаження гарантує, що прошивка повертається до відомого початкового стану, воно одночасно знищує весь прогрес прикладного рівня. У контексті SH2SC-EDT це означало б скидання `melodyIndex` до нуля і перезапуск мелодії з P_0 на кожному порушенні каналу, незалежно від того, скільки нот уже було успішно доставлено.

7.1.2 Поділ стану в C2P-ARQ

SH2SC-EDT уникає цього регресу шляхом поділу runtime-стану передавача на дві концептуально різні категорії:

- Криптографічний стан — стан, який має бути скинутий після кожного сеансу для збереження безпеки: `s_sessionNonce` (12 байтів), `seqNum` і `retryCount`. Збереження застарілого nonce через межу сеансу дозволило б повторно використати той самий IV для нового plaintext, порушуючи унікальність nonce і дозволяючи атаки відрізнення шифротекстів.
- Стан payload — стан, який кодує прогрес прикладного рівня і не повинен скидатися під час порушення сеансу: `melodyIndex` (2 байти). Скидання цього поля не додає жодної користі безпеці та руйнує властивість liveness, що мотивує дизайн Self-Healing.

`suspendSession()` реалізує цей поділ точно. Функція зануляє `s_sessionNonce` через `memset()`, скидає `seqNum = 0` і `retryCount = 0`, а також переводить FSM у `TxState::RECONNECTING` — залишаючи `melodyIndex` незмінним. Розміщений поруч коментар у вихідному коді читається: *“melodyIndex intentionally NOT reset — resume from point of failure.”* [60]

7.1.3 Інваріант самовідновлення

Нехай

$$\mathcal{M} = \langle P_0, P_1, \dots, P_{M-1} \rangle$$

є мелодією з M нот. Подією порушення на індексі k вважається виконання `suspendSession()`, коли `melodyIndex` дорівнює k . Позначимо k_j як значення `melodyIndex` у момент, коли відбувається j -те порушення. Інваріант Self-Healing C2P-ARQ стверджує, що наступний сеанс $\mathcal{S}_{j+1}$ починає передавання з P_{k_j} :

$$\text{melodyIndex}[\mathcal{S}_{j+1}] = k_j \quad \forall j \in \{1, \dots, n\}$$

Для будь-якої послідовності з $n \geq 0$ подій порушення нота, доставлена в першому `DataPacket` $\mathcal{S}_{j+1}$, є саме P_{k_j} , а не P_0 . Повна мелодія $\mathcal{M}$ зрештою доставляється без розривів за умови, що принаймні один сеанс для кожного решти суфікса

$$\langle P_{k_j}, \dots, P_{M-1} \rangle$$

завершується без порушення.

Автомат Self-Healing з точки зору TX показано в Figure 7.1.

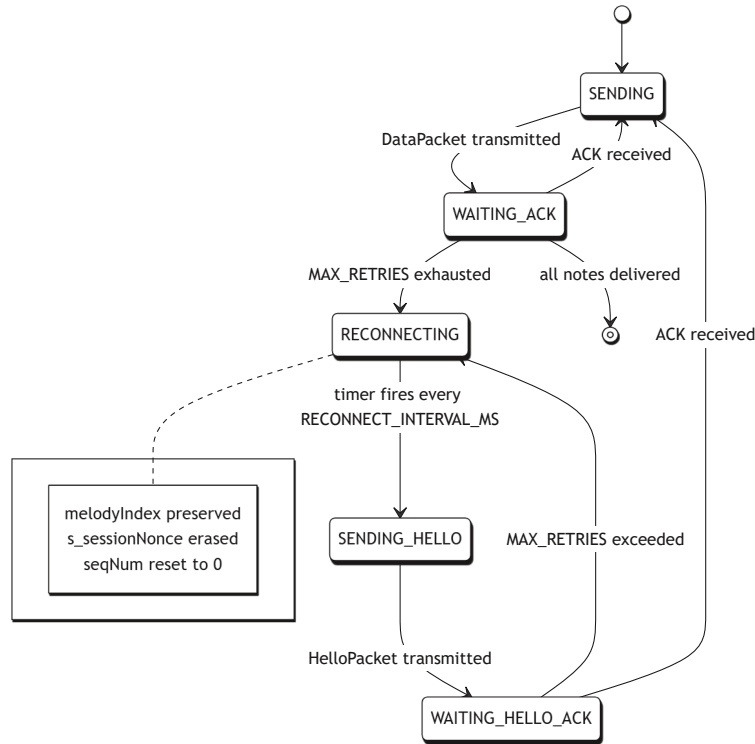


Figure 7.1: Автомат Self-Healing на TX-вузлі. Після вичерпання `MAX_RETRIES` `suspendSession()` стирає криптографічний стан і зберігає `melodyIndex`; неблокувальний таймер повторює спробу обміну кожні `RECONNECT_INTERVAL_MS`.

7.1.4 Неблокувальне опитування RECONNECTING

Після входу в `TxState::RECONNECTING` передавач не виконує busy-wait і не блокує виконання. Кожен виклик `tx_loop()` оцінює єдину умову на базі `millis()`:

```
1  if ((millis() - lastReconnectAttempt) >= RECONNECT_INTERVAL_MS) {
2      lastReconnectAttempt = millis();
3      currentState = TxState::SENDING_HELLO;
4      updateTxDisplay(currentState, seqNum, retryCount);
5  }
```

Listing 11: Non-blocking reconnect timer check in TX loop

Коли умова виконується, FSM переходить до `TxState::SENDING_HELLO`, який у наступному виклику `tx_loop()` надсилає свіжий `HelloPacket` із новим попсе, згенерованим CSPRNG. Якщо приймач залишається недосяжним, `WAITING_HELLO_ACK` вичерпує власний бюджет `MAX_RETRIES` і знову викликає `suspendSession()`, яка повторно входить у `RECONNECTING` і перезапускає 2000 ms timer. Це створює стійкий цикл auto-reconnect, який завершується лише тоді, коли приймач відповідає `ACK_BYTE`. У цей момент перехід `WAITING_HELLO_ACK` $\rightarrow$ `SENDING` переносить збережений `melodyIndex = kj` у новий сеанс — без втручання оператора.

Оператор може скасувати Self-Healing resumption, натиснувши кнопку TX під час `RECONNECTING`: це явно скидає `melodyIndex = 0` і примушує перезапуск мелодії з P_0 .

7.2 Розумний динамічний watchdog: адаптація тайм-ауту з урахуванням корисного навантаження

7.2.1 Проблема тупика на приймачі

Шлях завершення `FLAG_FIN`, описаний у [Section 5](#), забезпечує чисте закриття сеансу лише за припущення, що передавач і далі здатний надіслати фінальний автентифікований пакет. На практиці це припущення порушується двома режимами відмови: жорстким відключенням живлення на передавачі та сценарієм, проаналізованим у [Section 7.1](#), де передавач вичерпує `MAX_RETRIES` і входить у

12 байтів SRAM, а парсер прийматиме новий SYN лише після явного звільнення сеансу. Без активного механізму звільнення наступний сеанс ніколи не може бути встановлений — система входить у deadlock.

Статичний тайм-аут завершення сеансу не розв’язує цей deadlock коректно. Коротке статичне значення — менше за найдовшу ноту в мелодії — спрацьовує раніше, ніж передавач встигне надіслати наступний пакет, спричиняючи хибне скидання сеансу під час звичайного відтворення ноти. Довге статичне значення — обране консервативно, щоб перевищувати найдовшу можливу ноту — змушує приймач тримати session RAM повний статичний інтервал після зникнення передавача, непотрібно затримуючи відновлення зв’язку на весь період watchdog. Жоден із цих варіантів не може одночасно уникати false positive на довгих нотах і мінімізувати затримку повторного підключення після реальної втрати каналу.

7.2.2 Адаптація з урахуванням корисного навантаження

C2P-ARQ розв’язує цю дихотомію, використовуючи властивість payload, уже доступну на приймачі без додаткової вартості протоколу: поле тривалості `duration_ms`, що переноситься в кожному автентифікованому `DataPacket`. Оскільки це поле витягується з plaintext, перевіреного Poly1305, — а отже, гарантовано є автентичним, — приймач може обчислити точну нижню межу інтервалу мовчання, яка має минути до очікуваного наступного пакета. Тайм-аут watchdog для пакета k не є константою часу компіляції, а обчислюється динамічно всередині `authenticateAndPlay()` одразу після запуску ноти:

```
1  current_timeout_limit = static_cast<uint32_t>(durationMs)
2                          + NETWORK_GRACE_PERIOD_MS;
3  last_valid_packet_time = millis();
```

Listing 12: Adaptive watchdog timeout update after valid packet

До першого `DataPacket` сеансу — у проміжку після завершення `processHelloBody()` і до першого виклику `authenticateAndPlay()` — watchdog озброюється консервативним значенням за замовчуванням `current_timeout_limit = 5000 ms`. Це значення покриває випадок, коли канал глушиться одразу після SYN-handshake, ще до того, як якась тривалість ноти стала відомою.

7.2.3 Формальна умова тайм-ауту

Нехай $t_{\text{last}}^{(k)}$ позначає `timestamp millis()`, записаний приймачем одразу після автентифікації пакета k , а d_k — тривалість ноти, витягнуту з plaintext цього пакета. Dynamic Smart Watchdog спрацьовує — зануляючи `s_sessionNonce`, очищаючи `s_sessionActive` і перезапускаючи байтовий парсер — тоді і лише тоді, коли:

$$t_{\text{extnow}} - t_{\text{extlast}}^{(k)} > d_k + T_{\text{grace}}$$

де t_{now} — значення `millis()` у поточному виклику `rx_loop()`,

$$d_k \in \mathbb{N}_{>0}$$

є тривалістю в мілісекундах k -ї автентифікованої ноти, а $T_{\text{grace}} = 3000 \text{ ms}$ — фіксована константа `NETWORK_GRACE_PERIOD_MS`, вбудована в обидва образи прошивки. Права частина є мінімальним часом, який має минути між останнім автентифікованим пакетом і спрацюванням watchdog: d_k мілісекунд для самої ноти та T_{grace} мілісекунд додаткового запасу, що поглинає worst-case retry storm передавача і round-trip кадру.

Оцінка виконується в `rx_loop()` на кожній ітерації циклу, під захистом предиката `s_sessionActive`, щоб уникати зайвих обчислень, коли жоден сеанс не відкрито:

```

1  if (s_sessionActive &&
2      (millis() - last_valid_packet_time) > current_timeout_limit) {
3      memset(s_sessionNonce, 0, HELLO_NONCE_SIZE);
4      s_sessionActive = false;
5      resetParser();
6  }
```

Listing 13: RX watchdog expiration condition and parser reset

Повну логіку watchdog — від arming сеансу до умовного стирання — показано в Figure 7.2.

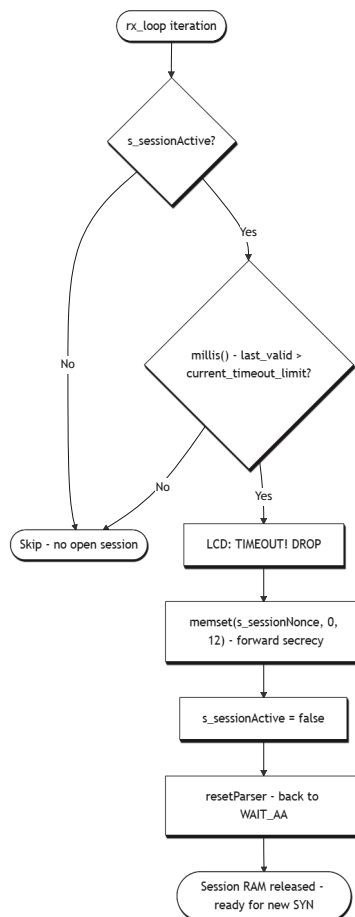


Figure 7.2: Шлях оцінювання Smart Dynamic Watchdog у `rx_loop()`. Умова `millis() - last_valid_packet_time > current_timeout_limit` перевіряється на кожній ітерації loop; гілка стирання виконується не більше одного разу за сеанс.

7.2.4 Відсутність гонок між TX і RX

Критичний інваріант має виконуватися: watchdog приймача ніколи не повинен спрацьовувати тоді, коли передавач ще перебуває в середині легітимного retry storm — тобто приймач не має занулювати `s_sessionNonce` до того, як передавач завершить свій цикл повторів `MAX_RETRIES × ACK_TIMEOUT_MS` і увійде в `TxState::RECONNECTING`. Якщо б ця race condition могла виникнути, перший `HelloPacket` reconnect-handshake був би оброблений приймачем, який уже звільнив свій session nonce, що є правильним результатом. Однак повторний `DataPacket`, який надійде після занулення `s_sessionNonce`, провалить MAC-перевірку та породить NACK — ескалюючи вже деградований канал до невідновлюваної десинхронізації до завершення наступного SYN-handshake.

Протокол усуває цю гонку за конструкцією. Worst-case TX retry storm обмежується 2500 ms, як уже виведено у [Section 5.3.3](#).

Грейс-період задовольняє:

$$T_{\text{grace}} = 3000 \text{ ms} > T_{\text{storm}} = 2500 \text{ ms}$$

запасом у 500 ms. Оскільки watchdog timeout дорівнює

$$d_k + T_{\text{grace}}$$

і

$$d_k \geq 0$$

приймач гарантовано утримуватиме свій session поnce принаймні на $T_{\text{grace}} = 3000 \text{ ms}$ довше за останній автентифікований пакет — довше, ніж передавач може перебувати в будь-якому стані `WAITING_*` перед переходом у `RECONNECTING`. Коментар у вихідному коді для `NETWORK_GRACE_PERIOD_MS` відображає це точно: *“Absorbs the worst-case TX retry storm: $\text{MAX_RETRIES} \times \text{ACK_TIMEOUT_MS} = 50 \times 50 \text{ ms} = 2500 \text{ ms}$. Value 3000 ms provides 500 ms margin above that maximum.”* Два вузли, таким чином, сходяться до своїх відповідних `idle`-станів — `TxState::RECONNECTING` і перезапуск байтового парсера до `WAIT_AA` — у правильному часовому порядку, що робить подальший SYN-handshake безумовно успішним.

7.3 Очищення застарілого буфера і відновлення після десинхронізації

7.3.1 Проблема отруєння застарілими байтами

UART-периферія ATmega328P має 64-байтовий апаратний ring buffer, яким керує драйвер `Arduino HardwareSerial`. Під час шумового сплеску — або в будь-який інтервал, коли процесор зайнятий криптографічними обчисленнями і не може обслуговувати буфер прийому — вхідні байти накопичуються в цьому FIFO без споживання. Коли сплеск закінчується, ці байти залишаються в буфері і будуть віддані наступному виклику `Serial.read()`, ніби вони щойно надійшли. Саме це створює тонкий режим відмови: якщо storm повторних передач генерує послідовність NACK-байтів, які все ще присутні в апаратному FIFO, коли надсилається наступний пакет, передавач прочитає їх як відповідь на новий пакет, а не на попередній — що спричинить хибні повтори або, у найгіршому випадку,

просування FSM на NACK, який ніколи не був призначений відповіддю на поточний номер послідовності.

Проблема посилюється асиметрією між TX і RX UART-буферами. Виклик `Arduino Serial.flush()` очищає лише *transmit* buffer — він блокує до повного виходу всіх накопичених outgoing bytes із TX shift register. На *receive* buffer він не впливає. У Arduino немає API, що атомарно дренує вхідні байти з апаратного FIFO; єдиний правильний механізм — ручний drain-loop, який читає та відкидає всі байти, що очікують, перед передаванням нового пакета. [53, 54, 55]

7.3.2 Активне очищення TX — `drainRxFifo()`

C2P-ARQ вирішує це на боці передавача через спеціальну inline-функцію, яку викликають перед кожною передачею пакета:

```
1  static inline void drainRxFifo() {
2      while (Serial.available() > 0) {
3          Serial.read();
4      }
5  }
```

Listing 14: TX helper for draining stale UART RX bytes

`drainRxFifo()` читає й відкидає всі байти, що наразі доступні в буфері прийому, у щільному циклі, споживаючи максимум 64 ітерації (глибина апаратного FIFO). Її викликають першою операцією в кожній із трьох функцій передавання — `sendHelloPacket()`, `sendPacket()` і `sendFinPacket()` — до виклику `Serial.write()`. Коментар у вихідному коді документує це точно: *“Prevents stale NACK bytes — accumulated from noise bursts before the session started — from being misread as responses to the packet that is about to be transmitted. Called at the start of every send function to guarantee that the first byte read after a send is always a fresh response.”*

Наслідком є сильний інваріант: перший байт, який передавач читає з каналу зворотного зв’язку після надсилання будь-якого пакета, гарантовано є відповіддю саме на цей пакет, а не залишком будь-якого попереднього шумового сплеску чи циклу повторів.

7.3.3 Пасивне очищення RX — `resetParser()`

На боці приймача отруєння FIFO виникає у зворотному напрямку: після MAC-збою або timeout парсера в апаратному FIFO можуть залишатися хвостові байти пошкодженого кадру або наступних шумових імпульсів, що надійшли, поки CPU виконував pipeline перевірки. Якщо ці байти не відкинути, вони будуть подані в `processReceivedByte()` на наступній ітерації `loop()` і можуть породити хибне співпадіння preamble або спотворити накопичення наступного легітимного кадру. `resetParser()` усуває цей шлях тим самим циклом дренажу, після якого виконується жорсткий reset FSM:

```
1  static void resetParser() {
2      while (Serial.available() > 0) {
3          Serial.read();
4      }
5      rx_index      = 0;
6      parseState    = WAIT_AA;
7      currentState = RxState::WAITING_SYNC_1;
8  }
```

Listing 15: RX parser reset with FIFO drain and state re-arm

Docstring прямо формулює намір дизайну: “*Called on MAC failure or parser timeout to prevent a noise burst that corrupted one frame from also corrupting the preamble detection of the next frame. Reads and discards all bytes in the 64-byte hardware FIFO (Serial.flush() only flushes TX; manual drain is required for RX).*” Після повернення `resetParser()` байтовий FSM перебуває у чистому, повністю визначеному початковому стані: `rx_index = 0`, `parseState = WAIT_AA`, `currentState = WAITING_SYNC_1`.

7.3.4 Пасивна синхронізація RX — пошук преамбули

Після того як FIFO дренажено, а парсер перебуває у `WAIT_AA`, приймач починає пасивно сканувати канал байт за байтом, відкидаючи все, доки не буде ідентифіковано коректний preamble `0xAA 0x55`. `processReceivedByte()` реалізує це через дві послідовні FSM-стадії:

- `WAIT_AA`: кожен вхідний байт перевіряється. Лише `SYNC_BYTE_1 = 0xAA` переводить парсер у `WAIT_55`; будь-який інший байт мовчки відкидається, а парсер лишається

- **WAIT_55**: перевіряється другий байт. **SYNC_BYTE_2 = 0x55** переводить у **READ_TYPE** (preamble підтверджено). Другий **0xAA** залишає парсер у **WAIT_55** — це коректно обробляє перекривний випадок **0xAA 0xAA 0x55**. Будь-який інший байт повертає до **WAIT_AA** — частковий preamble **0xAA X** відкидається, і сканування відновлюється.

Жодне тіло кадру не накопичується, доки обидва байти preamble не будуть послідовно зіставлені. Шлюз є безумовним: **FLAG_SYN**, **FLAG_DAT** і **FLAG_FIN** однаково підлягають вимозі **0xAA 0x55**. Коментар у вихідному коді для **WAIT_AA** читається: *“Any other byte (including FLAG_SYN=0x01 without preamble) — channel noise, stay.”*

Поєднаний TX-drain і RX-preamble-hunt процес відновлення після десинхронізації показано в Figure 7.3.

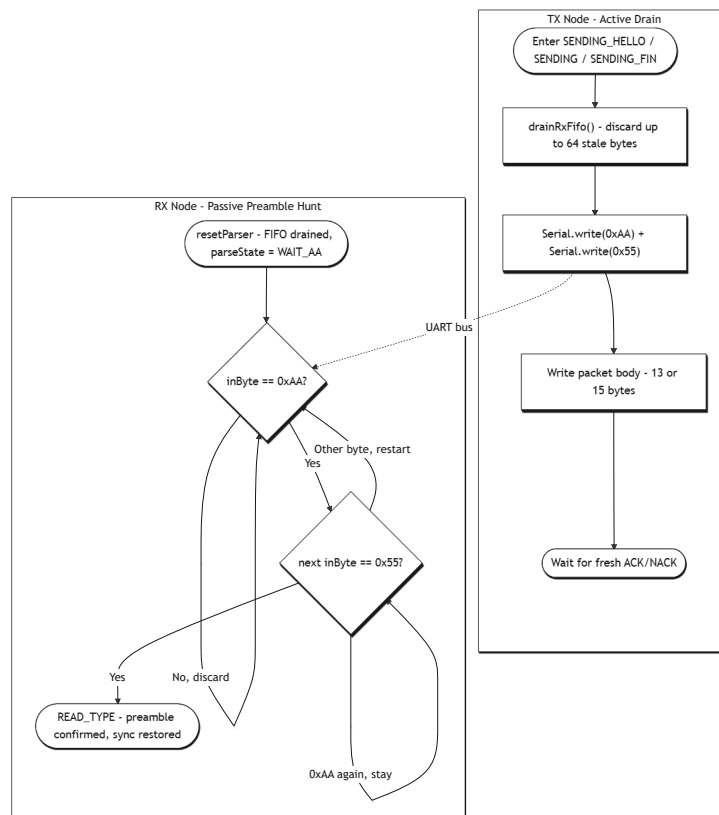


Figure 7.3: Двостороннє відновлення після десинхронізації, спричиненої сплеском шуму. TX-вузол очищує свій FIFO прийому перед передаванням; RX-вузол відкидає довільні сміттеві байти, доки не буде знайдено преамбулу **0xAA 0x55**.

7.3.5 Ймовірність хибної синхронізації

Двобайтовий preamble **0xAA 0x55** обрано так, щоб мінімізувати ймовірність того, що випадковий шум каналу утворить хибне співпадіння preamble. Нехай b_i позначає значення довільного байта, рівномірно вибраного з

$$\{0, \dots, 255\}$$

Ймовірність того, що два послідовні шумові байти становитимуть коректний preamble, дорівнює [56, 57]:

$$P(b_i = \mathbf{0xAA} \wedge b_{i+1} = \mathbf{0x55}) = \frac{1}{256} \times \frac{1}{256} = 2^{-16} \approx 1.52 \times 10^{-5}$$

де b_i і b_{i+1} — два послідовні незалежно рівномірно розподілені випадкові байти, $\mathbf{0xAA} = 170_{10}$ — SYNC_BYTE_1, а $\mathbf{0x55} = 85_{10}$ — SYNC_BYTE_2. Ймовірність $2^{-16} \approx 1.52 \times 10^{-5}$ відповідає очікуваній частоті хибних співпадінь preamble на двобайтове вікно за найгіршого припущення, що джерело шуму є рівномірним генератором байтів — максимальним за ентропією супротивником для статистики значень байтів. На практиці шумовий джерело на базі XOR pulse-generator 1 kHz породжує бітові інверсії, а не рівномірно випадкові байтові значення, що призводить до корельованих спотворень, які групуються навколо околу початкового значення даних. Отже, хибна частота preamble за такої фізично реалістичної моделі шуму є строго нижчою за теоретичну межу 2^{-16} .

Після хибного співпадіння preamble парсер переходить до READ_TYPE. Хибний байт READ_TYPE, який не збігається з жодним розпізнаним flag bit (FLAG_SYN = 0x01, FLAG_DAT = 0x02, FLAG_FIN = 0x04), негайно скидає парсер до WAIT_AA: вихідний код у гілці нерозпізаного прапора читається *“No recognised flag bit — channel noise, restart preamble scan.”* Щоб хибне тіло пройшло до стадії dispatch, хибний байт READ_TYPE має містити принаймні один із бітів 0, 1 або 2 — імовірність $7/256$ — а накопичені байти тіла мають породити Poly1305 MAC, який проходить memcmp проти укороченого 8-байтового тегу, імовірність чого дорівнює 2^{-64} для будь-якого обчислювально обмеженого супротивника. Отже, сумарна ймовірність хибнопозитивного спрацювання є незначною за всіх реалістичних умов експлуатації.

8 Модель супротивника та симуляція глушіння

[Section 6](#) і [Section 7](#) встановлюють два необхідні стовпи коректності: криптографічну безпеку під AEAD-конструкцією та детерміновану логіку відновлення за обмеженої кількості повторів. Ці результати гарантують конфіденційність, цілісність і властивості відновлення стану на рівні протоколу за припущень дизайну C2P-ARQ. Однак самі по собі вони недостатні для доведення операційної стійкості в присутності тривалого фізичного порушення каналу.

Liveness і відмовостійкість мають бути продемонстровані емпірично за умов ворожого каналу, де доставлення кадрів неодноразово перешкоджається активною інтерференцією, а не доброзичливим випадковим шумом. Мета цього розділу — піддати C2P-ARQ найсильнішому фізичному стрес-тесту, доступному в середовищі SimulIDE, та оцінити, чи зберігається end-to-end прогрес сеансу без втручання оператора.

8.1 Можливості вузла атакувальника

У тестовому стенді SH2SC-EDT Attacker не є програмованим мікроконтролером і не виконує логіку прошивки. Він реалізований як виділена апаратна підсхема, що вбудована в UART-шлях даних і діє як фізичний Man-in-the-Middle між передавачем і приймачем. Відповідно, він не має доступу до протоколу: не розбирає структури пакетів, не обчислює і не перевіряє теги автентифікації та не генерує кадри, сумісні з C2P-ARQ.

Його пасивні можливості надаються інструментацією SimulIDE. За допомогою осцилографа та logic analyzer супротивник може спостерігати бітову форму сигналу, вимірювати часові характеристики передавання та інферувати морфологію трафіку (структуру сплесків, idle-gap і cadence повторних передач). Оскільки кадри з payload захищені ChaCha20-Poly1305, таке спостереження розкриває лише часові й канално-осцурапу метадані, а не автентифіковану семантику plaintext.

Його активна можливість — це фізична інтерференція каналу: супротивник може захопити лінію передавання й ін'єктувати руйнівну сигналізацію, яка пошкоджує символи в польоті та робить прийняті кадри недійсними. Ця модель відображає pressure на UART-канал рівня hardware jamming і створює adversarial bas-

is для оцінювання liveness і fault tolerance протоколу під час тривалої деградації. Внутрішню конструкцію інструменту глушіння навмисно відкладено до [Section 8.2](#).

Підсхему Attacker, використану в тестовому стенді SH2SC-EDT, показано в Figure 8.1.

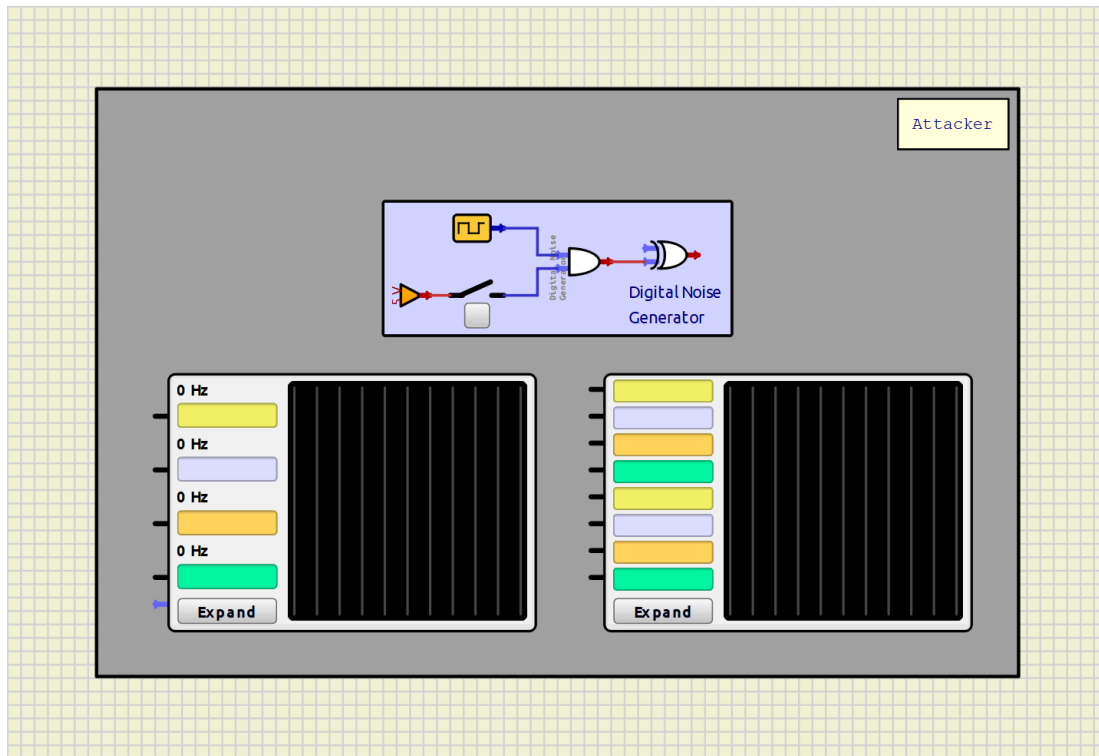


Figure 8.1: Огляд підсхеми Атакувальника в SimulIDE, оснащеної осцилографом і логічним аналізатором для пасивного моніторингу та генератором цифрового шуму для активного порушення каналу.

8.2 Архітектура цифрового генератора шуму та XOR-ін'єкція

Digital Noise Generator у SimulIDE реалізовано як компактний ланцюг логічних елементів, єдина мета якого — накладати керовану бінарну інтерференцію на forward UART-канал. Його топологія складається з ручного шляху увімкнення 5 V, джерела імпульсів 2000 Hz, AND-елемента, який або пропускає, або пригнічує потік завади, та XOR-елемента, який ін'єктує отриману хвилю глушіння в чистий послідовний вихід Node A перед тим, як сигнал досягне Node B.

Логічну архітектуру цифрового генератора шуму показано в Figure 8.2.

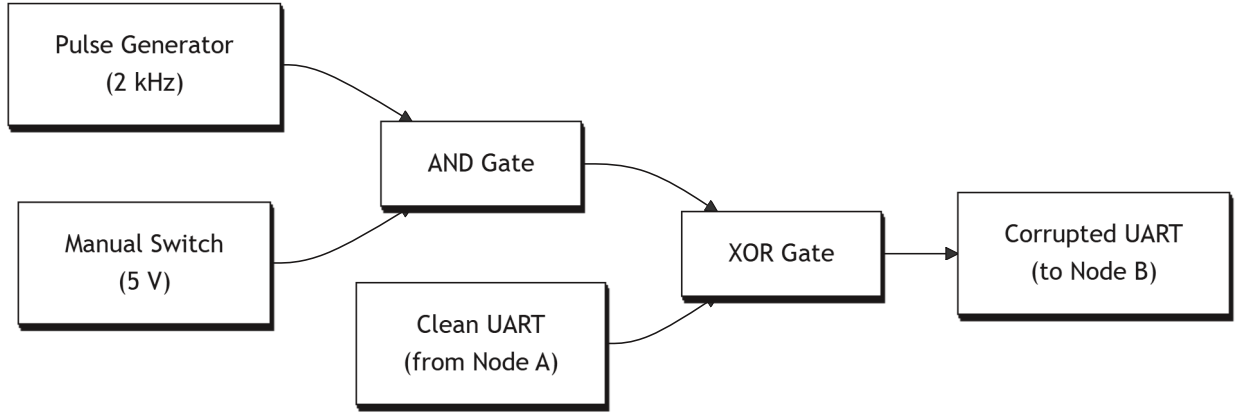


Figure 8.2: Логічний потік ін'єкції цифрового шуму у випробувальному стенді SH2SC-EDT SimulIDE. Джерело імпульсів 2000 Гц умовно вмикається ручним перемикачем 5 В через AND-елемент і далі ін'єктується в чистий UART-потік через XOR-каскад, утворюючи спотворений серіальний сигнал на вході приймача.

На логічному рівні AND-стадія працює як gate увімкнення. Нехай

$$P(t) \in \{0, 1\}$$

позначає імпульсний сигнал 2000 Hz, а нехай

$$S \in \{0, 1\}$$

позначає стан ручного перемикача, де $S = 0$ відповідає відкритому стану, а $S = 1$ — замкненому. Тоді сигнал завади, поданий на XOR-вхід, є

$$N(t) = P(t) \wedge S$$

так що, коли перемикач відкритий,

$$N(t) = P(t) \wedge 0 = 0$$

для всіх t , і шлях шуму електрично заблокований. У цій спокійній конфігурації XOR-стадія отримує нульовий disturbance input і, отже, поводить себе прозоро щодо серійної лінії.

Механізм спотворення керується булевою алгеброю XOR. Якщо $V_{\text{uart}}(t)$ позначає миттєвий логічний рівень на чистій UART-лінії, то вихід, що подається до Node B, є

$$V_{\text{out}}(t) = V_{\text{uart}}(t) \oplus N(t)$$

що дає дві робочі ідентичності

$$V_{\text{uart}} \oplus 0 = V_{\text{uart}}, \quad V_{\text{uart}} \oplus 1 = \neg V_{\text{uart}}$$

Ці співвідношення формалізують фізичну поведінку ін'єктора. Коли перемикач відкритий, вихід AND примусово низький і UART-сигнал проходить через XOR-стадію без змін. Коли перемикач замкнений, вихід AND відтворює імпульсний сигнал 2000 Hz, і кожен інтервал, коли цей сигнал логічно високий, змушує XOR-стадію інвертувати миттєвий рівень, який наразі несе UART-провідник. Отже, канал не замінюється незалежною хвилею; він умовно інвертується по полярності в реальному часі.

Деструктивний ефект впливає з часових співвідношень між джерелом завади та асинхронною serial-лінією. За 9600 bps один UART-біт займає

$$T_{\text{bit}} = \frac{1}{9600} \approx 104 \mu s$$

тоді як джерело завади 2000 Hz має період

$$T_{\text{jam}} = \frac{1}{2000} = 500 \mu s$$

що істотно коротше за тривалість повного 10-бітового UART-символу (приблизно **1.04 ms** разом зі start і stop-бітами). Отже, після увімкнення ін'єктора інверсії полярності відбуваються на підкадровому часовому масштабі й перетинають кілька символів інтервалів під час передавання пакета. Будь-яка така інверсія, що вражає автентифікований заголовок, шифротекст або укорочений Poly1305 tag, змінює отриману бітову структуру й примушує перевірку цілісності C2P-ARQ детерміновано провалитися. Digital Noise Generator, таким чином, є фізично обґрунтованим інструментом бітових інверсій: він не моделює втрату пакетів на програмному рівні, а спотворює електричну репрезентацію кадру в польоті.

Внутрішню gate-level архітектуру цифрового генератора шуму показано в Figure 8.3.

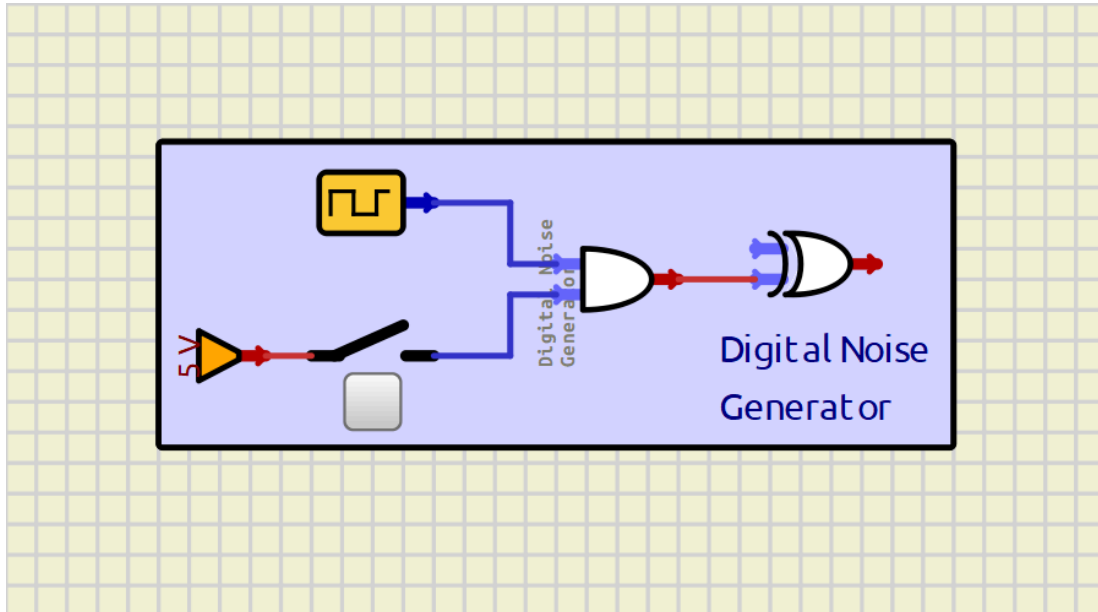


Figure 8.3: Внутрішня архітектура генератора цифрового шуму в SimulIDE. Тактовий сигнал 2000 Гц проходить через AND-елемент за допомогою ручного перемикача, а потім ін'єктується в UART-лінію даних через XOR-елемент для спричинення детермінованих бітових інверсій.

8.3 Результати симуляції: стійкість під час тривалого глушіння

У цьому підрозділі наведено емпіричну поведінку SH2SC-EDT під час тривалої фізичної інтерференції, використовуючи хронологічний сценарний метод, а не ізольовані знімки. Хронологія простежується від холодного старту, через спроби handshake та десинхронізацію протоколу під час активного глушіння, до остаточного відновлення через шлях self-healing C2P-ARQ. Мета — перевірити властивості liveness і відмовостійкості у ворожому середовищі, де однієї лише криптографічної коректності недостатньо як доказу операційної безперервності.

У всіх сценаріях використовуються два синхронізовані інструменти. Осцилограф надає фізичний погляд, розкриваючи інтерференцію на рівні напруги та інверсії лінії в реальному часі. Logic analyzer надає погляд на data-link/protocol, показуючи, чи можуть спостережувані патерни фронтів бути інтерпретовані як валідний framed traffic приймачем-парсером. Поеднана методика встановлює прямий причинний ланцюг між електричним спотворенням і протокольною відмовою або відновленням.

8.3.1 Сценарій 1: ініціалізація системи під активним глушінням

У момент $t = 0$ обидва вузли вмикаються, коли канал атакувальника вже активний. Отже, перша фаза відкриття протоколу намагається виконатися під безпосереднім впливом завади, а не на чистому baseline. На боці передавача (Node A) шлях відкриття C2P-ARQ залишається обмеженим циклом handshake (SENDING_HELLO/WAITING_HELLO_ACK) і, після повторного ACK timeout або відхилення, повторно входить у RECONNECTING відповідно до бюджету повторів (MAX_RETRIES = 50) та reconnect timer (RECONNECT_INTERVAL_MS = 2000\, \text{ms}). Жоден authenticated session не встановлюється, і до доставлення даних прогрес не переходить.

На боці приймача (Node B) жоден коректний preamble не накопичується на рівні парсера (WAIT_AA/WAIT_55), тоді як відображуваний RX FSM залишається у станах очікування синхронізації (WAITING_SYNC_1). Іншими словами, канал переносить переходи, але не межі кадрів, прийнятні для шлюзу SYNC_BYTE_1/SYNC_BYTE_2. Це очікуваний режим відмови під час безперервного глушіння на старті: активність парсера без admission до протоколу.

Фізичний механізм описується моделлю XOR injection

$$S_{rx}(t) = S_{tx}(t) \oplus N(t)$$

де $S_{tx}(t)$ — чиста UART-хвиля від Node A, $N(t)$ — процес завади атакувальника, а $S_{rx}(t)$ — хвиля, що спостерігається на вході Node B. Щоразу, коли

$$N(t) = 1$$

миттєвий рівень UART інвертується; отже, навіть короткі високі інтервали в $N(t)$ можуть зробити недійсними структуру start/data/stop в асинхронних серійних символах ще до будь-якої перевірки на рівні протоколу.

Figure 8.4 показує повний стенд під час фази запуску під атакою. Передавач не отримав валідного шляху підтвердження для відкриття сеансу, а приймач залишається прив'язаним до синхронізації. Цей глобальний огляд підтверджує, що експеримент починається у найсильнішій ворожій конфігурації: перед протоковою активністю немає жодного періоду прогріву на чистому каналі.

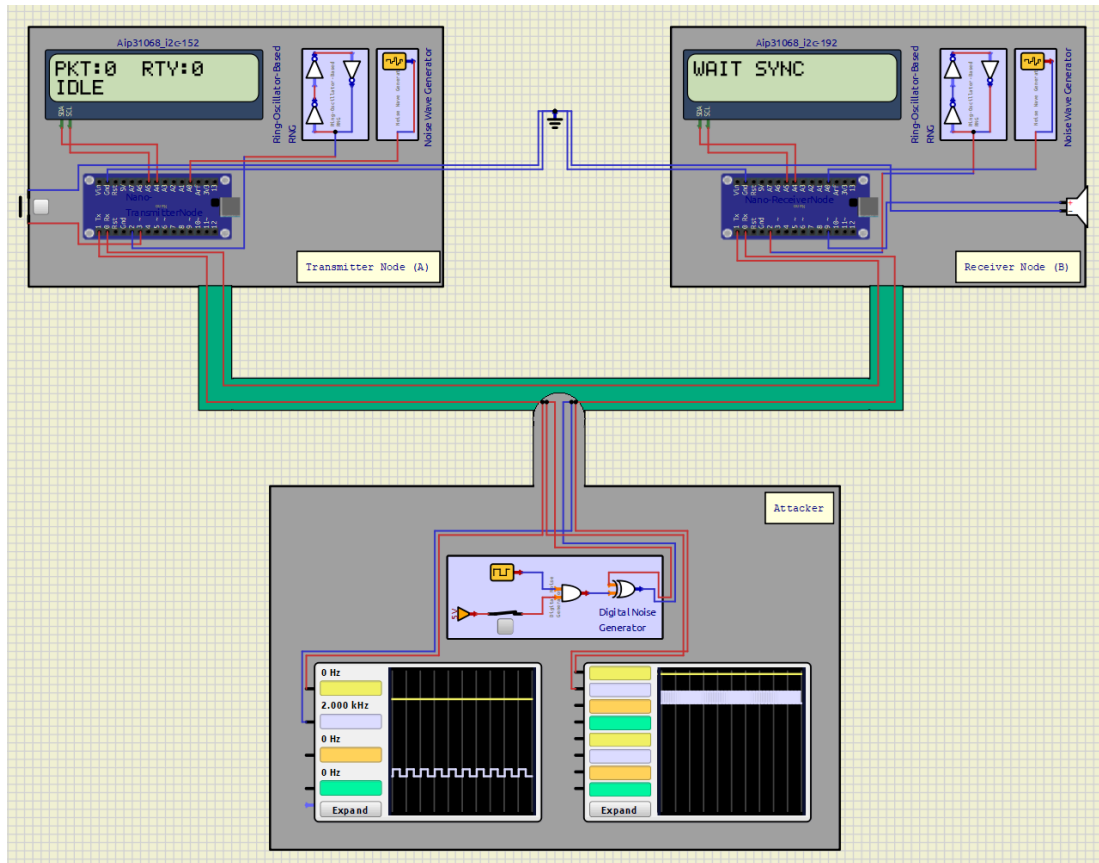


Figure 8.4: Повна топологія SH2SC-EDT під час запуску за активного глушіння. Підсхему атакувальника увімкнено від старту, і обидва кінці починають роботу зі збуреним прямим каналом.

Figure 8.5 надає фізичний доказ. Осцилограф показує стійку енергію завади на контрольованій лінії з окремими вкрапленнями трансмітерних імпульсів. Ключове спостереження тут — не просто зміна амплітуди, а часовий забруднюючий ефект: символіві вікна, які на UART-інтервалі мали б залишатися електрично стабільними, неодноразово перетинаються подіями інтерференції, що робить чисту реконструкцію байта статистично малоімовірною на старті.

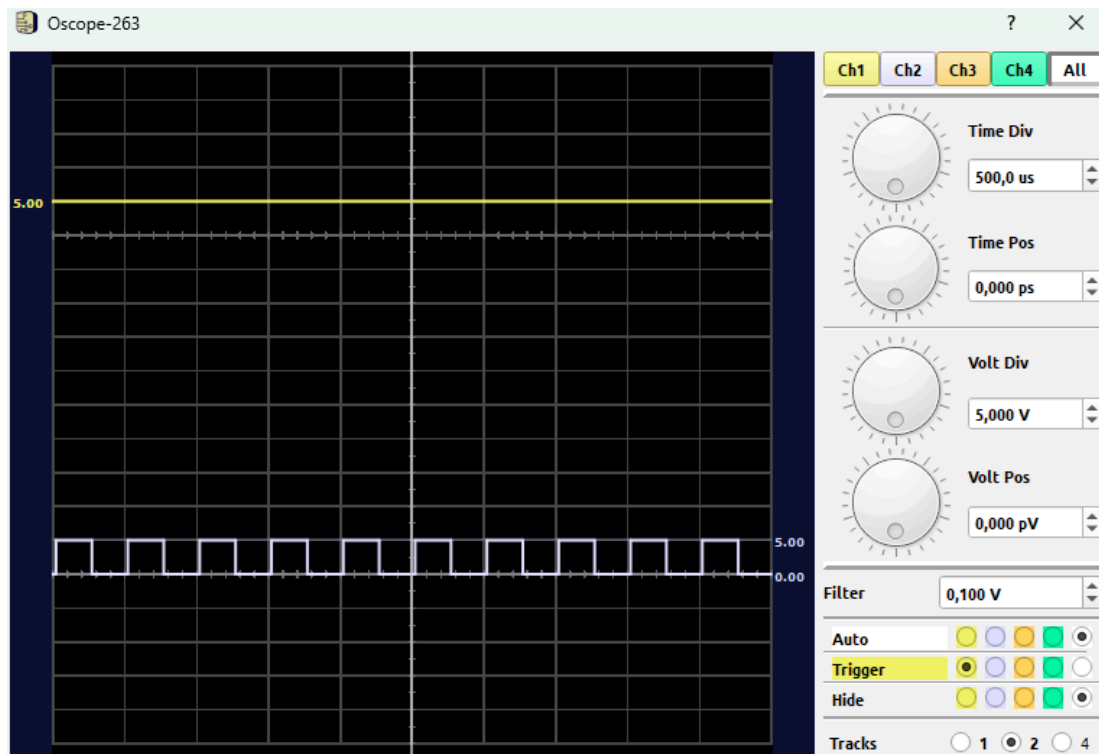


Figure 8.5: Осцилографічне підтвердження безперервних перешкод під час запуску. Збурена лінія залишається під домінуванням активності, спричиненої атакувальником, тоді як легітимні передавальні імпульси накладаються лише періодично.

Figure 8.6 показує відповідний наслідок на рівні протоколу. Logic analyzer інтерпретує спотворену хвилю як швидкі, несемантичні переходи, які не стабілізуються у валідну граматику кадру. Отже, приймач не може зачепитися за preamble-шлюз (0xAA 0x55), і парсер не може перейти до стійкого шляху накопичення payload. Поєднані докази Figure 8.5 і Figure 8.6 демонструють строгий фізико-протокольний причинний зв'язок: інверсії на рівні напруги, індуковані активним глушінням, достатні для того, щоб запобігти bootstrap сеансу на межі data-link.

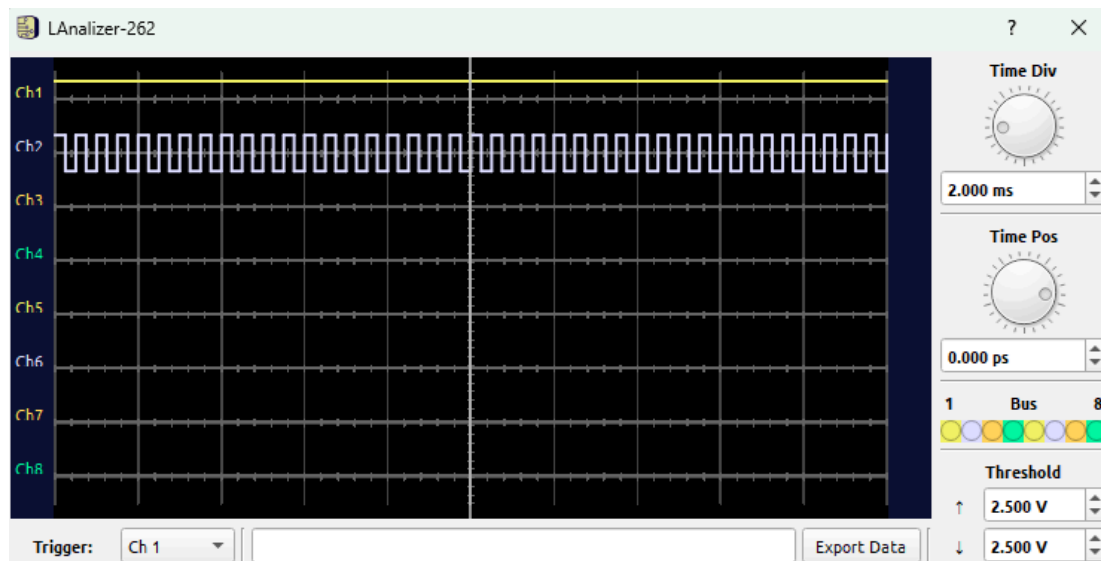


Figure 8.6: Подання логічного аналізатора того самого інтервалу запуску. Швидкі несемантичні переходи станів домінують у захопленому потоці, не даючи надійно виявити преамбулу та сформувати кадр.

8.3.2 Сценарій 2: базове передавання корисного навантаження (чистий канал)

Після десинхронізації на старті, спостережуваної у Сценарії 1, атакувальний пристрій тимчасово вимикається шляхом відкриття перемикача noise-injector. У цій конфігурації Digital Noise Generator вимкнений, а процес завади є спокійним:

$$N(t) = 0$$

Мета цього сценарію — встановити базову лінію того, як легітимна, зашифрована доставка DATA у C2P-ARQ проявляється на фізичному і протокольному рівнях, коли канал чистий, а кінцеві точки вже синхронізовані.

У базовому стані Node A (TX) передає зашифрований data packet, що містить application payload, а Node B (RX) успішно його приймає, виконує автентифіковане дешифрування, виконує декодовану команду та відповідає acknowledgment. Figure 8.7 фіксує глобальну топологію цього стану.

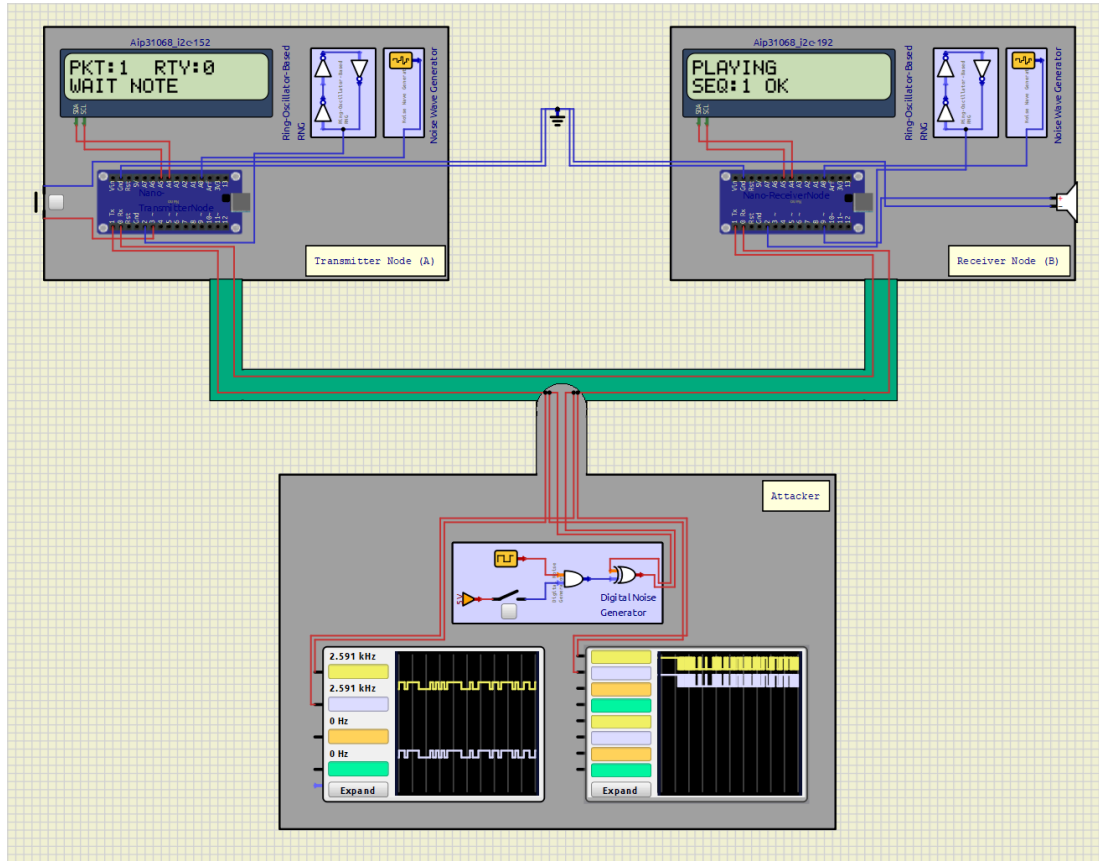


Figure 8.7: Топологія SH2SC-EDT під час базового передавання payload. Перемикач атакувальника відкрито, шумовий шлях електрично вимкнено, а Node A та Node B обмінюються даними без перешкод.

Вигляд топології робить припущення про чистий канал явним. Коли перемикач відкритий, ін'єктований noise-input до XOR-стадії утримується низьким; отже, forward UART-провідник несе waveform передавача без інверсій, індукованих атакувальником. Це ізолює поведінку протоколу від фізичного пошкодження каналу і надає референсний trace, з яким можна порівнювати наступні сценарії з активованою атакою.

Фізичний прояв цього baseline показано осцилографічним захопленням у Figure 8.8. За XOR injection model, що використовується в цьому розділі, отримана на приймачі хвиля є

$$S_{rx}(t) = S_{tx}(t) \oplus N(t) = S_{tx}(t) \oplus 0 = S_{tx}(t)$$

тому канал електрично прозорий. Осцилографічний trace демонструє чисті, різкі квадратні переходи між 0 V і 5 V, а також стабільні символові інтервали без бітових інверсій, накладених на UART-потік.

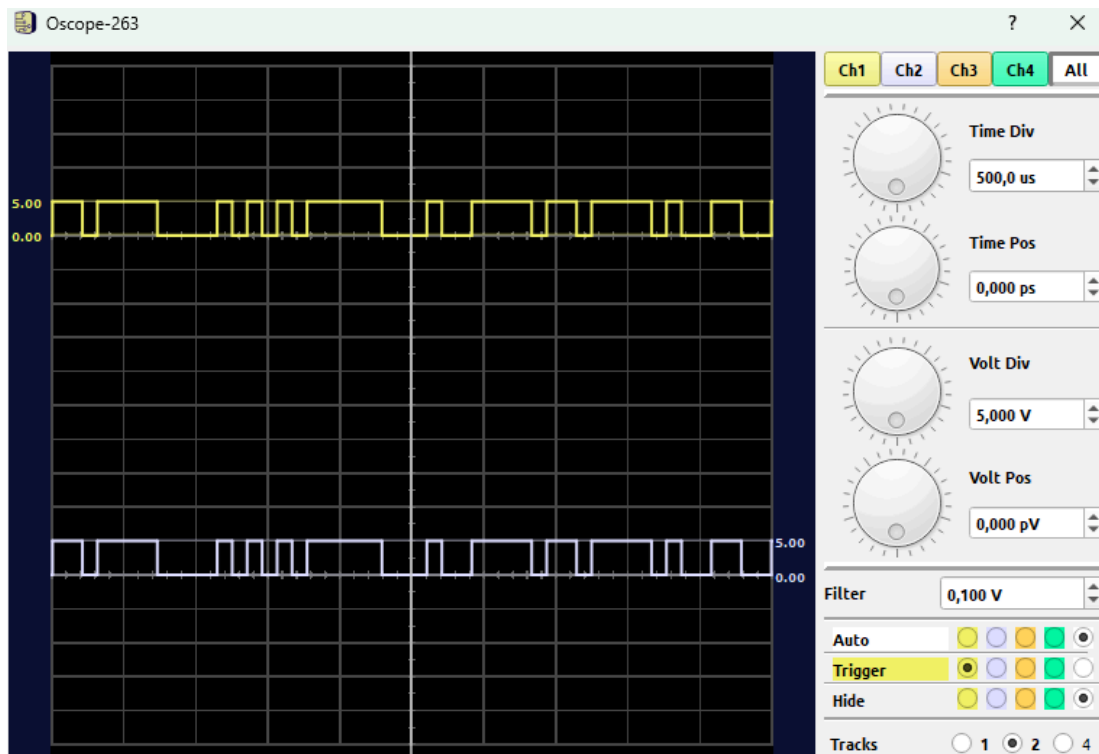


Figure 8.8: Осцилографічне підтвердження базової роботи за вимкненого шумового шляху. XOR-ін'єктор пропускає UART-сигнал без змін, утворюючи чисту квадратну форму без інверсій, спричинених атакувальником.

На рівні протоколу ця фізична чистота відповідає успішному framed exchange: Node A надсилає повний DATA-пакет, а Node B відповідає ACK після отримання й виконання дешифрованого payload. Потік повідомлень підсумовано в Figure 8.9.

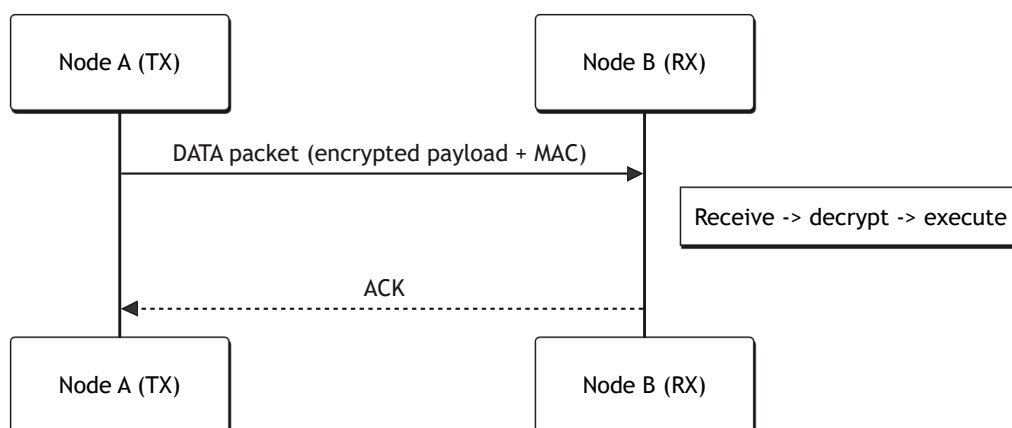


Figure 8.9: Успішний обмін DATA на ACK у чистому каналі C2P-ARQ. Приймач підтверджує кадр лише після отримання та виконання автентифікованого розшифрованого payload.

Figure 8.10 надає відповідний вигляд logic-analyzer. Аналізатор захоплює безперервне, добре framed передавання даних, у якому статичний preamble слідується тілом пакета, що містить зашифрований payload і MAC tag. Цей trace встановлює візуальну базову сигнатуру валідного C2P-ARQ frame за відсутності глушіння: стабільне захоплення preamble, безперервний потік байтів через тіло пакета та цілісна структура кадру, яку приймач може розібрати та валідувати end-to-end.

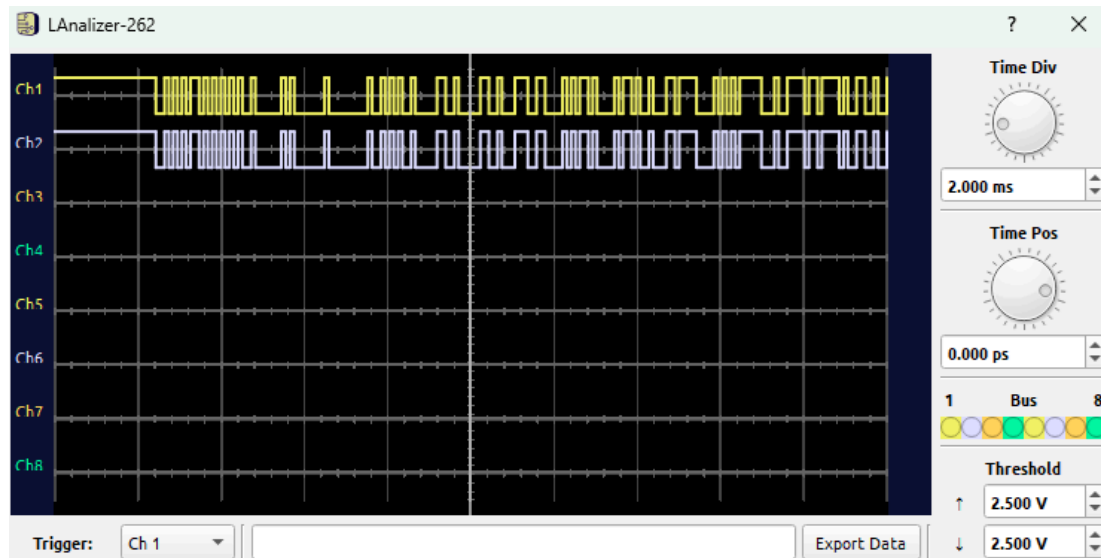


Figure 8.10: Базовий знімок логічного аналізатора валідного кадру даних C2P-ARQ у чистому каналі. Після статичної преамбули йдуть зашифрований payload і MAC-тег, що забезпечує безперешкодне захоплення та верифікацію кадру.

8.3.3 Сценарій 3: криптографічна варіація у наступних пакетах (чистий канал)

За все ще вимкненого Digital Noise Generator передавач продовжує надсилати наступний послідовний DATA-пакет тим самим чистим forward channel. Node A і Node B залишаються синхронізованими, і обмін завершується успішно, підтверджуючи, що будь-які спостережувані відмінності у захопленій формі пакета не спричинені фізичним пошкодженням каналу, а є наслідком криптографічної варіації на рівні протоколу.

Figure 8.11 надає глобальну топологію для цього другого чистого channel delivery. Перемикач атакувальника лишається відкритим, а LCD вузлів підтверджують просування до наступного пакета: Node A повідомляє збільшений індекс пакета без

повторів (PKT:2, RTY:0), тоді як Node B підтверджує автентифікований прийом (SEQ:2 OK) під час виконання payload.

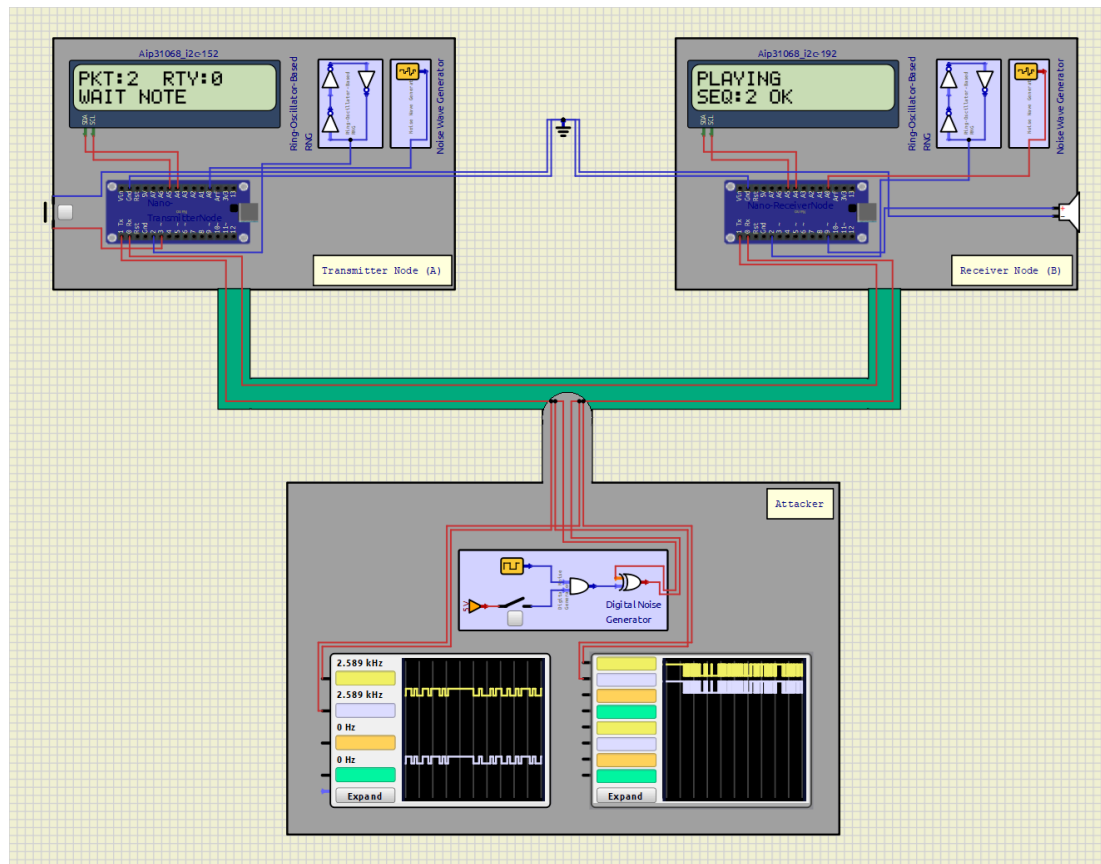


Figure 8.11: Топологія SH2SC-EDT під час наступної послідовної чистої передачі DATA. Атакувальник залишається вимкненим, а стан LCD підтверджує збільшений номер пакета без повторних передач.

Фізичний рівень залишається стабільним у Figure 8.12. Осцилографічне захоплення показує чисту UART-хвилю, сумісну з baseline у Figure 8.8: різкі переходи і стабільні логічні рівні без інверсій полярності, викликаних завадою. Це обмежує інтерпретацію наступних протокольних trace: канал електрично надійний, тому будь-які відхилення в pattern даних, спостережуваних logic analyzer, мають походити з вмісту пакета, а не з шумового пошкодження.

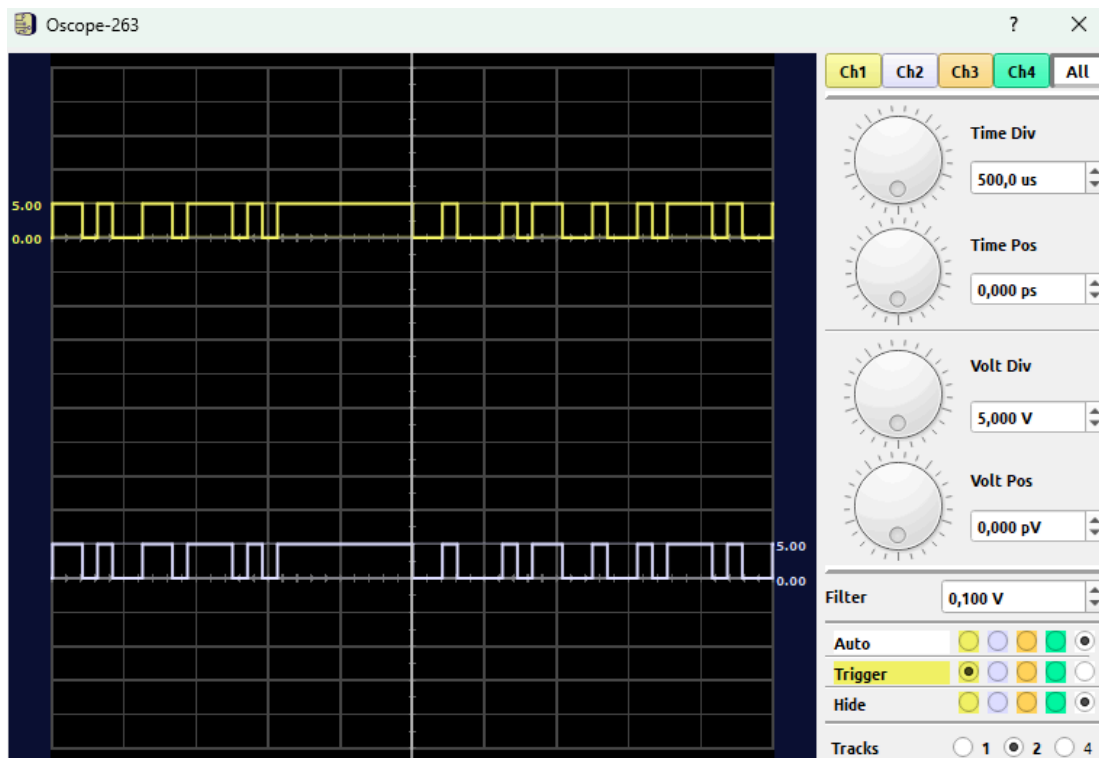


Figure 8.12: Осцилографічне підтвердження подальшої роботи на чистому каналі під час наступного DATA-пакета. Фізичний рівень залишається чистим і стабільним, ізолюючи відмінності на рівні пакета від впливу перешкод.

Криптографічний ефект найчіткіше видно у захопленні logic analyzer в Figure 8.13. У прямому порівнянні з попереднім чистим пакетом (Figure 8.10), preamble виглядає візуально ідентичним, як і очікується: ця синхронізаційна структура свідомо статична, щоб підтримувати детерміноване frame acquisition на приймачі. Натомість ділянка після preamble демонструє істотно іншу послідовність high/low без жодної візуально очевидної кореляції з payload/MAC-сегментом попереднього пакета.

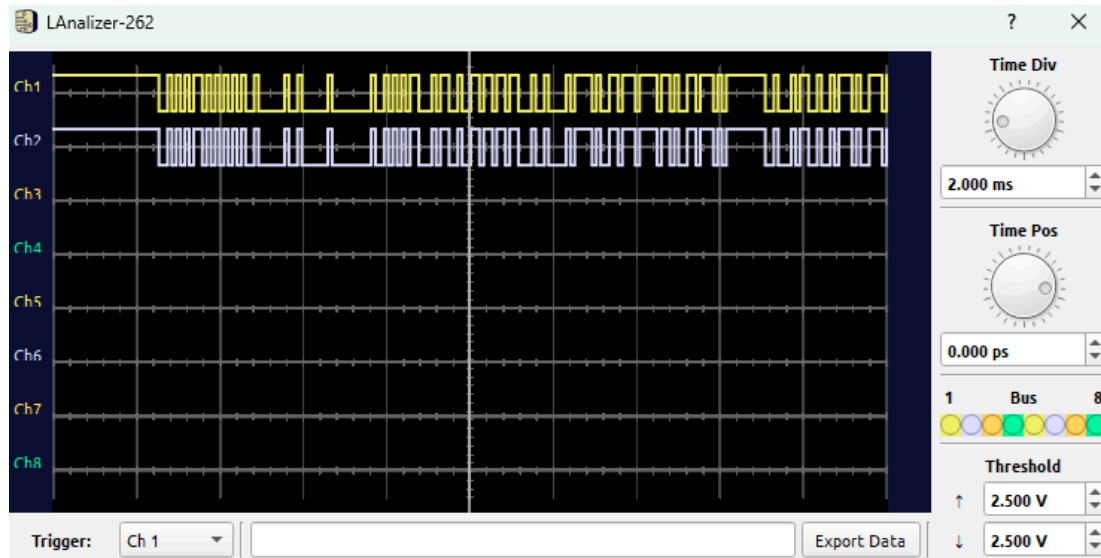


Figure 8.13: Знімок логічного аналізатора наступного послідовного кадру DATA за умов чистого каналу. Преамбула збігається з попереднім пакетом, тоді як шифротекст і область MAC не демонструють видимої кореляції, що узгоджується із семантичною стійкістю ChaCha20-Poly1305 за зміни nonce.

Ця відсутність спостережуваної структури є очікуваним проявом semantic security на рівні data-link. Навіть якщо розмір прикладної команди між сусідніми пакетами не змінюється, інкрементований номер послідовності в протоколі породжує окремий per-packet nonce, що дає новий ChaCha20 keystream і, відповідно, інший шифротекст для зашифрованого payload. Оскільки Poly1305 tag обчислюється за authenticated header і ciphertext, байти MAC також змінюються між пакетами. Отже, отриманий після preamble waveform поводить себе як унікальний для одного пакета шаблон для зовнішнього спостерігача, запобігаючи надійному зіставленню між послідовними передаваннями та підриваючи будь-яку спробу кореляційного traffic inference на forward channel.

8.3.4 Сценарій 4: активне глушіння, пошкодження корисного навантаження і десинхронізація станів

У цьому сценарії атакувальник повторно активує Digital Noise Generator під час передавання пакета з payload. Отже, forward channel керується не спокійним disturbance process,

$$N(t) \neq 0$$

і вхід приймача стає XOR-композиційною хвилею, а не прямою репрезентацією transmitter line. Експеримент оголює критичний режим відмови у вбудованій комунікації під інтерференцією: передавач і приймач можуть увійти у розбіжні оперативні стани навіть тоді, коли обидва кінці продовжують коректно виконувати свою локальну логіку.

Figure 8.14 фіксує глобальну топологію під час цього інтервального нападу. Node A вже просунувся до packet (#145) і спостерігається у стані WAIT ACK із двома вже спожитими повторними спробами (RTY: 2). Node B, однак, все ще виконує попередній автентифікований payload, що відповідає packet (#144), позначений як PLAYING SEQ:144 OK. Ця одночасна спостережуваність встановлює десинхронізацію станів: передавач намагається доставити наступний stop-and-wait DATA frame, тоді як приймач лишається зайнятим виконанням на рівні застосунку попередньої команди.

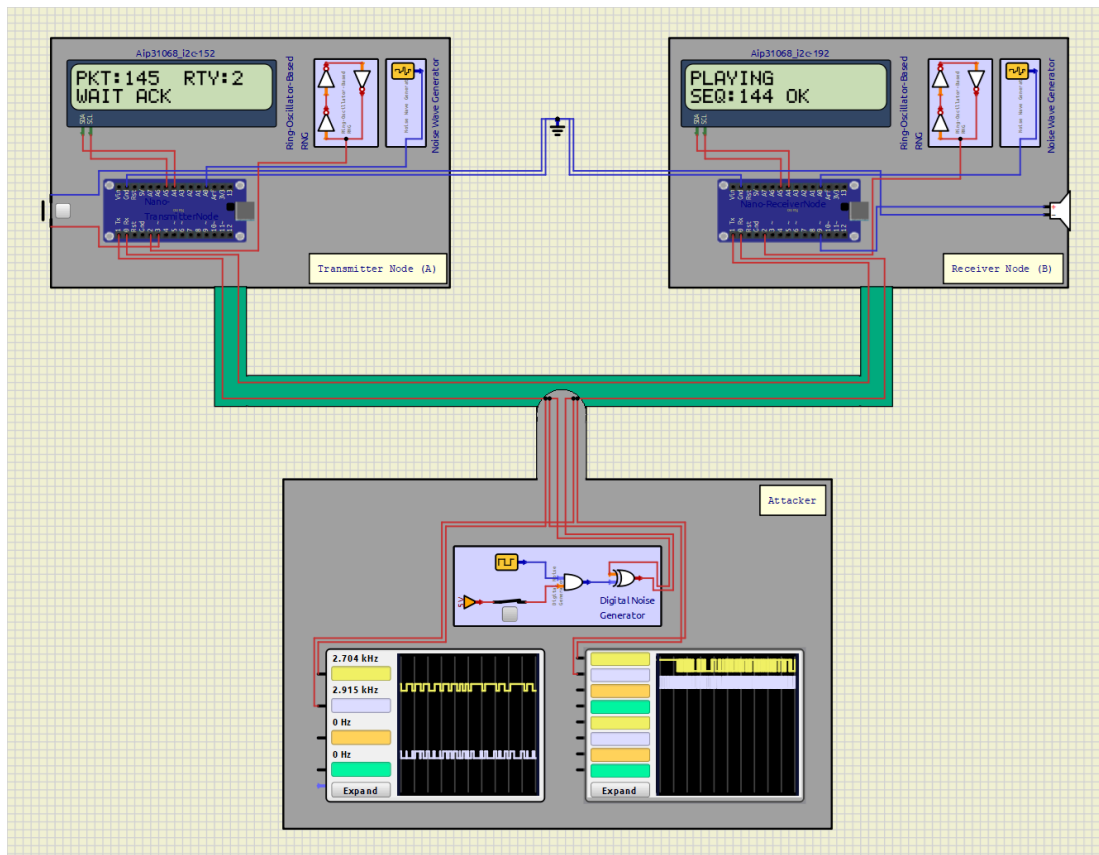


Figure 8.14: SH2SC-EDT topology under active jamming during payload delivery. Node A is stalled in WAIT ACK for PKT:145 with RTY:2, while Node B is still in PLAYING state for SEQ:144 OK, evidencing protocol-state divergence under interference.

Фізичний механізм, що спричиняє цю розбіжність, показано в Figure 8.15. За увімкненого атакувальника waveform на боці приймача задовольняє

$$S_{rx}(t) = S_{tx}(t) \oplus N(t)$$

тому будь-який інтервал, у якому

$$N(t) = 1$$

породжує інверсію миттєвого UART-рівня. Осцилографічний trace демонструє суттєво спотворений serial-сигнал порівняно з чистим baseline: переходи відбуваються всередині символічних вікон, які в іншому разі лишалися б стабільними, і бітова структура, необхідна для надійної асинхронної реконструкції, неодноразово порушується. За таких умов коректно отримати повний preamble, header, ciphertext і MAC як єдиний coherent frame стає малоімовірним навіть якщо приймач був би виключно присвячений парсингу.

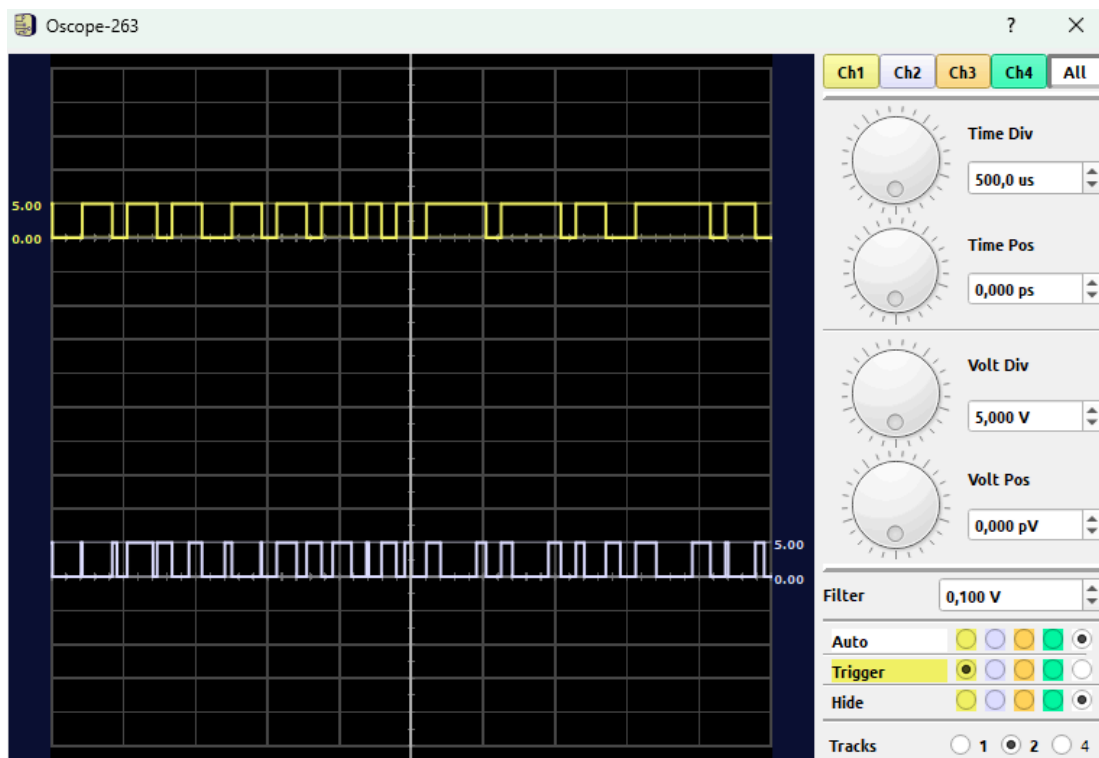


Figure 8.15: Oscilloscope evidence of physical-layer corruption under active jamming. XOR-injected disturbance introduces bit-level inversions into the forward UART waveform during DATA transmission.

Протокольний прояв показано в logic analyzer захопленні Figure 8.16. На відміну від чистих baselines (Figure 8.10 і Figure 8.13), jammed trace не демонструє стійкого,

інтерпретованого packet body після послідовного preamble. Натомість захоплена послідовність high/low доміновано несемантичними переходами, які стирають структурні інваріанти, потрібні для framing та authenticated processing. Це спостереження забезпечує ключовий security-relevant висновок: відмова тут не є AEAD-відхиленням добре сформованого ciphertext, а є попереднім знищенням цілісності пакета на межі data-link так, що валідний кадр може взагалі не бути поданий до криптографічної стадії перевірки.

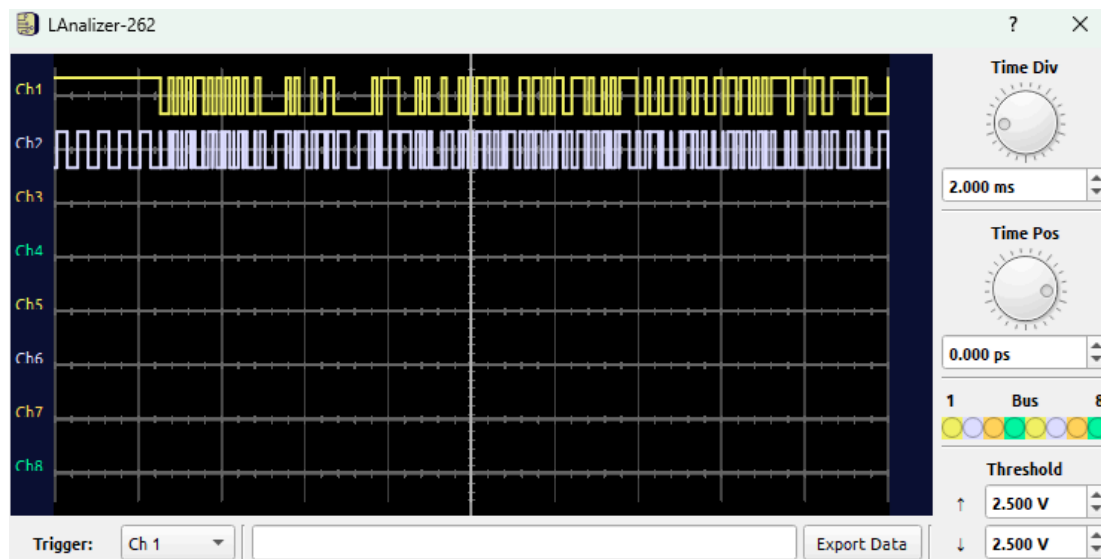


Figure 8.16: Logic-analyzer view of the jammed forward channel. The packet's structural integrity is destroyed relative to the clean baseline, preventing stable framing and end-to-end validation.

Наслідки ARQ і на рівні застосунку видно безпосередньо на дисплеях вузлів. Figure 8.17 показує Node A, застряглий на PKT:145 з RTY:2 у WAIT ACK, що підтверджує активні спроби stop-and-wait повторно проштовхнути той самий пакет payload через зіпсований канал без отримання потрібного acknowledgement. Figure 8.18 показує Node B у стані PLAYING з SEQ:144 OK, підтверджуючи, що приймач і далі зайнятий попередньою автентифікованою командою і ще не просунувся до наступного значення sequence. Навіть якщо Node B завершить поточне виконання і відновить активне слухання, фізичні та логічні докази (Figure 8.15 і Figure 8.16) свідчать, що спроби доставити packet (#145) у цей проміжок розриваються на рівні waveform.

Результуючу взаємодію підсумовано в Figure 8.19. Поки Node B залишається у тривалому вікні виконання packet (#144), Node A передає packet (#145) і входить у стан WAIT ACK. Кожна спроба передачі, зіпсована глушінням, не породжує валідний acknowledgement, що веде до послідовних повторів.

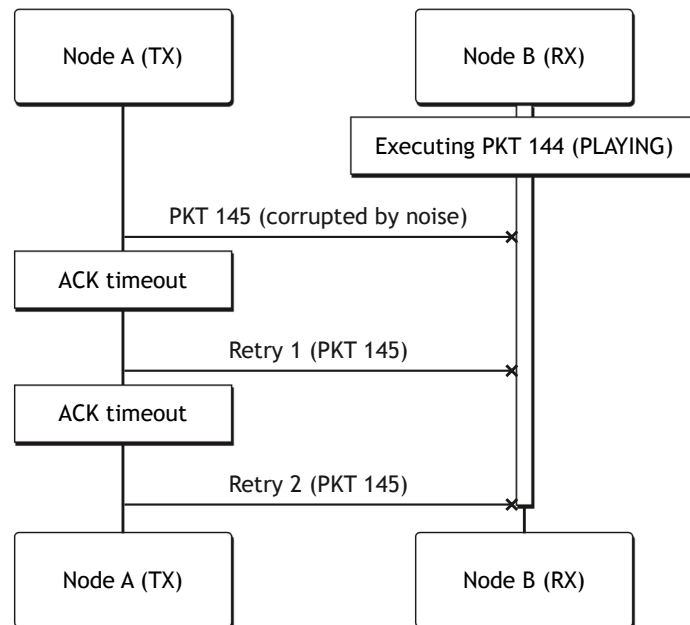


Figure 8.19: Десинхронізація за активного глушіння. Node B залишається зайнятим PKT 144, тоді як заглушені передачі PKT 145 не дають ACK, змушуючи Node A повторювати спроби.

8.3.5 Сценарій 5: тайм-аут передавача та призупинення сеансу

Активне глушіння триває без жодної зміни у фізичному механізмі атаки. Відмінною подією в цьому сценарії є, отже, не новий waveform signature, а протокольний захист: ARQ-логіка передавача досягає hardcoded retransmission bound для поточного DATA-пакета (`MAX_RETRIES` = 50). Замість того щоб безкінечно лишатися у стані очікування, FSM передавача трактує цю умову як серйозну й тривалу деградацію каналу та ініціює контрольоване призупинення сеансу.

На Node A цей перехід спостерігається як явний teardown поточного криптографічного сеансу з подальшим входом у стан RECONNECTING. З операційної точки зору така поведінка запобігає зависанню транспортного двигуна мікроконтролерного класу в нескінченному циклі повторів під час тривалої denial-of-service ситуації: коли бюджет повторів вичерпано, система повертається до reconnect-

ion posture і пізніше може знову встановити свіжий сеанс, коли канал стане придатним.

На Node B жодного відповідного переходу в цей момент не спостерігається. Приймач повністю зайнятий виконанням фізичного payload попередньо автентифікованого пакета (#144), підтримуючи поточне відтворення delay/melody і ще не досягнувши жодного локального порога тайм-ауту. У цьому стані він не знає, що передавач уже припинив сеанс і перейшов до reconnection logic; вузол локально коректний щодо своєї останньої перевіреної команди, тоді як глобально сеанс уже було перервано передавачем.

Осцилографічні та logic-analyzer фігури тут навмисно опущено, оскільки фізичні умови каналу залишаються тими самими, що і в Сценарії 4: XOR noise injector продовжує збурювати forward UART-лінію тим самим способом. Фокус у цьому сценарії — суворо на upper-layer timeout і state-machine response на передавачі.

Переходження передавача в стан призупинення сеансу та reconnection posture показано в Figure 8.20.

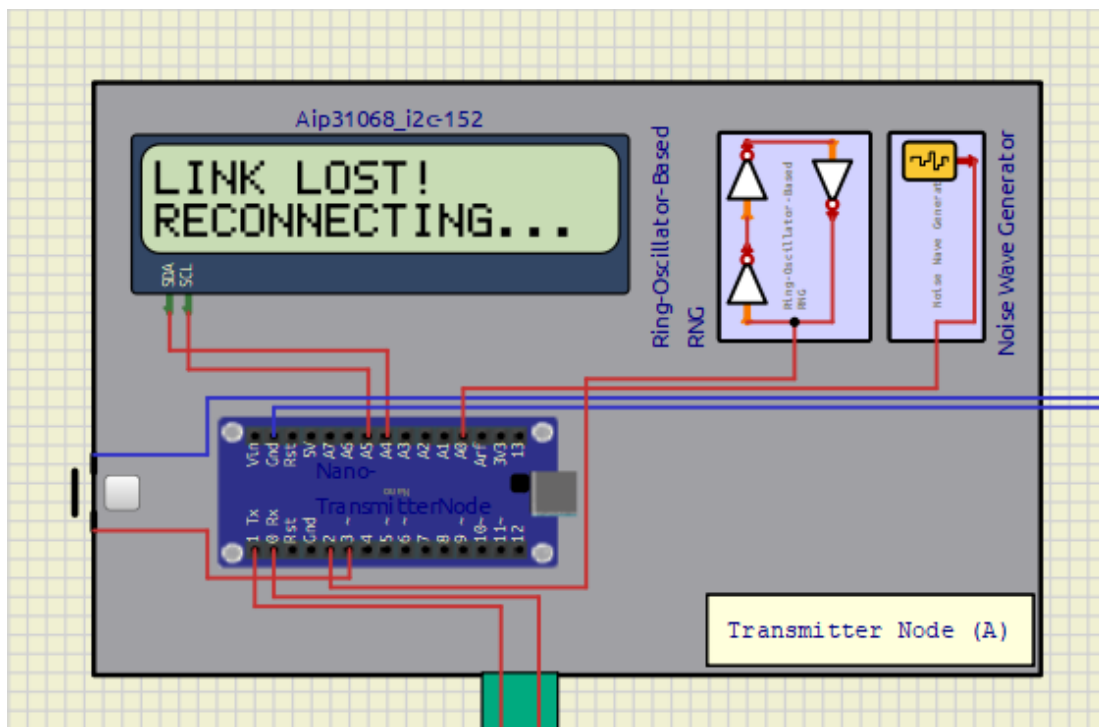


Figure 8.20: Вузол А входить у запобіжний режим призупинення сеансу після вичерпання бюджету повторних передавань. Повідомлення LCD LINK LOST! RECONNECTING... фіксує момент, коли передавач залишає поточний сеанс і переходить до логіки повторного з'єднання за тривалого активного глушіння.

8.3.6 Сценарій 6: взаємне завершення сеансу та мертвий ефір

У цьому сценарії Digital Noise Generator залишається ввімкненим, але transport-layer interaction перейшла в інтерстиціальний стан, у якому на forward channel немає жодних валідних protocol frames. Сеанс тепер завершено на обох кінцях: Node A вже призупинив свій сеанс і залишається в RECONNECTING, тоді як Node B повернувся після тривалого виконання на рівні застосунку і згодом оголосив сеанс мертвим через свій fail-safe, керований watchdog.

На приймачі (Node B) блокувальний payload, пов'язаний із packet (#144), нарешті завершився. Коли вузол відновлює listening на мережевому рівні, його internal watchdog спрацьовує, бо жоден authenticated packet не надходив упродовж тривалості execution gap і наступного dead interval. У результаті Node B коректно скидає сеанс, очищує свій криптографічний стан і входить у статус TIMEOUT! DROP, що означає контрольоване звільнення session RAM і security context замість безкінечного утримання під час мовчання, викликаного атакувальником.

На передавачі (Node A) стан лишається RECONNECTING, як і було встановлено в Сценарії 5. У цій позі Node A готується ініціювати новий сеанс, щойно forward channel стане придатним. Проте за наявності активного атакувальника середовище й далі доміноване енергією завади, і не існує стабільного інтервалу, у якому можна спостерігати валідний обмін кадрами.

Тут включено лише осцилографічне захоплення для цього проміжного стану. Оскільки жоден вузол не передає валідні C2P-ARQ кадри, logic analyzer лише декодував би послідовність випадкових перемикачів, які не містять preamble acquisition, структури заголовка чи будь-якого змістовного data-link контексту. Осцилограф, навпаки, безпосередньо відображає фізичний стан лінії і тому може показати, що канал насичений ворожою інтерференцією навіть тоді, коли легітимні кінцеві точки є квазіспокійними.

Панорамний вигляд dead-air стану подано в Figure 8.21. Ключове спостереження: підсхема атакувальника лишається ввімкненою, тоді як жоден з endpoint не підтримує активний криптографічний сеанс: передавач перебуває в reconnection posture, а приймач скинув session state після висновку, що канал більше не живий.

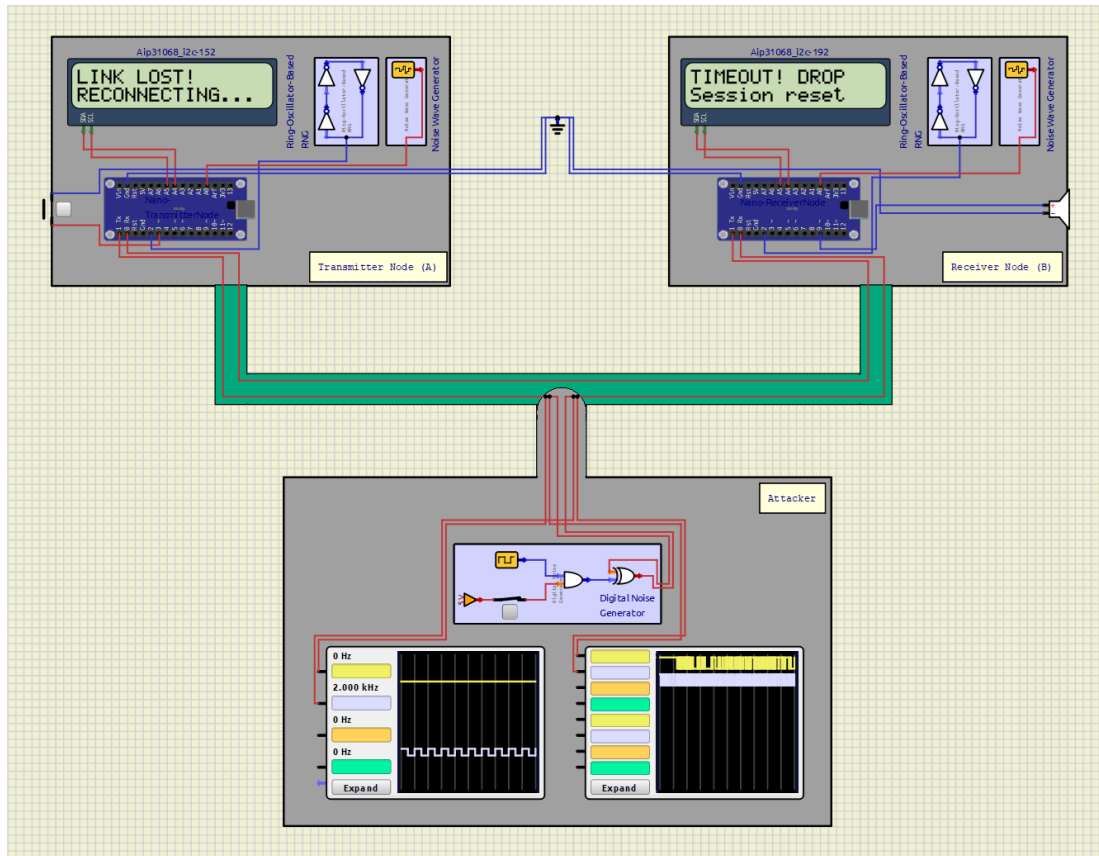


Figure 8.21: Топологія dead-air за активного глушіння після взаємного завершення сеансу. Атакувальник залишається увімкненим, тоді як Node А перебуває в стані RECONNECTING, а Node В скинув сеанс після спрацювання watchdog.

Figure 8.22 фіксує докази коректного teardown на боці приймача. Повідомлення LCD `TIMEOUT! DROP / Session reset` демонструє, що fail-safe механізм Node В активно відкинув застарілий session context і очистив свій криптографічний стан, запобігаючи безстроковому утриманню компрометованого або недосяжного сеансу під час тривалого глушіння.

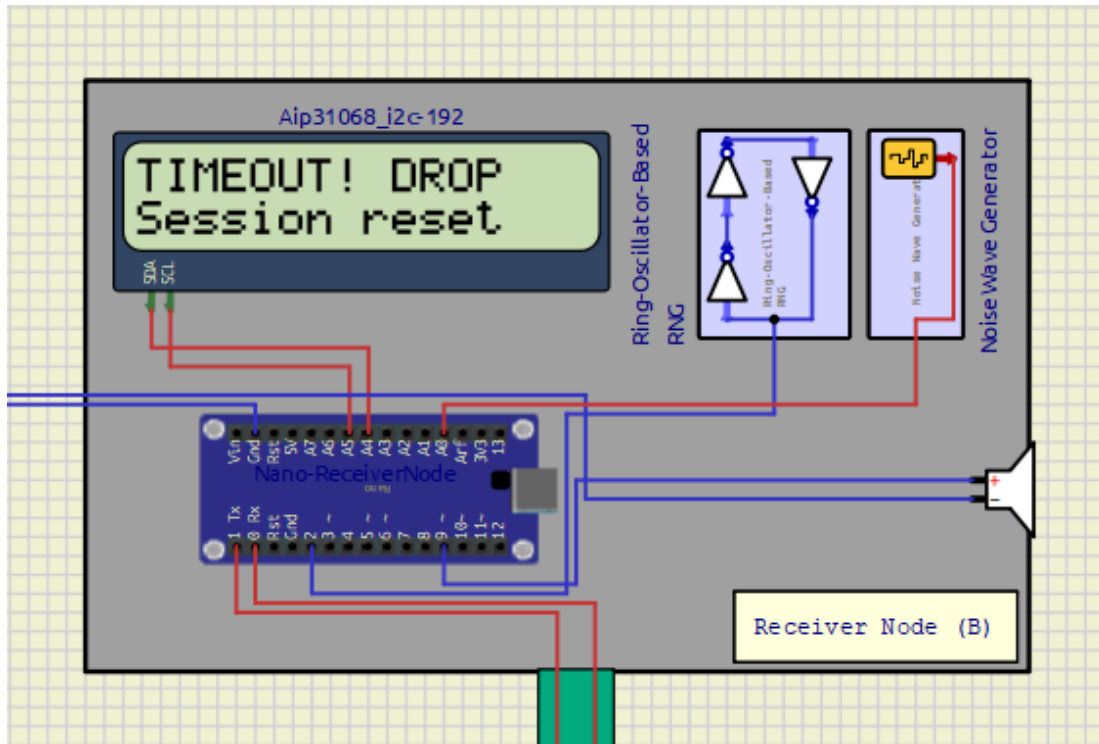


Figure 8.22: Node B вказує на завершення сеансу, кероване watchdog. Повідомлення LCD TIMEOUT! DROP / Session reset підтверджує, що приймач очистив покинутий сеанс і повернувся до безпечного режиму очікування за продовженого придушення каналу.

Фізику цього dead-air каналу показано за XOR injection model під час idle transmitter. Якщо вихід передавача є квазістатичним і утримується на сталому логічному рівні,

$$S_{tx}(t) = Const$$

тоді receiver-side line стає

$$S_{rx}(t) = Const \oplus N(t)$$

де $N(t)$ — процес завади атакувальника, поданий цифровим генератором шуму. Оскільки $Const$ не несе інформаційно-змістовних переходів, будь-яка спостережувана edge activity на вході приймача зумовлена виключно $N(t)$. Figure 8.23 візуально підтверджує цей стан: trace осцилографа доміновано безперервною інтерференцією, що демонструє, що фізичне середовище в цьому інтерстиціальному стані заповнене лише енергією супротивника.

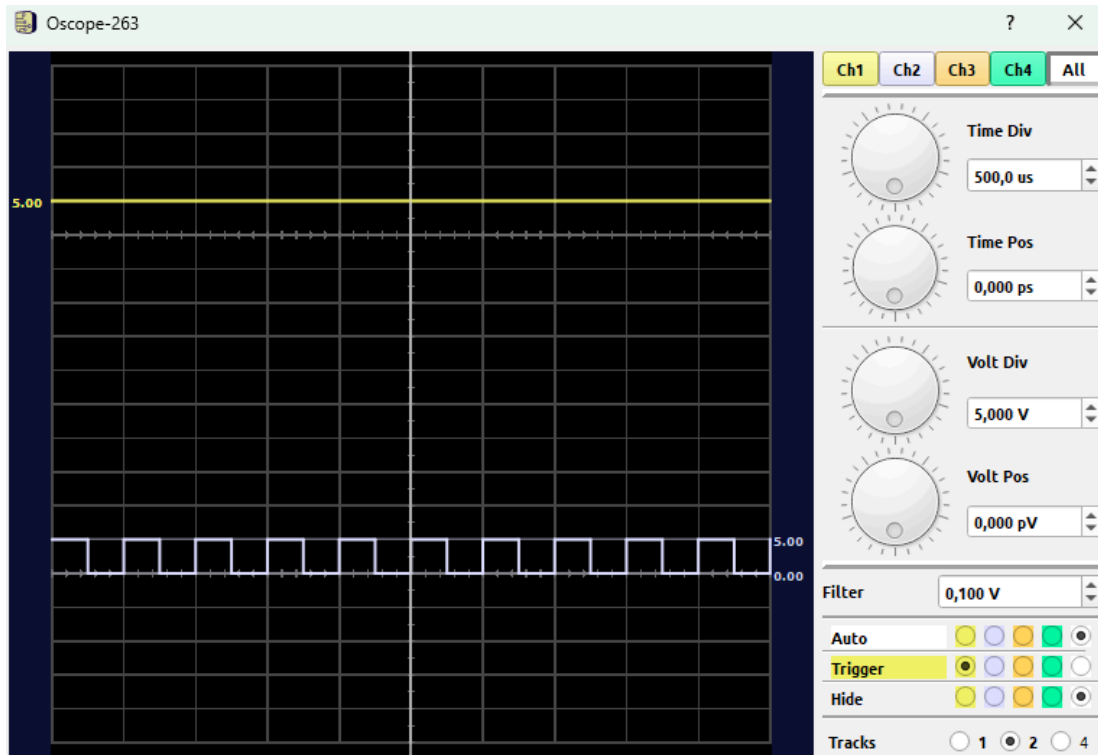


Figure 8.23: Осцилографічне підтвердження dead air, домінованого атакувальником. За фіксованого низького рівня передавача XOR-ін'єктор переносить процес збурення на вхід приймача, утворюючи форму сигналу, де переходи зумовлені виключно шумом.

8.3.7 Сценарій 7: спроби повторної ініціалізації сеансу під активним глушінням

Після взаємного завершення сеансу, встановленого у Сценарії 6, SH2SC-EDT переходить у шлях self-healing reconnection, тоді як Digital Noise Generator залишається ввімкненим. Відповідно, forward channel і далі підлягає безперервній XOR-ін'єктованій заваді, і система має намагатися переініціалізувати сеанс без будь-якого чистого фізико-рівневого інтервалу.

На передавачі (Node A) попередній стан сеансу очищено, і запитується новий криптографічний сеанс. Цей перехід підтверджується скиданням лічильника пакетів на нуль (PKT:0) і повторним випромінюванням C2P-ARQ HelloPacket у очікуванні відповіді на handshake (WAIT SYN). Лічильник повторів уже досяг RTY:21, що вказує на стійку невдачу отримати валідний ACK за умов тривалої інтерференції.

На приймачі (Node B) вузол лишається в dropped state (TIMEOUT! DROP). Оскільки forward channel усе ще глушиться, приймач не може надійно захопити й декодувати вхідні HelloPacket кадри і, отже, не може встановити новий session ponсе чи відповісти acknowledgement. Результуюча поведінка системи є стабільним,

детермінованим reconnection loop: спроби повторної ініціалізації сеансу безперервно виконує Node A, тоді як Node B лишається безпечно скинутим і не бере на себе жодних зобов'язань, доки не буде спостережено коректний session-open frame.

Панорамний вигляд цього reconnection interval показано в Figure 8.24. Jammer явно активний, і обидва кінці перебувають у post-teardown станах: Node A намагається встановити новий handshake, тоді як Node B перебуває у fail-safe стані, що не дозволяє йому діяти на неавтентифікований або структурно пошкоджений трафік.

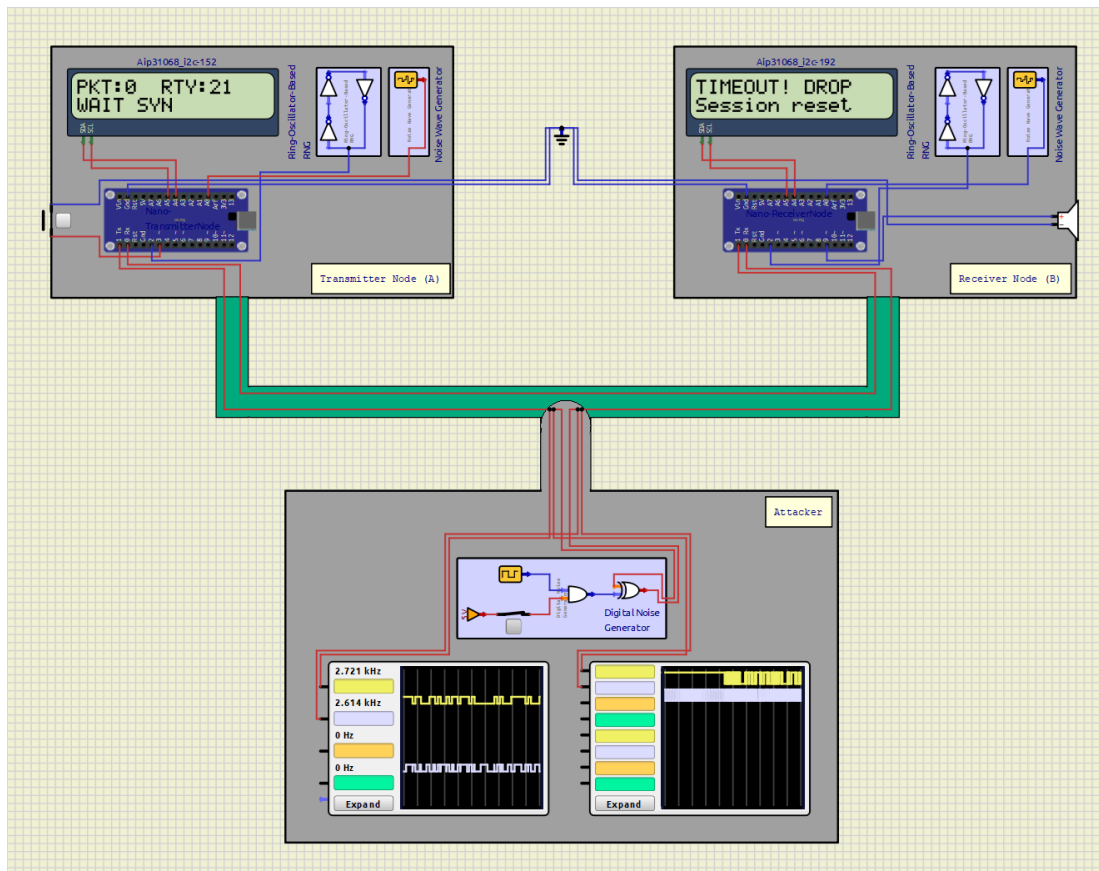


Figure 8.24: Топологія під час спроб повторної ініціалізації сеансу за активного глушіння. Атакувальник залишається увімкненим, тоді як Node A неодноразово передає кадри HelloPacket, а Node B залишається у стані скинутого сеансу.

Figure 8.25 пов'язує reconnection attempt з явним FSM-visible станом обох вузлів. Дисплей передавача підтверджує, що це не продовження попереднього data stream, а новий запит сеансу (лічильник скинуто), і що повторні тайм-аути вже накопичуються. Симетрично, дисплей приймача підтверджує, що watchdog-driven

teardown залишив вузол у безпечному disconnected стані, доки не буде розібрано валідний session-open frame.

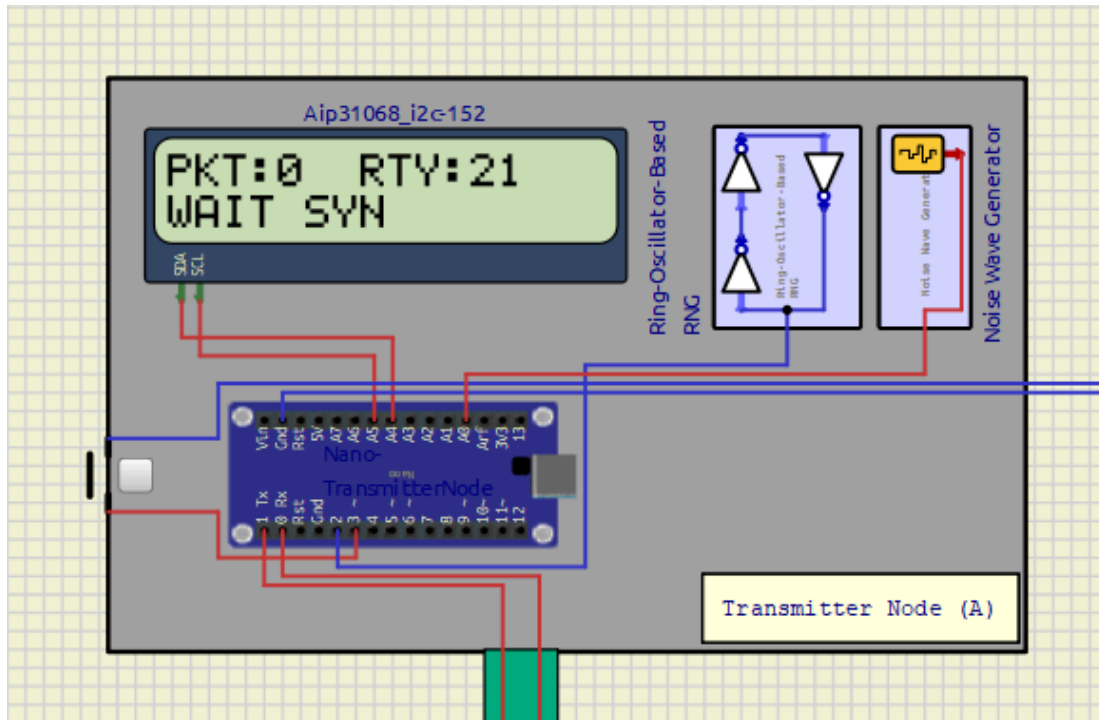


Figure 8.25: Стан вузлів під тиском повторного з'єднання. Node A показує PKT:0 RTY:21 WAIT SYN, підтверджуючи нову спробу сеансу та повторні спроби, тоді як Node B залишається у TIMEOUT! DROP, що означає: новий сеанс не був прийнятий.

На фізичному рівні передавач більше не є квазіспокійним, як у Сценарії 6. Натомість він активно випромінює HelloPacket waveform у канал, який і далі домінований процесом завади атакувальника. За моделлю XOR injection отриманий на приймачі сигнал задовольняє

$$S_{rx}(t) = S_{tx_{hello}}(t) \oplus N(t)$$

де $S_{tx_{hello}}(t)$ — чиста UART-хвиля, що відповідає повторним HelloPacket передачам, а $N(t)$ — disturbance, згенерований атакувальником.

Figure 8.26 демонструє очікуваний наслідок

$$N(t) \neq 0$$

під час активного передавання: чиста, періодична структура асинхронних серійних бітів неодноразово порушується інверсіями, індукованими завадою, тому receiver-side

waveform більше не зберігає стабільні per-bit windows, необхідні для детермінованого декодування.

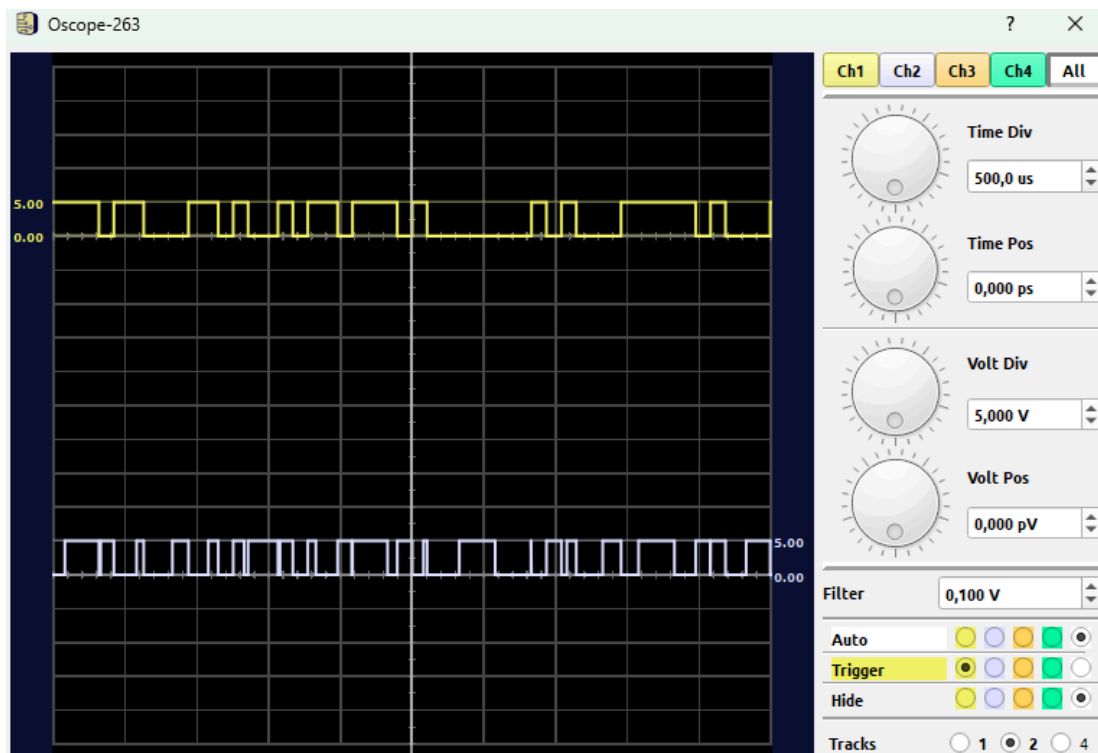


Figure 8.26: Осцилографічне підтвердження передавання HelloPacket за тривалого глушіння. Активна форма сигналу обміну руйнується збуренням, ін'єктованим через XOR, що заважає стабільному відновленню символів на вході приймача.

Figure 8.27 надає валідацію структури протоколу попри ворожий канал. Порівняно з encrypted DATA packets, захопленими раніше у Figure 8.10 і Figure 8.13, спостережуваний frame є коротшим за тривалістю, що відповідає дизайну HelloPacket (preamble плюс компактне session-request body, без ciphertext-and-MAC tail, характерного для DATA frame). Критично, preamble pattern лишається візуально збереженим і інваріантним для всіх типів пакета. Така інваріантність є структурною вимогою C2P-ARQ: фіксований preamble дає змогу parser-у приймача зберігати детермінований synchronization gate навіть тоді, коли поля вищого рівня спотворені шумом.

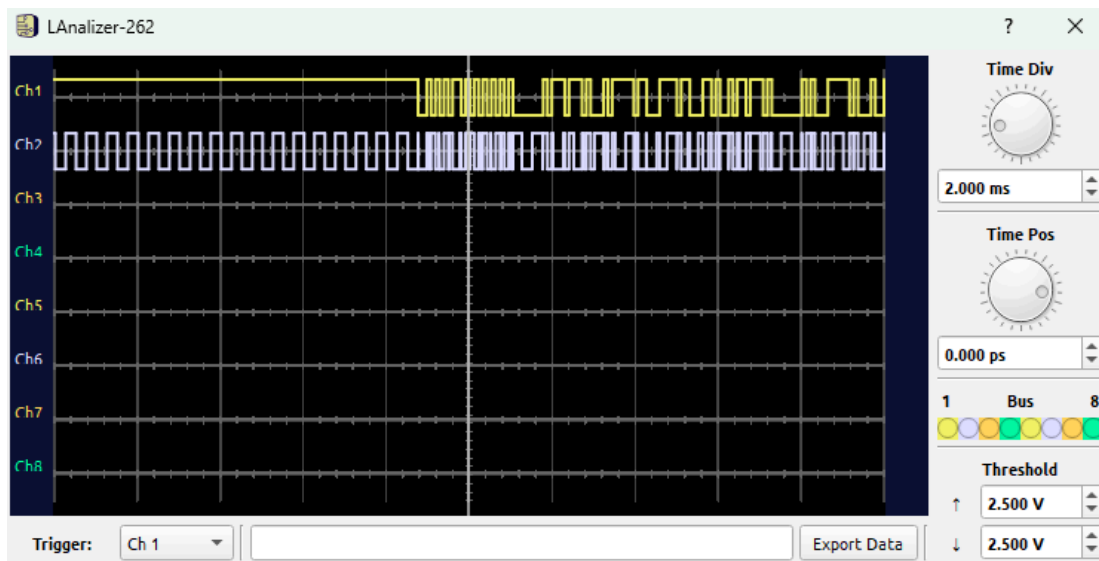


Figure 8.27: Подання логічного аналізатора повторних спроб HelloPacket під час глушіння. Загальна тривалість кадру помітно коротша за зашифровані DATA-кадри сценаріїв 2 і 3, тоді як синхронізаційна преамбула залишається строго ідентичною, фізично підтверджуючи специфікацію кадру C2P-ARQ.

Reconnect loop підсумовано в Figure 8.28. Node A повторно надсилає HelloPacket session requests; кожна спроба фізично пошкоджується поточним глушінням і не породжує acknowledgment, що спричиняє ARQ timeout і інкремент retry counter. Node B лишається disconnected і не надсилає жодної відповіді, доки не буде прийнято коректний session-open frame.

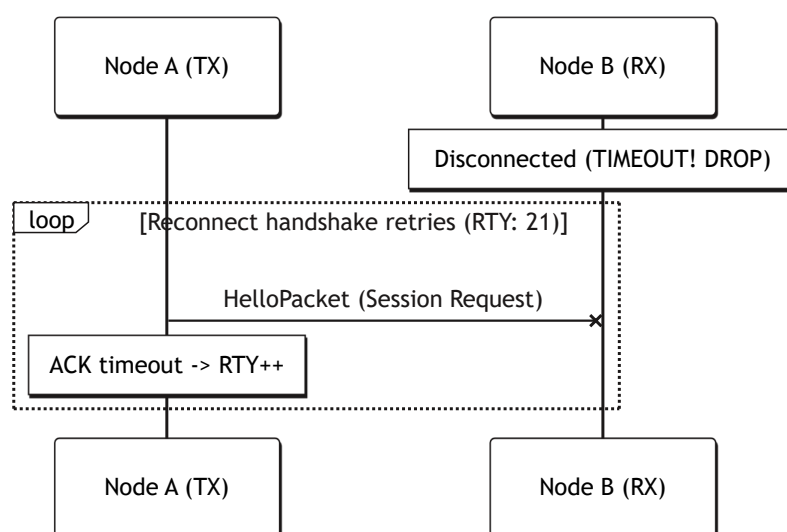


Figure 8.28: Спроби повторної ініціалізації сеансу за активного глушіння. Пошкоджені передачі HelloPacket не дають ACK, змушуючи Node A циклічно

залишатися у WAIT SYN зі зростаючим лічильником повторів, тоді як Node B безпечно залишається скинутим.

8.3.8 Сценарій 8: успішне відновлення сеансу та збереження стану застосунку

У цьому сценарії тривале фізичне denial нарешті припиняється: атакувальник вимикає Digital Noise Generator, і процес завади стає спокійним,

$$N(t) = 0$$

Після очищення forward channel шлях self-healing C2P-ARQ може завершитися. Session request у вигляді HelloPacket, що надсилає передавач, тепер доставляється з достатньою структурною цілісністю, щоб приймач його допустив, встановлюючи новий криптографічний сеанс. Нормальне передавання даних відновлюється негайно після цього.

Панорамний вигляд моменту відновлення показано в Figure 8.29. Jammer вимкнено, і система повертається до нормальної роботи, де обидва кінці активно беруть участь у transport-layer exchange.

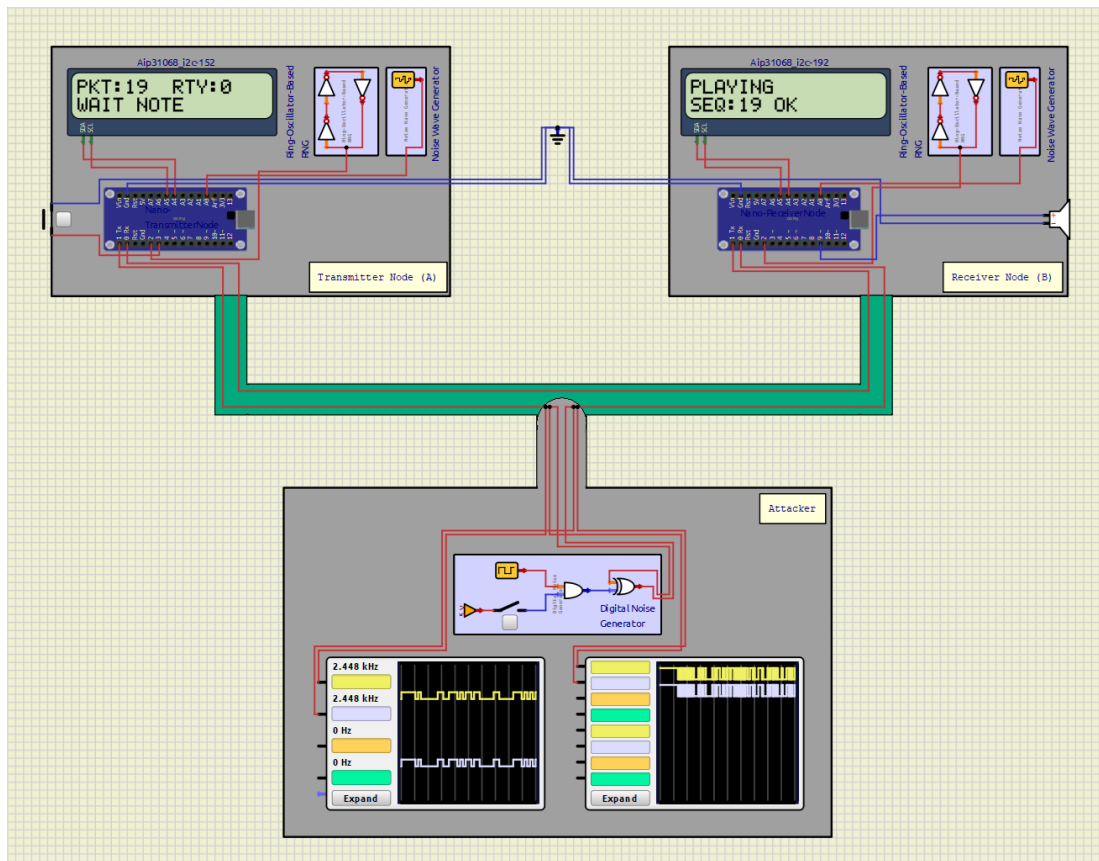


Figure 8.29: Recovery success topology. With the Digital Noise Generator deactivated, the forward channel is clear and SH2SC-EDT resumes normal C2P-ARQ operation.

LCD вузлів у Figure 8.30 і Figure 8.31 підтверджують, що end-to-end синхронізацію відновлено в межах щойно встановленого сеансу. Node A повідомляє, що обробляє 19-й пакет нового сеансу (PKT: 19 WAIT NOTE), тоді як Node B підтверджує автентифіковане просування до тієї самої точки (PLAYING SEQ: 19 OK). Така відповідність означає, що і handshake, і наступний DATA stream декодуються та підтверджуються узгоджено.

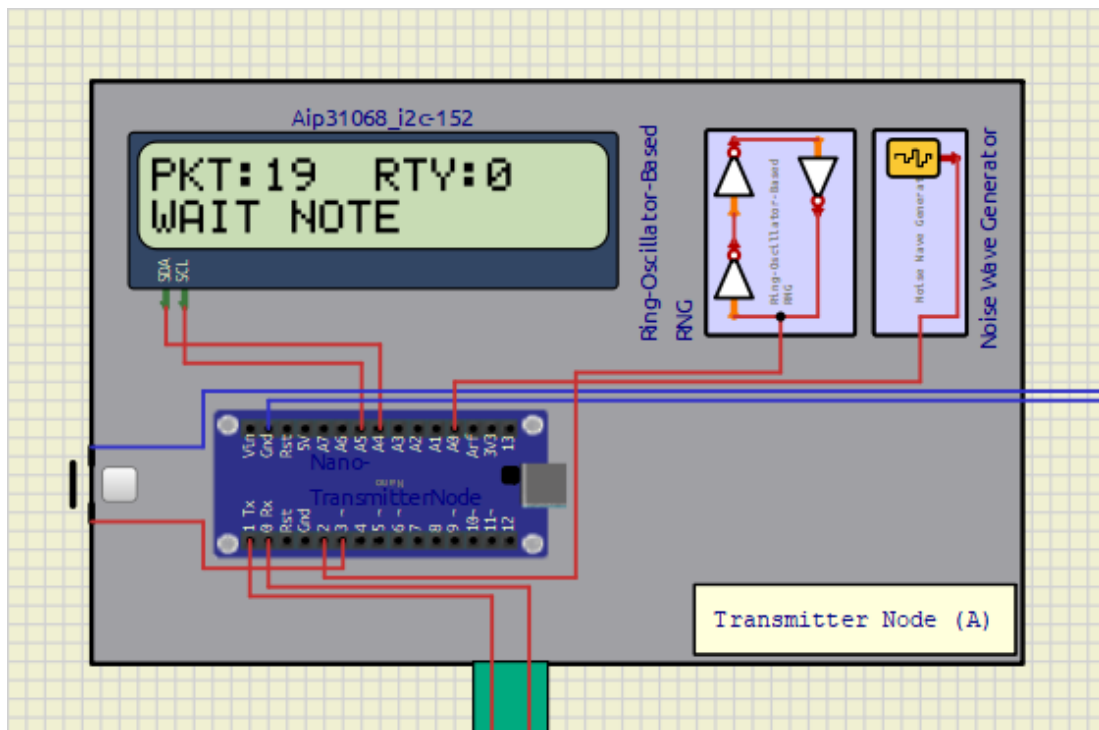


Figure 8.30: Node A status after successful recovery. PKT: 19 indicates forward progress within the new session, and WAIT NOTE indicates normal inter-packet pacing has resumed.

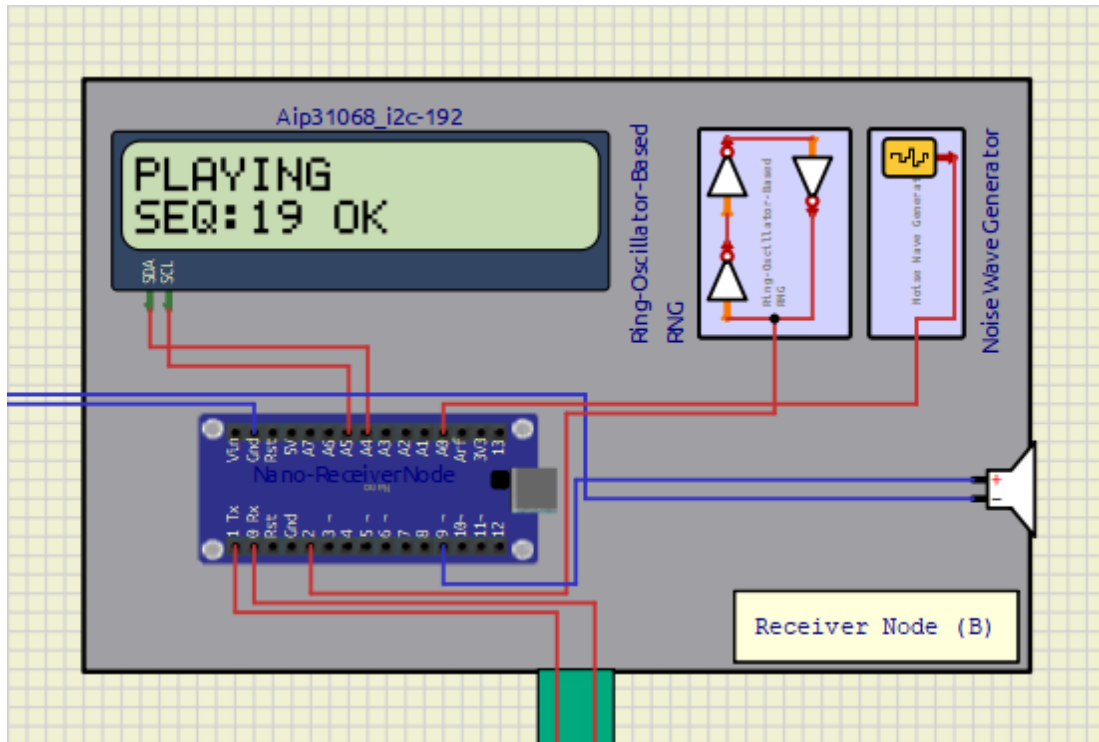


Figure 8.31: Node B status after successful recovery. PLAYING SEQ: 19 OK confirms authenticated reception and application-layer execution at the synchronized sequence position.

Це спостереження підкреслює центральний архітектурний поділ у SH2SC-EDT: Link/Session Layer навмисно є ефемерним і може бути переініціалізований, тоді як Application Layer payload stream є станозалежним і стійким. Оскільки новий криптографічний сеанс встановлюється знову, номери послідовності пакетів перезапускаються з нуля, щоб зберегти *nonce integrity* для ChaCha20-Poly1305 під новим контекстом сеансу. Однак стан payload застосунку не скидається: виконання мелодії продовжується з точно тієї ноти, на якій попередній сеанс було перервано, забезпечуючи zero data loss попри тривалу перерву.

Фізичне очищення каналу безпосередньо показано осцилографом у Figure 8.32. За XOR injection model отриманий на приймачі сигнал стає

$$S_{rx}(t) = S_{tx}(t) \oplus 0 = S_{tx}(t)$$

тому forward channel електрично прозорий за відсутності завади атакувальника.

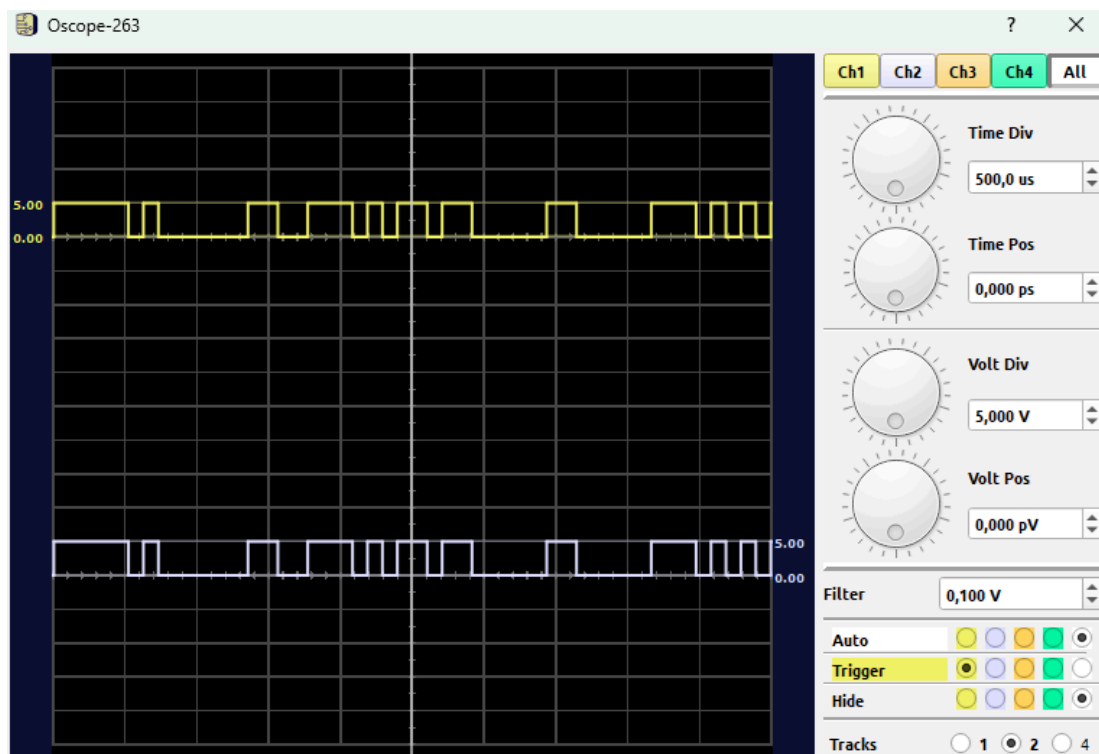


Figure 8.32: Oscilloscope evidence of recovery. With the noise path disabled, the XOR stage passes the transmitter UART waveform without inversion, confirming that the medium is free of adversarial interference.

На рівні протоколу Figure 8.33 показує чистий, безперервний data frame, сумісний із успішним framing і декодуванням. На відміну від jammed traces Сценаріїв 4 і 7, захоплений потік зберігає стабільну структуру по всьому frame, підтверджуючи, що відновлення — це не лише локальний перехід FSM, а end-to-end restoration transport exchange.

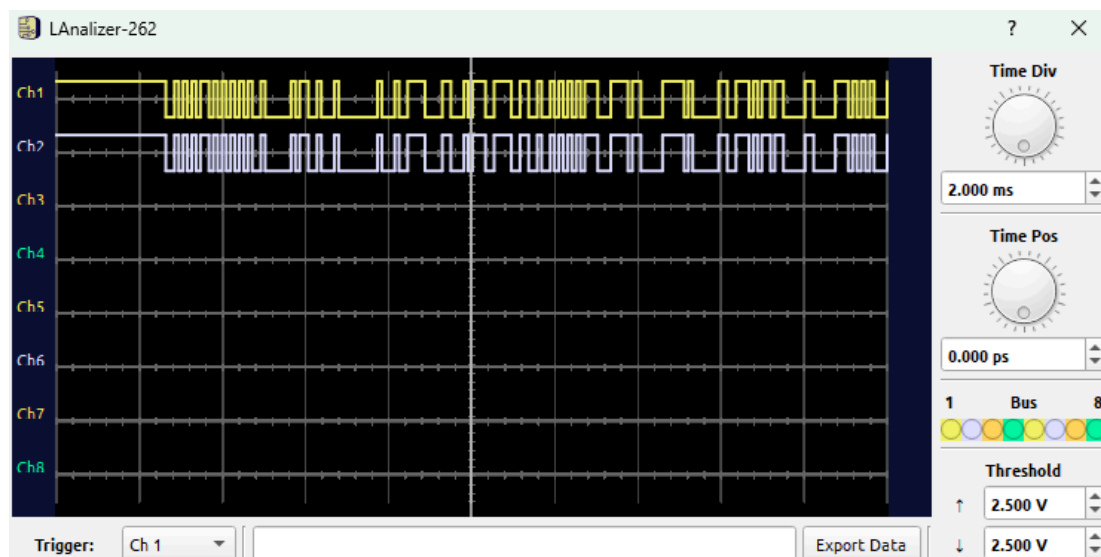


Figure 8.33: Logic-analyzer confirmation of successful protocol-level recovery. The data frame is captured without interruption, consistent with a clear channel and correct C2P-ARQ decoding.

Успішне self-healing відновлення після припинення глушіння підсумовано в Figure 8.34. Новий сеанс встановлено, а виконання payload відновлено з збереженого стану застосунку.

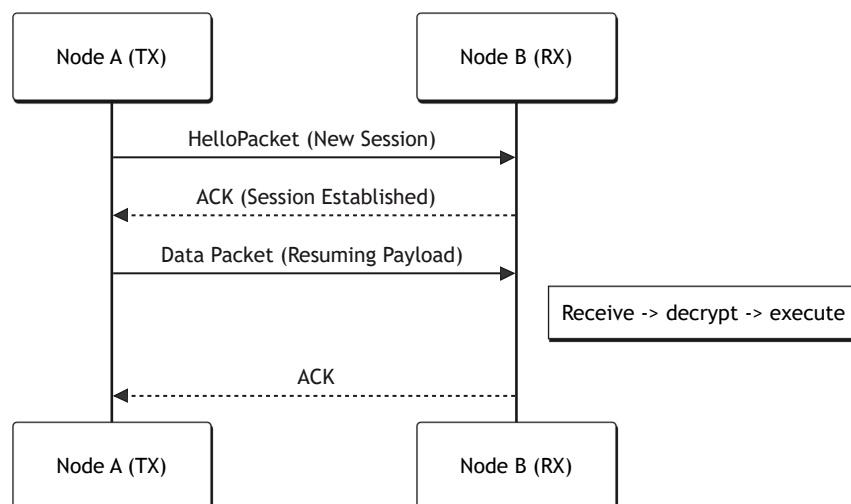


Figure 8.34: Успішне самовідновлення після припинення глушіння. Новий сеанс встановлено, а виконання payload відновлюється зі збереженого стану застосунку.

8.3.9 Сценарій 9: легітимне завершення сеансу і уникнення аналізу трафіку

У цьому фінальному сценарії передавання payload на рівні застосунку завершено повністю. Digital Noise Generator залишається вимкненим, а процес завади є спокійним,

$$N(t) = 0$$

Отже, система переходить від стійкого зашифрованого передавання даних до явного, автентифікованого шляху завершення сеансу.

Node A (TX) надсилає фінальний повнорозмірний зашифрований кадр, який несе команду завершення (FINAL flag) всередині захищеного payload. Після отримання acknowledgment від приймача Node A виходить з активного transport lifecycle і переходить у свій terminal IDLE стан. Node B (RX) отримує фінальний кадр, виконує автентифіковане дешифрування, ідентифікує умову завершення та закриває криптографічний сеанс. Під час цього Node B також записує загальну кількість виконаних payload chunks, забезпечуючи явну кінцеву точку аудиту в кінці сеансу.

Панорамний вигляд моменту закриття сеансу подано в Figure 8.35. На відміну від jammed інтервалів вище, система перебуває в спокої за задумом, а не через denial: канал тихий, і обидва кінці перейшли у terminal states, сумісні з завершеним передаванням.

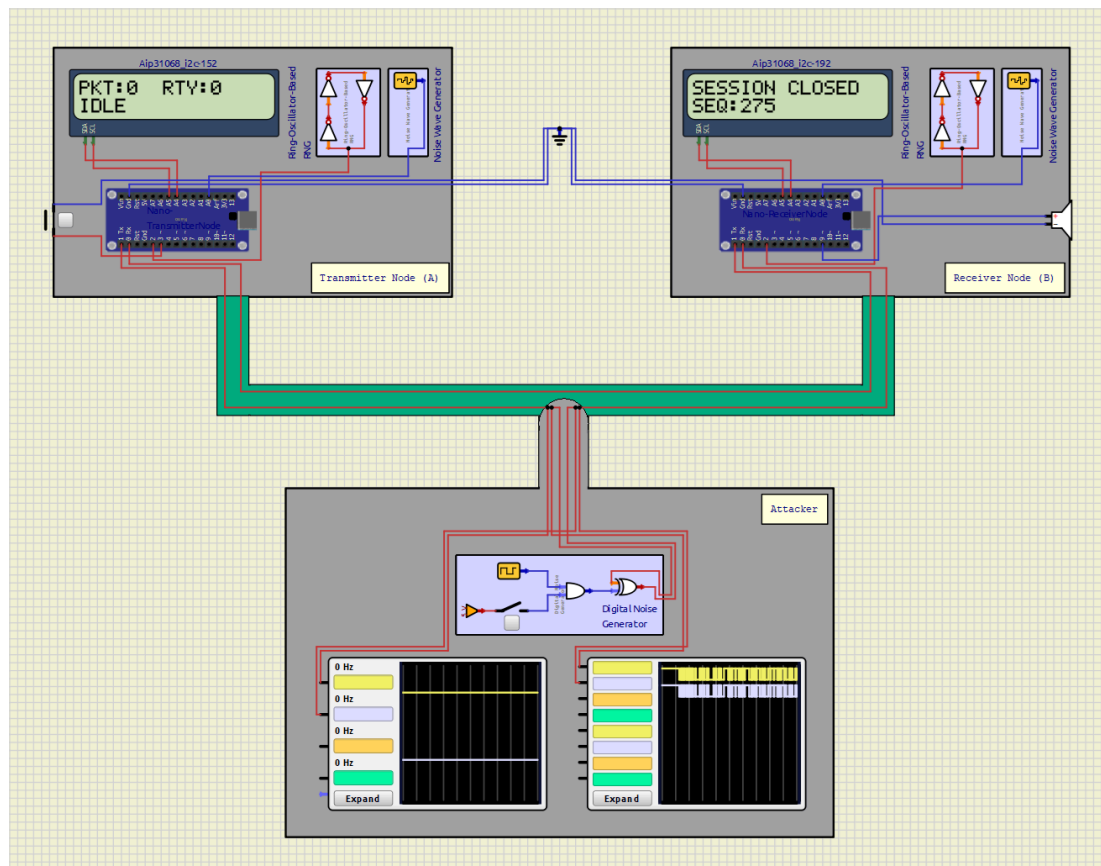


Figure 8.35: Підсумкова топологія після легітимного завершення сеансу. Атакувальник залишається вимкненим, а обидва кінці перейшли у свої постсеансові стани спокою.

Figure 8.36 і Figure 8.37 надають докази завершення на рівні вузлів. PKT: 0 IDLE на Node A вказує на те, що передавач припинив усю session activity і більше не випромінює protocol frames. SESSION CLOSED SEQ: 275 на Node B підтверджує, що приймач розпізнав команду завершення і закрив сеанс контрольованим чином, одночасно демонструючи загальне число authenticated payload chunks, виконаних під час завершеної передачі.

Транзитивний LCD-перехід передавача до post-session idle показано в Figure 8.36.

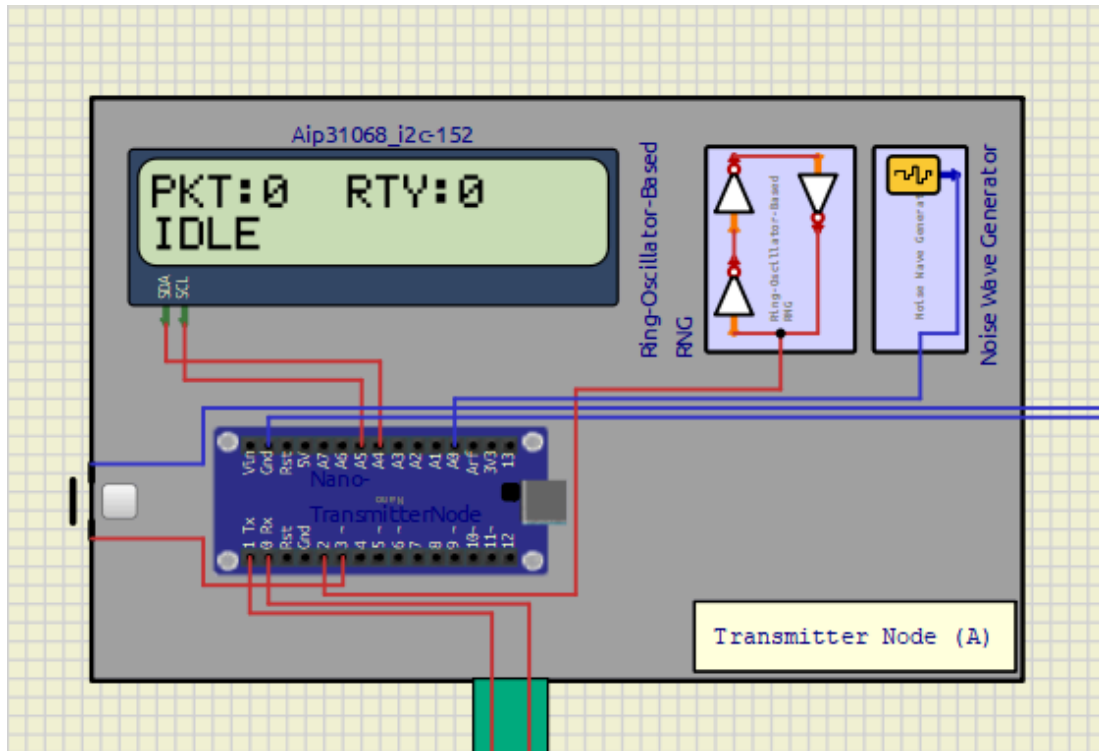


Figure 8.36: Післясеансовий стан вузла А. PKT: 0 IDLE означає, що передавач завершив останній автентифікований обмін і повернувся до свого кінцевого стану бездіяльності.

LCD-перехід приймача до post-session closed показано в Figure 8.37.

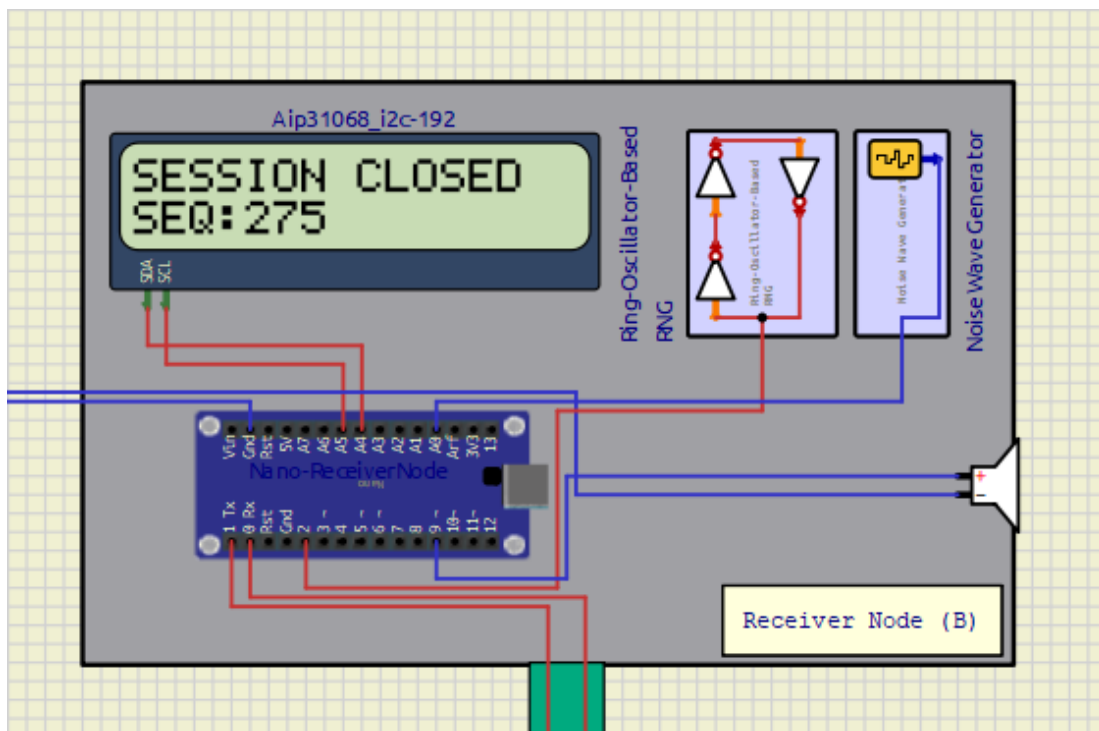


Figure 8.37: Післясеансовий стан вузла B. SESSION CLOSED SEQ: 275 підтверджує легітимне завершення та повідомляє загальну кількість автентифікованих блоків payload, виконаних під час сеансу.

Фізичну спокійність каналу після сеансу зафіксовано осцилографом у Figure 8.38. За відсутності передачі та відсутності завади від атакуювальника лінія задовольняє

$$S_{tx}(t) = 0, \quad N(t) = 0, \quad S_{rx}(t) = S_{tx}(t) \oplus N(t) = 0$$

тому приймальний вхід колапсує в flatline, сумісний із легітимним завершенням зв'язку.

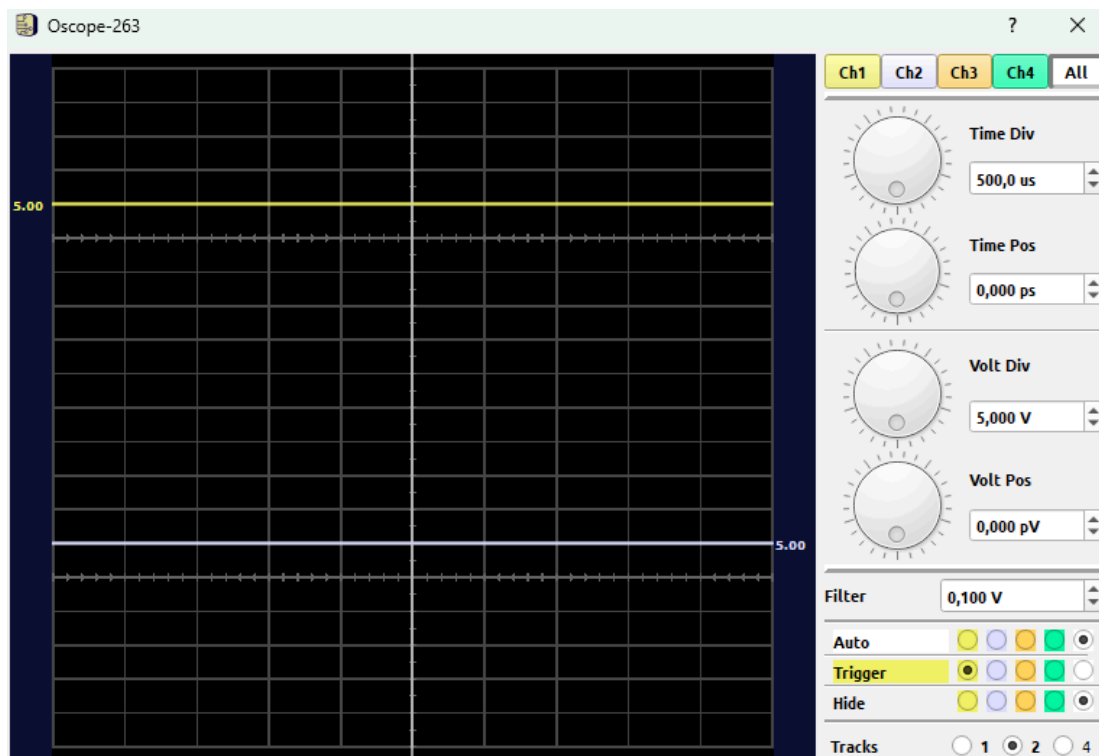


Figure 8.38: Осцилографічне підтвердження мертвого каналу після легітимного завершення. Прямая лінія узгоджується з нульовим рівнем лінії прийому за умов відсутності передачі та шуму.

Криптографічна властивість безпеки цього механізму завершення найчіткіше проявляється в logic analyzer захопленні Figure 8.39. Фінальний пакет навмисно випромінюється як повнорозмірний кадр, що структурно імітує стандартний encrypted data packet: він починається з ідентичного synchronization preamble, а його on-wire duration збігається зі звичайним footprint data frame. Такий дизайн усуває тривіальний oracle traffic analysis, заснований на евристиці розміру пакетів:

сторонній спостерігач, який бачить лише довжину кадрів, не може здогадатися про завершення сеансу лише тому, що з'явився коротший terminating frame. Натомість команда завершення кодується всередині зашифрованого payload і, отже, видима лише автентифікованому endpoint.

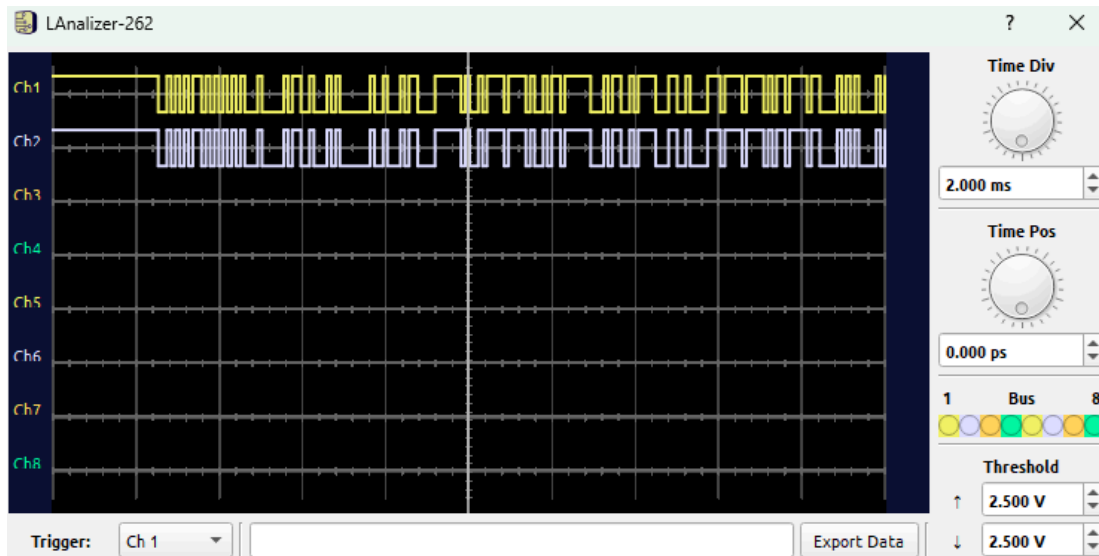


Figure 8.39: Знімок логічного аналізатора фінального завершального кадру. Кадр має повний розмір і збігається за преамбулою зі звичайним зашифрованим трафіком даних, що підтримує маскування завершення шляхом усунення інференсу за довжиною пакета.

Обмін завершення підсумовано в Figure 8.40. Термінаційний пакет передається і підтверджується так само, як у будь-якій звичайній stop-and-wait data delivery, після чого передавач входить у IDLE, а приймач — у SESSION CLOSED.

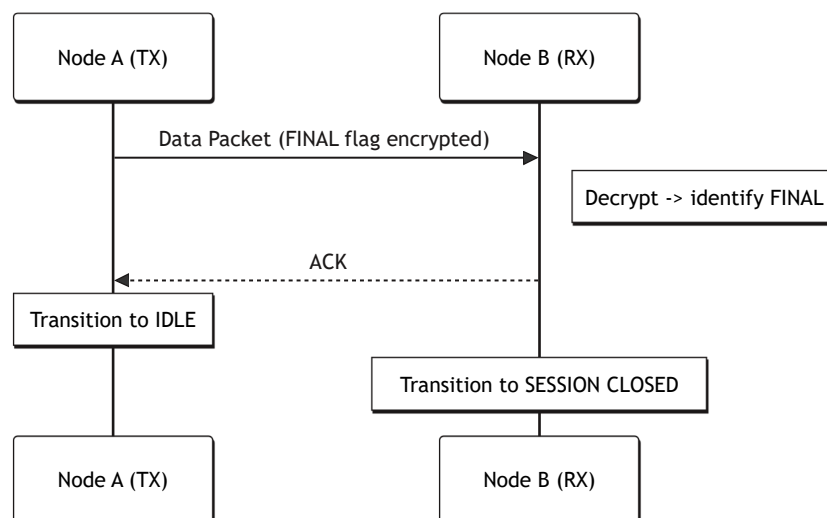


Figure 8.40: Легітимне завершення сеансу. Команду FINAL передають у повнорозмірному зашифрованому кадрі та підтверджують перед тим, як обидва кінці перейдуть у свої кінцеві післясеансові стани.

9 Висновки та подальші роботи

У цій статті представлено наскрізний дизайн, реалізацію та емпіричну валідацію апаратно-програмного комплексу SH2SC-EDT і його транспортного протоколу C2P-ARQ, який інтегрує неблокувальне керування скінченним автоматом, автентифіковане шифрування ChaCha20-Poly1305 та адверсаріально обґрунтовані експерименти з глушінням у SimulIDE в межах обмежень класу ATmega328P. Наступні підрозділи узагальнюють головні продемонстровані ефекти протоколу, а саме стійкість під час тривалої деградації каналу, автономне self-healing відновлення сеансу і криптографічні гарантії безпеки, а потім визначають наступні дослідницькі траєкторії, сфокусовані на міграції до IoT-команд для актуаторів і розширеній оптимізації живлення через deep-sleep режимами.

9.1 Підсумок ефективності протоколу

Наведені результати встановлюють, що SH2SC-EDT і його транспортне ядро C2P-ARQ досягають узгодженого синтезу криптографічної гарантії та детермінованого вбудованого виконання. На всьому протоколі було успішно інтегровано ChaCha20-Poly1305 AEAD у строго неблокувальну FSM-архітектуру Stop-and-Wait ARQ, що забезпечило конфіденційність полів payload і сувору автентифікацію кадрів на data-link рівні через authenticated binding метаданих керування та ciphertext. Емпіричні trace також підтверджують, що прогрес протоколу визначається обмеженими переходами на базі millis(), а не блокувальними шляхами керування, зберігаючи детермінованість таймінгу під обмеженнями ресурсів класу AVR.

Дослідження також перевіряє центральну властивість Self-Healing під час тривалої фізичної деградації каналу. [Section 8](#) демонструє, що під час тривалого глушіння система автономно виявляє невідновлювані умови каналу через обмежену кількість повторів і логіку тайм-аутів, виконує безпечне завершення криптографічного сеансу зі стиранням nonce та переходить у контрольовану reconnection-поведінку без втручання оператора. Коли цілісність каналу відновлюється, повторне встановлення сеансу відбувається безпечно через freshly generated TRNG-seeded session material, підтверджуючи, що відновлення є одночасно автономним і криптографічно коректним.

Нарешті, експерименти підтверджують, що SH2SC-EDT відокремлює безперервність сеансу від прогресу застосунку, що є ключовою умовою відмовостійкого доставлення без семантичних втрат. Хоча transport context навмисно повторно ініціалізується після порушення, application state зберігається, що дає змогу точно відновлювати Auto-Resume з перерваної позиції payload і тим самим демонструє zero data loss під час простоїв. Крім того, фінальний повнорозмірний автентифікований пакет завершення зберігає структурну невідрізнимість на дроті від звичайного зашифрованого трафіку, зменшуючи тривіальне traffic-analysis інферування завершення сеансу за пакетними довжинами.

9.2 Міграція до команд керування актуаторами

Хоча нинішня експериментальна кампанія використовувала аудіопayload (послідовну мелодію) як proof-of-concept для станозалежного, часово залежного виконання, найближчою дослідницькою траєкторією є міграція прикладного рівня до прямого керування фізичними IoT-актуаторами. На практиці це охоплює профілі команд для сервоприводів, промислових реле, smart lock, ланцюгів роботизованої кінематики та SCADA-підключених endpoint, де кожен автентифікований пакет відповідає матеріально значущій керувальній дії, а не акустичній події. Така міграція зберігає протокольне ядро, водночас піднімаючи домен валідації від демонстраційного відтворення сигналу до safety-critical cyber-physical actuation.

Цей перехід технічно обґрунтований семантикою виконання, уже продемонстрованою C2P-ARQ. Робочі процеси фізичних актуаторів за своєю природою є послідовними та часто незворотними на проміжних кроках; отже, вони потребують детермінованого доставлення команд із суворим порядком і без мовчазних пропусків. Обмежена поведінка ARQ, прагнення до zero-loss delivery і здатність Auto-Resume разом забезпечують гарантії безперервності, потрібні для багатокрокових механічних процедур, що мають завершуватися точно відповідно до специфікації, навіть якщо суворе глушіння перериває зв'язок посеред операції. У результаті порушення сеансу перетворюється з фатальної помилки процесу на відновлювану транспортну подію без втрати прогресу на рівні застосунку.

З погляду кіберфізичної безпеки розширення платформи до керування актуаторами ще більше підсилює значення автентифікованого шифрування. ChaCha20-Poly1305 у стеку C2P-ARQ надає криптографічне відхилення повторно відтворених кадрів і спроб зловмисної ін'єкції payload, які в актуатор-орієнтованих розгортаннях могли б інакше спричинити небезпечні переходи стану, механічне

напруження обладнання або катастрофічні фізичні наслідки. Отже, запланована міграція — це не зміна припущень протоколу, а пряме застосування валідаваних властивостей SH2SC-EDT щодо безпеки та стійкості у вищих за впливом operational contexts CPS.

9.3 Глибокий сон і оптимізація живлення

Залишковим системним обмеженням, типовим для deterministic polling архітектур, є постійна енергетична вартість безперервного спостереження за каналом і безперервної оцінки finite-state ARQ. У поточній реалізації SH2SC-EDT цей вибір є методологічно доречним для перевірки часової прозорості та відмовостійкості, однак він безпосередньо обмежує розгортання у battery-operated та energy-harvesting IoT-середовищах, де довготривала автономність визначається бюджетом idle-power, а не піковою обчислювальною потребою.

Негайною дослідницькою траєкторією є, отже, протокольно обґрунтована інтеграція апаратних режимів Deep Sleep у життєвий цикл C2P-ARQ. Зокрема, вузли повинні переходити в ultra-low-power стани під час інтервалів IDLE та під час тривалих очікувань на рівні застосунку, зокрема міжзонних затримок виконання payload, які не потребують активної участі CPU. Така інтеграція має зберегти встановлені не блокувальні семантики станів, трактуючи вхід у sleep і wake-up як явні переходи FSM, тим самим зберігаючи детерміновану поведінку протоколу при мінімізації duty cycle у спокійні періоди.

Щоб узгодити агресивне зниження споживання з надійною безперервністю сеансу, пробудження слід делегувати апаратним асинхронним перериванням, наприклад pin-change triggers, згенерованим першим спостережуваним фронтом preamble вхідного кадру. Така модель reactivate-first interrupt дозволила б мікроконтролеру залишатися електрично сплячим доти, доки не з'явиться справжня активність каналу, а потім відновлювати автентифіковану обробку C2P-ARQ із обмеженою затримкою. Поєднання продемонстрованої криптографічної стійкості C2P-ARQ з такою агресивною економією енергії, ймовірно, дасть повністю автономний, високо безпечний та надзвичайно енергоефективний комплекс SH2SC-EDT, придатний для довготривалого польового розгортання.

References

1. Andrea, I., Chrysostomou, C., Hadjichristofi, G.: Internet of Things: Security Vulnerabilities and Challenges. In: IEEE Symposium on Computers and Communication (ISCC) (2015).
2. Al-Zubaidie, M., others: A Comprehensive Analysis of Code Injection Attacks in IoT. IEEE Access. (2020).
3. Smith, R.H., others: Vulnerabilities of COTS RF Modules: Replay Attacks on Accessibility Technology. In: Research Symposium (2022).
4. Deogirikar, J., Vidoushi, A.: Security Attacks in IoT: A Survey. In: International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud) (2017).
5. Neely, M., others: Security Analysis of Sub-GHz Radio Communications and Replay Attack Methodology. In: Australian Information Security Management Conference (2013).
6. Shen, Y., others: Task-Specific Vulnerability Analysis with Case-Driven Insights in RF Sensing. arXiv preprint. (2024).
7. Trend Micro ICS-CERT: A Security Analysis of Radio Remote Controllers: Command Injection and Replay Vulnerabilities. Trend Micro (2018).
8. Dolev, D., Yao, A.C.: On the Security of Public Key Protocols. IEEE Transactions on Information Theory. 29, 198–208 (1983).
9. Backes, M., Pfizmann, B., Waidner, M.: Soundness Limits of Dolev-Yao Models. In: Workshop on Formal and Computational Cryptography (FCC) (2006).
10. Vigano, L., others: Formalization of the Dolev-Yao adversary in Protocol Analysis. In: AAMAS (2006).
11. Smyth, B.: Formal Verification of Cryptographic Protocols with Automated Tools, (2011).
12. Cremers, C.: Formal Verification of Cryptographic Protocols: ProVerif and Scyther. IJCN. (2010).
13. Dupressoir, F., others: Formal Verification of Protocols using General-Purpose Tools, (2014).
14. Lewandowski, S.M.: Frameworks for Component-Based Client/Server Computing (Fat vs Thin Clients). ACM Computing Surveys. 30, (1998).

15. Negris, T., others: Thin Client Technology and User Acceptance. TUDublin School of Computing (2009).
16. IMM DTU: Client-Server Architectural Patterns: Thin or Thick? DTU Compute (2017).
17. Dash, A., Dasgupta, S.: Event-Driven Programming for Embedded Systems using FSM. Semantic Scholar. (2015).
18. Lee, E.A.: CFSM and Process Networks in Embedded Systems, (2003).
19. Billington, J., others: Formal Verification of Stop-and-Wait ARQ Using Petri Nets. In: ACM CCS (2005).
20. Berthomieu, B., others: Modeling and Verification of Embedded Real-Time Systems using Constraint Time Petri Nets. Mathematics / MDPI. (2020).
21. Shannon, C.: Entropy Harvesting in Silicon: TRNGs and PUFs. Information Security. (1948).
22. Fischer, V., others: Thermal Jitter from Free-Running Ring Oscillator in TRNGs. IEEE Transactions on Computers. (2021).
23. Tuncer, T., others: True Random Number Generator based on Ring Oscillators. IJRASET. (2023).
24. Wang, Y., others: Design of True Random Number Generator Based on Multi-stage Feedback Ring Oscillator. ResearchGate. (2024).
25. Kim, J., others: SRAM Data Remanence based TRNG/PUF stability for Key Generation. In: ISLPED ACM/IEEE (2017).
26. Turan, M.S., others: NIST SP 800-90B: Recommendation for the Entropy Sources Used for Random Bit Generation. NIST Special Publication (2018).
27. Barker, E., Kelsey, J.: NIST SP 800-90A Rev 1: Recommendation for Random Number Generation Using Deterministic Random Bit Generators. NIST (2015).
28. Bertoni, G., Daemen, J., Peeters, M., Van Assche, G.: On the Indifferentiability of the Sponge Construction. In: EUROCRYPT 2008 / Springer LNCS (2008).
29. Keccak Team: The Sponge and Duplex Constructions, (2011).
30. Melo, V.: Sponge construction in lightweight cryptography (SymFrog-512). IACR ePrint. (2024).

31. Dobraunig, C., others: Ascon Specification (Sponge-based Lightweight Cryptography). NIST LWC Finalist (2019).
32. Aumasson, J.-P., others: ChaCha20-based CSPRNG for Embedded Systems. Cryptology ePrint Archive. (2020).
33. OpenBSD / Wikipedia: Cryptographically Secure Pseudorandom Number Generator: arc4random and ChaCha20, (2013).
34. Rogaway, P.: Authenticated-Encryption with Associated-Data. In: ACM Conference on Computer and Communications Security (CCS) (2002).
35. Bozhko, V.: RFC 9771: Properties of Authenticated Encryption with Associated Data (AEAD) Algorithms. IRTF/IETF (2025).
36. Bernstein, D.J.: ChaCha, a variant of Salsa20. In: SASC 2008 (2008).
37. Bernstein, D.J.: The Poly1305-AES Message-Authentication Code. In: Fast Software Encryption (FSE) (2005).
38. Nir, Y., Langley, A.: RFC 7539: ChaCha20 and Poly1305 for IETF Protocols. IRTF (2015).
39. Nir, Y., Langley, A.: RFC 8439: ChaCha20 and Poly1305 for IETF Protocols. IETF (2018).
40. Housley, R.: RFC 8103: Using AEAD_CHACHA20_POLY1305 with CMS. IETF (2017).
41. Procter, G.: A Security Analysis of the Composition of ChaCha20 and Poly1305. Cryptology ePrint Archive, Paper 2014/613. (2014).
42. Coutinho, M., others: Differential and linear cryptanalysis of ChaCha. In: SBSEg / SBC (2020).
43. Joux, A.: Authentication Failures in NIST GCM (Nonce reuse in Stream Ciphers). In: NIST Block Cipher Techniques (2006).
44. Gueron, S., others: IETF length-encoding block and nonce reuse hazard (AES-GCM-SIV). RFC 8452 (2018).
45. Langley, A., others: POLYVAL and Length Block Encoding in AEAD. IETF Draft (2015).
46. Planck Collaboration: Planck 2018 results. VI. Cosmological parameters. *Astronomy & Astrophysics (A&A)*. 641, A6 (2020).

47. NASA WMAP Science Team: Universe Age Estimates based on WMAP, (2012).
48. ESA/NASA: Planck reveals an almost perfect Universe: Age estimates, (2013).
49. Chen, S.: Simplex Stop and Wait with ARQ. *Computer Networks*. (2003).
50. Fairhurst, G., Wood, L.: RFC 3366: Advice to Link Designers on Link ARQ. IETF Best Current Practice (2002).
51. Al-Jubari, A.: Performance of ARQ protocols over unreliable channels. *Semantic Scholar*. (2010).
52. De Vuyst, S., others: Performance Analysis of Stop-and-Wait ARQ for Wireless Channels with Bursty Errors. In: *HET-NETs* (2004).
53. Kumar, A., others: UART module using Verilog HDL (PISO/SIPO). *International Journal of Advanced Research*. (2025).
54. Sharma, P., others: UART Architecture with Built-in FIFO Buffering Feature. *VLSI Design*. (2021).
55. Shao, e.al.: Defending Embedded Systems Against Buffer Overflow. In: *Princeton University / ACM* (2004).
56. Massey, J.L.: Optimum Frame Synchronization. *IEEE Transactions on Communications*. 20, 115–119 (1972).
57. Recayte, E., Munari, A.: Packet detection and sync-word correlation. *Elsevier*. (2023).
58. Halderman, J.A., others: Lest We Remember: Cold Boot Attacks on Encryption Keys. In: *17th USENIX Security Symposium* (2008).
59. Yang, Z., others: Dead Store Elimination (Still) Considered Harmful. In: *26th USENIX Security Symposium* (2017).
60. Huang, Y., Kintala, C.M.R.: Software Rejuvenation: Analysis, Module and Applications. In: *25th International Symposium on Fault-Tolerant Computing (FTCS-25)* (1995).